M3 File Format Specification

Format: Blizzard M3 Model Format (StarCraft II / Heroes of the Storm) **Byte Order**: Little-endian **Alignment**: All chunks are 16-byte aligned (padded with `0xAA`) **Header Magic**: `MD34`

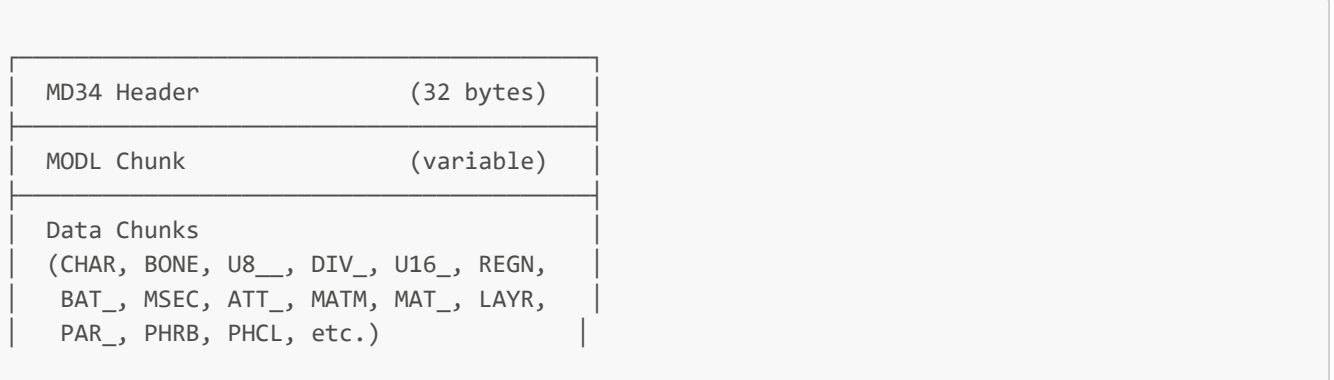
This document describes the **MD34** revision of the M3 format — the release-build format used by StarCraft II (from launch onward) and Heroes of the Storm. An earlier beta revision (MD33) is covered in Appendix C.

Authorship attribution: Fernando A. Sahmkow

Table of Contents

---

1. File Structure Overview



```
├──────────────────────────────────────────┤
│  Index Table          (N × 16 bytes)     │
│  (at offset specified in header)         │
└──────────────────────────────────────────┘
```

The M3 format is chunk-based. Each chunk is referenced through a central **Index Table** located at the end of the file. The header provides the offset to this table and a reference to the MODL root chunk, which in turn contains references to all other data.

Reading order:

1. Read the 32-byte header at offset 0 (magic = `MD34`)
2. Seek to `header.indexOffset` and read `header.indexCount` index entries
3. Locate the MODL chunk via the header's model reference
4. MODL contains `Ref<T>` fields pointing to every sub-chunk

---

2. Primitive Types

```
// Scalar types
using u8  = uint8_t;
using u16 = uint16_t;
using u32 = uint32_t;
using u64 = uint64_t;
using i8  = int8_t;
using i16 = int16_t;
using i32 = int32_t;
using f32 = float;

// Vector types
struct Vector2f { f32 x, y; };
struct Vector3f { f32 x, y, z; };
struct Vector4f { f32 x, y, z, w; };
struct Quat     { f32 x, y, z, w; };
struct ColorBGRA { u8 b, g, r, a; };
struct ColorBGR  { u8 b, g, r; };
struct Matrix44f { f32 m[4][4]; };                  // row-major

// Bounding extents
struct Extent {
    Vector3f min;       // AABB minimum
    Vector3f max;       // AABB maximum
    f32      radius;    // Bounding sphere radius
};
```

Tag String Encoding

Tag names (FourCC) are stored as 4 ASCII bytes in **reverse order** when read as a little-endian uint32. The bytes `4C 44 4F 4D` decode to `"LDOM"` raw, which reversed gives `"MODL"`. Always reverse the 4-byte string when reading.

---

3. Core Structures

3.1 Ref<T> (12 bytes)

Typed references are the primary mechanism for linking chunks together. A `Ref<T>` points into the Index Table to locate target data of type `T`, where `T` is the chunk type (identified by its FourCC tag) that the reference resolves to.

```
template<typename T>
struct Ref {
    u32 entries;            // Number of T elements at the target
    u32 index;              // 0-based index into the Index Table
    u32 flags;              // Tool metadata (see Appendix G); safe to ignore
};
```

When `entries > 0`, seek to `indexTable[index].offset` and read `entries` items of type `T`. A `Ref<T>` with `entries == 0` is null.

> **Index values**: `indexTable[0]` is always the header entry itself. The MODL chunk typically occupies `indexTable[1]`. A null reference has `entries == 0` (the `index` value is ignored when entries is zero).

> **`Ref<?>`**: A few fields use `Ref<?>` to indicate that the target chunk type is unknown. These references have never been observed as non-null in the entire SC2/HotS corpus (~56,000 files), nor are they documented in any known tooling, so their intended type cannot be determined.

> **Flags field**: The `flags` field does not affect model geometry, animation, or rendering. It appears to be tool/pipeline metadata written by the Blizzard export toolchain — 93.5% of all Refs in the corpus have `flags == 0`. Parsers should read it for round-trip fidelity but may safely ignore its value. See **Appendix G** for the full corpus analysis.

3.2 AnimRef<T> (variable size)

Animatable references hold a default value and a link to animation data. The total size depends on the contained type `T`.

```
template<typename T>
struct AnimRef {
    u16 interpType;         // Interpolation (0=none, 1=linear, 2=hermite, 3=bezier)
    u16 flags;              // Animation flags
    u32 animId;             // Animation identifier (links to STC animation data)
    T   initValue;          // Initial/default value
    T   nullValue;          // Null/reset value
    i32 unused;             // Typically -1
};
```

Common AnimRef sizes:

Type	sizeof(T)	Total size
`AnimRef<f32>`	4	20 bytes
`AnimRef<u16>`	2	16 bytes
`AnimRef<u32>`	4	20 bytes
`AnimRef<Vector2f>`	8	28 bytes
`AnimRef<Vector3f>`	12	36 bytes
`AnimRef<Quat>`	16	44 bytes

Type	sizeof(T)	Total size
AnimRef<ColorBGRA>	4	20 bytes
AnimRef<Extent>	28	68 bytes

3.3 Flag (4 bytes)

```
struct Flag {
    u32 value;
    bool get(int bit) const { return (value >> bit) & 1; }
};
```

3.4 Version-Gated Fields

Many chunk types have multiple versions that add or remove fields. Later versions typically append new fields or insert them at specific points, changing the total struct size. In the pseudocode throughout this document, version-dependent fields are shown with `if (version >= N)` guards.

When a version gate is not met, that field is absent from the binary layout and all subsequent fields shift down by the size of the omitted field. Parsers must read fields sequentially, skipping gated fields that do not apply to the chunk version being parsed. See §17 for the definitive version-to-size mapping for every chunk type.

---

4. File Header

Tag: MD34 | **Size**: 32 bytes | **Index entry version**: 11

```
struct MD34Header {
    char      magic[4];         // "MD34" (LE uint32 0x4D443334)
    u32       indexOffset;      // Byte offset to the Index Table
    u32       indexCount;       // Number of IndexEntry structs
    Ref<MODL> modelRef;         // Reference to MODL chunk
};
```

The `modelRef` is a standard 12-byte `Ref<MODL>`. In practice, `modelRef.index = 1` (pointing to `indexTable[1]`, since `indexTable[0]` is the header's own entry).

---

5. Index Table

Located at `header.indexOffset`. Contains `header.indexCount` entries, each 16 bytes:

```
struct IndexEntry {
    char tag[4];                // FourCC tag (reversed, see §2)
    u32  offset;                // Byte offset to chunk data
    u32  count;                 // Number of items in this chunk
    u32  version;               // Struct version for this chunk type
};
```

The `version` field determines the struct layout for that chunk type. Multiple entries can share the same tag (e.g., many `CHAR` entries for different strings, many `LAYR` entries for different layers).

Chunk data size: `indexTable[i].count × chunkSize(tag, version)` — see §17.

---

6. Model Root

Tag: `MODL` | **Versions**: 23 (0x17), 24 (0x18), 25 (0x19), 26 (0x1A), 28 (0x1C), 29 (0x1D), 30 (0x1E)

The MODL chunk is the root of all model data. It contains `Ref<T>` fields pointing to every other chunk type.

```
struct MODL {
    // ── Core ──────────────────────────────────────────────────────
    Ref<CHAR>       name;                           // model file path
    u32             flags;                          // Model flags (see below)
    Ref<SEQS>       sequences;
    Ref<STC_>       subTrackCollections;
    Ref<STG_>       animationGroups;
    Ref<BSET>       boneAnimationSets;              // always null
    u32             animationSplitCount;            // Always 0
    Ref<STS_>       animationStates;
    Ref<BONE>       bones;
    u32             skinBoneCount;                  // Bones affecting skin
    u32             vertexFlags;                    // Vertex format bits (§9.2)
    Ref<U8__>       vertices;                       // vertex blob
    Ref<DIV_>       divisions;
    Ref<U16_>       boneLookup;
    Extent          bounds;                         // Model bounding volume
    Extent          collisionBounds;                // Collision bounding volume
    Ref<U16_>       collisionFaces;
    Ref<VEC3>       collisionVerts;
    Ref<VEC3>       collisionNormals;

    // ── Scene objects ─────────────────────────────────────────────
    Ref<ATT_>   attachmentPoints;
    Ref<U16_>   attachmentPointAddons;
    Ref<LITE>   lights;
    Ref<SHBX>   shadowBoxes;
    Ref<CAM_>   cameras;
    Ref<U16_>   camerasAddons;

    // ── Materials ─────────────────────────────────────────────────
    Ref<MATM>   materialMaps;
    Ref<MAT_>   standardMaterials;
    Ref<DIS_>   displacementMaterials;
    Ref<CMP_>   compositeMaterials;
    Ref<TER_>   terrainMaterials;
    Ref<VOL_>   volumeMaterials;
    Ref<HAI_>   hairMaterials;                      // defunct — always null
    Ref<CREP>   creepMaterials;
    if (version >= 25) {
        Ref<VON_>   volumeNoiseMaterials;
    }
    if (version >= 26) {
        Ref<STBM>   stbMaterials;
    }
    if (version >= 28) {
```

```
        Ref<REF_>   reflectionMaterials;
    }
    if (version >= 29) {
        Ref<LFLR>   lensFlareMaterials;
    }
    if (version >= 30) {
        Ref<MADD>   dataDrivenMaterials;
    }

    // ── Effects ──────────────────────────────────────────
    Ref<PAR_>   particleEmitters;
    Ref<PARC>   particleEmitterCopies;
    Ref<RIB_>   ribbonEmitters;
    Ref<PROJ>   projections;

    // ── Physics ──────────────────────────────────────────
    Ref<FOR_>   forces;
    Ref<WRP_>   warps;
    Ref<VVOL>   viewVolumes;
    Ref<PHRB>   rigidBodies;
    Ref<PHCT>   physicsConstraints;
    Ref<PHYJ>   physicsJoints;
    if (version >= 28) {
        Ref<PHCL>   clothPhysics;
    }
    Ref<IK2J>   ikTwoJoints;
    if (version >= 24) {
        Ref<IKCC>   ikCCD;
    }
    Ref<IKJT>   ikJoints;
    Ref<PAOB>   oneBoneSolvers;

    // ── Behaviors & data ─────────────────────────────────
    Ref<PATU>   turretBehaviors;
    Ref<TRGD>   triggerData;
    Ref<IREF>   initialReference;                   // inverse bind-pose

    // ── Inline + trailing data ───────────────────────────
    SSGS        tightHitTest;                       // Tight hit-test shape
    Ref<SSGS>   fuzzyHitTestObjects;
    Ref<ATVL>   attachmentVolumes;
    Ref<U16_>   attachmentVolumesAddon0;
    Ref<U16_>   attachmentVolumesAddon1;
    Ref<BBSC>   billboardBehaviors;
    Ref<TMD_>   trailingModels;                     // defunct
    u32         m3aAnimHash;                        // Hash for .m3a animation file
binding
    Ref<U32_>   m3aAnimHashes;                      // array of .m3a hashes
};
```

MODL Flags (bitmasks):

Mask	Name	Description
0x00001	tangents	Tangents computed
0x00002	bonesFixed	Bone transforms fixed
0x00004	uvDensitiesComputed	UV densities computed

Mask	Name	Description
0x00008	relativeBounds	Uses relative bounds
0x00010	sectionBoundsFixed	Section bounds fixed
0x00020	trackSetsComputed	Track sets computed
0x00040	trackCollectionSorted	Track collection sorted
0x00080	acceptsSplats	Model accepts splats
0x00800	trackAnimatedBaseFlagValid	Animated base flag valid
0x01000	fileDirty	File marked dirty
0x04000	fowDoNotUseTint	FOW: do not tint
0x08000	instancedVB	Uses instanced vertex buffer
0x10000	forceSampledFOW	Force sampled FOW
0x20000	instancedModel	Instanced model
0x40000	neverUseFOW	Never use FOW
0x80000	boneAnimatedFlagSolved	Bone animated flags solved
0x100000	allowLocalLightShadows	Allow local light shadows
0x200000	avoidSampledFOW	Avoid sampled FOW

6.1 MODL Version Offset Map

Byte offsets from MODL start for each field across all MD34 versions. Dashes indicate the field does not exist in that version.

Field	v23	v24	v25	v26	v28	v29	v30
attachmentPoints	0x0E4	0x0E4	0x0E4	0x0E4	0x0E4	0x0E4	0x0E4
attachmentPointAddons	0x0F0	0x0F0	0x0F0	0x0F0	0x0F0	0x0F0	0x0F0
lights	0x0FC	0x0FC	0x0FC	0x0FC	0x0FC	0x0FC	0x0FC
shadowBoxes	0x108	0x108	0x108	0x108	0x108	0x108	0x108
cameras	0x114	0x114	0x114	0x114	0x114	0x114	0x114
camerasAddons	0x120	0x120	0x120	0x120	0x120	0x120	0x120
materialMaps	0x12C	0x12C	0x12C	0x12C	0x12C	0x12C	0x12C
standardMaterials	0x138	0x138	0x138	0x138	0x138	0x138	0x138
displacementMaterials	0x144	0x144	0x144	0x144	0x144	0x144	0x144
compositeMaterials	0x150	0x150	0x150	0x150	0x150	0x150	0x150
terrainMaterials	0x15C	0x15C	0x15C	0x15C	0x15C	0x15C	0x15C
volumeMaterials	0x168	0x168	0x168	0x168	0x168	0x168	0x168
hairMaterials	0x174	0x174	0x174	0x174	0x174	0x174	0x174
creepMaterials	0x180	0x180	0x180	0x180	0x180	0x180	0x180

Field	v23	v24	v25	v26	v28	v29	v30
volumeNoiseMaterials	—	—	0x18C	0x18C	0x18C	0x18C	0x18C
stbMaterials	—	—	—	0x198	0x198	0x198	0x198
reflectionMaterials	—	—	—	—	0x1A4	0x1A4	0x1A4
lensFlareMaterials	—	—	—	—	—	0x1B0	0x1B0
materialAddData	—	—	—	—	—	—	0x1BC
particleEmitters	0x18C	0x18C	0x198	0x1A4	0x1B0	0x1BC	0x1C8
particleEmitterCopies	0x198	0x198	0x1A4	0x1B0	0x1BC	0x1C8	0x1D4
ribbonEmitters	0x1A4	0x1A4	0x1B0	0x1BC	0x1C8	0x1D4	0x1E0
projections	0x1B0	0x1B0	0x1BC	0x1C8	0x1D4	0x1E0	0x1EC
forces	0x1BC	0x1BC	0x1C8	0x1D4	0x1E0	0x1EC	0x1F8
warps	0x1C8	0x1C8	0x1D4	0x1E0	0x1EC	0x1F8	0x204
viewVolumes	0x1D4	0x1D4	0x1E0	0x1EC	0x1F8	0x204	0x210
rigidBodies	0x1E0	0x1E0	0x1EC	0x1F8	0x204	0x210	0x21C
physicsConstraints	0x1EC	0x1EC	0x1F8	0x204	0x210	0x21C	0x228
physicsJoints	0x1F8	0x1F8	0x204	0x210	0x21C	0x228	0x234
clothPhysics	—	—	—	—	0x228	0x234	0x240
ikTwoJoints	0x204	0x204	0x210	0x21C	0x234	0x240	0x24C
ikCCD	—	0x210	0x21C	0x228	0x240	0x24C	0x258
ikJoints	0x210	0x21C	0x228	0x234	0x24C	0x258	0x264
oneBoneSolvers	0x21C	0x228	0x234	0x240	0x258	0x264	0x270
turretBehaviors	0x228	0x234	0x240	0x24C	0x264	0x270	0x27C
triggerData	0x234	0x240	0x24C	0x258	0x270	0x27C	0x288
initialReference	0x240	0x24C	0x258	0x264	0x27C	0x288	0x294
(inline hit-test)	0x24C	0x258	0x264	0x270	0x288	0x294	0x2A0
fuzzyHitTestObjects	0x2B8	0x2C4	0x2D0	0x2DC	0x2F4	0x300	0x30C
attachmentVolumes	0x2C4	0x2D0	0x2DC	0x2E8	0x300	0x30C	0x318
attachmentVolumesAddon0	0x2D0	0x2DC	0x2E8	0x2F4	0x30C	0x318	0x324
attachmentVolumesAddon1	0x2DC	0x2E8	0x2F4	0x300	0x318	0x324	0x330
billboardBehaviors	0x2E8	0x2F4	0x300	0x30C	0x324	0x330	0x33C
trailingModels	0x2F4	0x300	0x30C	0x318	0x330	0x33C	0x348
m3aAnimHash	0x300	0x30C	0x318	0x324	0x33C	0x348	0x354
m3aAnimHashes	0x304	0x310	0x31C	0x328	0x340	0x34C	0x358

MODL version sizes: v23=784, v24=796, v25=808, v26=820, v28=844, v29=856, v30=868 bytes.

7. Animation System

The M3 animation system uses four chunk types working together:

```
SEQS (Sequences) ──► STG_ (Groups) ──► STC_ (Sub-Track Containers)
                                              │
                                        STS_ (State lookup)
                                              │
                                        SDXX (Keyframe blocks)
```

7.1 SEQS — Animation Sequence

Tag: `SEQS` | **Versions**: v1=96 bytes, v2=92 bytes

```
struct SEQS {
    i32             id;                 // Sequence ID (always -1)
    i32             index;              // Sequence index (always -1)
    Ref<CHAR>       name;               // sequence name
    u32             animStart;          // First frame (animation ticks)
    u32             animEnd;            // Last frame
    f32             moveSpeed;          // Movement speed
    Flag            flags;              // Sequence flags
    u32             frequency;          // Playback frequency (0-100)
    u32             replayStart;        // Replay start (default 1)
    u32             replayEnd;          // Replay end (default 1)
    u32             blendTime;          // Blend time in ms (default 100)
    if (version <= 1) {
        u32         unknown05;          // v1 only (v2 omits this)
    }
    Extent          bounds;             // Bounding volume
    Ref<U8__>       animationSets;
};
```

SEQS Flags:

Mask	Name	Description
0x1	notLooping	Sequence does not loop
0x2	alwaysGlobal	Always plays globally
0x4	unknown0x4	Unknown
0x8	globalInPreviewer	Global playback in editor

7.2 STC_ — Animation Sub-Track Container

Tag: `STC_` | **Version**: v4 = 204 bytes

Each STC contains a set of animated properties for one sub-animation. It references actual keyframe data via 13 `Ref<T>` fields (one per animation data type).

```
struct STC {
    Ref<CHAR>   name;                       // track name
    u16         runsConcurrent;             // Runs alongside others
    u16         animPriority;               // Priority for blending
    u16         stsIndex;                   // Index into STS array
    u16         padding;
    Ref<U32_>   animIds;                    // animation IDs
    Ref<U32_>   animRefs;                   // animation ref indices
    u32         unknown;                    // Always 0
    // 13 References to animation data blocks (one per SDXX type):
    Ref<SDEV>   sdev;                       //  [0] -> SDEV (f32)
    Ref<SD2V>   sd2v;                       //  [1] -> SD2V (Vector2f)
    Ref<SD3V>   sd3v;                       //  [2] -> SD3V (Vector3f)
    Ref<SD4Q>   sd4q;                       //  [3] -> SD4Q (Quat)
    Ref<SDCC>   sdcc;                       //  [4] -> SDCC (ColorBGRA)
    Ref<SDR3>   sdr3;                       //  [5] -> SDR3 (Vector3f, rotation)
    Ref<SDU8>   sdu8;                       //  [6] -> SDU8 (u8)
    Ref<SDS6>   sds6;                       //  [7] -> SDS6 (i16)
    Ref<SDU6>   sdu6;                       //  [8] -> SDU6 (u16)
    Ref<SDS3>   sds3;                       //  [9] -> SDS3 (i32)
    Ref<SDU3>   sdu3;                       // [10] -> SDU3 (u32)
    Ref<SDFG>   sdfg;                       // [11] -> SDFG (Flag)
    Ref<SDMB>   sdmb;                       // [12] -> SDMB (Extent)
};
```

7.3 STG_ — Animation Group

Tag: `STG_` | **Version**: v0 = 24 bytes

```
struct STG {
    Ref<CHAR>   name;                       // group name
    Ref<U32_>   stcIndices;                 // indices into STC array
};
```

7.4 STS_ — Animation State

Tag: `STS_` | **Version**: v0 = 28 bytes

```
struct STS {
    Ref<U32_>   animIds;                    // animation IDs
    i32         parentIndex;                // Parent STS index (-1 = none)
    i32         nextIndex;                  // Next sibling (-1 = none)
    i32         childIndex;                 // First child (-1 = none)
    i16         unknownShort1;              // Default -1
    u16         unknownShort2;              // Default 0
};
```

7.5 External Animation Files (.m3a)

The M3 format supports splitting animations into separate `.m3a` files that contain additional sequences loadable on demand.

7.5.1 M3A File Format

An `.m3a` file uses the **identical MD34 container format** as a regular `.m3` file (header, index table, MODL, standard chunks). The difference is in content: an `.m3a` contains animation data (BONE, SEQS, STC_, STG_, STS_, keyframe blocks) but **no renderable geometry** (no vertex data, no mesh batches).

Although an `.m3a` file contains a `DIV_` chunk (to satisfy MODL references), all fields in the division are zeroed — including the draw flag — so it carries no renderable geometry.

7.5.2 Linking: MODL Trailing Fields

The link between a base `.m3` model and its `.m3a` files is established through two fields at the **very end** of the MODL chunk:

```
u32         m3aAnimHash;        // Single hash for .m3a binding
Ref<U32_>   m3aAnimHashes;      // array of additional .m3a hashes
```

- **`m3aAnimHash`** (`u32`): Hash identifier binding this model to one `.m3a` file. A value of `0` means no single-hash binding.
- **`m3aAnimHashes`** (`Ref<U32_>`): Array of `u32` hash values, each identifying an additional `.m3a` file.

The `.m3a` file's own MODL contains these same trailing fields (its own `m3aAnimHash`), which can be used to verify the link or chain further animation files.

> **Hash algorithm**: The hash computation is internal to Blizzard's toolchain. Third-party exporters typically leave these fields at zero.

7.5.3 Loading M3A Animations

1. **Parse the base** `.m3` normally
2. **Read the trailing MODL fields** to discover linked `.m3a` files
3. **Parse each** `.m3a` as a standard M3 file
4. **Merge animations**: append the `.m3a`'s SEQS, STC_, STG_, STS_, and keyframe data into the base model's animation set
5. The skeleton (`BONE`) in the `.m3a` should match the base model's bone hierarchy

---

8. Resolving an AnimRef

An `AnimRef<T>` field holds both a default value and a link to keyframed animation data. To retrieve the animated keyframes for a given sequence:

8.1 Overview

```
AnimRef<T>.animId
       |
       ▼
STC_.animIds[]  ── find matching animId at position K ──►  STC_.animRefs[K]
                                                                  |
                                                encodes: slot (high 16) + index (low 16)
                                                                  |
                                                                  ▼
                                                STC_.sdXX[slot] ──► SDXX AnimBlock
```

```
                                                    |
                                              frames[] + keys[]
```

8.2 Step-by-Step

1. **Read the AnimRef**: Extract `animId`, `interpType`, and `initValue`. If `animId == 0`, the property is not animated — use `initValue` as a constant.
2. **Select the STC**: Each animation sequence (SEQS) maps to a group (STG_), which lists one or more STC indices. Pick the STC for the desired sequence.
3. **Find the animId**: Search `STC.animIds` for `AnimRef.animId`. Let `K` be the match index. If no match, use `initValue`.
4. **Decode the animRef index**: Read `STC.animRefs[K]`:
   - **Slot**: `animRefs[K] >> 16` — which of the 13 SD reference slots to use
   - **Index**: `animRefs[K] & 0xFFFF` — the AnimBlock index within the SD chunk
5. **Follow the SD reference**: Use the slot to select the STC `Ref<T>` field (slot 0 → `sdev`, slot 3 → `sd4q`, etc.). Read the AnimBlock at the decoded index.
6. **Read keyframes** from the AnimBlock:

   ```
   struct AnimBlock {
       Ref<I32_> frames;           // frame times
       u32       flags;
       u32       endFrame;
       Ref<T>    keys;             // values
   };
   ```

   Interpolate between keyframe pairs using `AnimRef.interpType`.

8.3 Slot-to-Type Mapping

Slot	STC Field	SD Tag	Key Type
0	`sdev`	SDEV	f32
1	`sd2v`	SD2V	Vector2f
2	`sd3v`	SD3V	Vector3f
3	`sd4q`	SD4Q	Quat
4	`sdcc`	SDCC	ColorBGRA
5	`sdr3`	SDR3	Vector3f
6	`sdu8`	SDU8	u8
7	`sds6`	SDS6	i16
8	`sdu6`	SDU6	u16
9	`sds3`	SDS3	i32
10	`sdu3`	SDU3	u32

Slot	STC Field	SD Tag	Key Type
11	sdfg	SDFG	Flag
12	sdmb	SDMB	Extent

8.4 Interpolation Types

Value	Method
0	None (step/hold)
1	Linear
2	Hermite
3	Bezier

8.5 Example: Resolving Bone Position

```
// Given: bone.position is AnimRef<Vector3f> with animId = 42
// Want: keyframes for sequence 0

// 1. Find the STC covering sequence 0
u32 stcIdx = stg.stcIndices[0];
STC& stc = stcArray[stcIdx];

// 2. Search STC.animIds for animId 42
int K = -1;
for (int i = 0; i < stc.animIds.entries; i++) {
    if (animIdsData[i] == 42) { K = i; break; }
}
if (K < 0) return bone.position.initValue; // not animated

// 3. Decode animRefs[K]
u32 ref   = animRefsData[K];
u32 slot  = ref >> 16;      // should be 2 (SD3V for Vector3f)
u32 index = ref & 0xFFFF;   // AnimBlock index within SD3V array

// 4. Read the AnimBlock from STC.sd3v
auto& sd3vEntry = indexTable[stc.sd3v.index];
AnimBlock* blocks = readAt<AnimBlock>(sd3vEntry.offset);
AnimBlock& block = blocks[index];

// 5. Read frames and keys
i32* frames    = readRef<i32>(block.frames);
Vector3f* keys = readRef<Vector3f>(block.keys);
// Interpolate using bone.position.interpType
```

---

9. Skeleton & Mesh

9.1 BONE — Skeleton Bone

Tag: BONE | **Version**: v1 = 160 bytes

```
struct BONE {
    i32                 id;                 // Bone ID (default -1)
    Ref<CHAR>           name;
    Flag                flags;              // Bone flags
    i16                 parentIndex;        // Parent bone (-1 = root)
    u16                 padding;
    AnimRef<Vector3f>   position;           // Bind position
    AnimRef<Quat>       rotation;           // Bind rotation
    AnimRef<Vector3f>   scale;              // Bind scale
    AnimRef<u32>        batching;           // Batch visibility toggle
};
```

Bone Flags:

Mask	Name	Description
0x0001	inheritTranslation	Inherit parent translation
0x0002	inheritScale	Inherit parent scale
0x0004	inheritRotation	Inherit parent rotation
0x0010	billboard1	Billboard mode 1
0x0040	billboard2	Billboard mode 2
0x0100	project2D	2D projection mode
0x0200	animated	Has animation data
0x0400	inverseKinematics	IK bone
0x0800	skinned	Affects mesh skin
0x2000	real	Real bone (not helper)
0x4000	batch1	Primary batch bone
0x8000	batch2	Descendant of batch1 bone

9.2 Vertex Data (embedded in U8__)

Vertex data is stored as a raw byte blob in a `U8__` chunk. The format is determined by `MODL.vertexFlags`:

```
// Vertex format flags (1-based bit positions)
// Bit 10: Has vertex color (adds 4 bytes)
// Bit 18: Has UV layer 1
// Bit 19: Has UV layer 2
// Bit 20: Has UV layer 3
// Bit 21: Has UV layer 4
// Bit 30: Has UV layer 5

struct Vertex {
    Vector3f    position;       // 12 bytes at offset 0
    u8          boneWeights[4]; // normalized u8[4] at offset 12 (divide by 255.0)
    u8          boneIndices[4]; // u8[4] at offset 16
    i8          normal[3];      // signed at offset 20 (divide by 127.0)
    i8          sign;           // tangent-space handedness at offset 23 (see note)
    ColorBGRA   vertexColor;    // only if bit 10 set (4 bytes)
```

```
    i16         uv[N][2];       //  4×N bytes (N = number of UV sets)
    i8          tangent[3];     // tangent vector (divide by 127.0, see note)
    i8          tangentSign;    // tangent-space handedness (duplicate of sign)
};
// Total = 24 + (hasColor ? 4 : 0) + (numUVs × 4) + 4
```

> **Tangent handedness**: The `sign` byte at offset 23 and the `tangentSign` at the end of each vertex encode the tangent-space handedness. The GPU vertex format determines interpretation: with `R8G8B8A8_SNORM`, values map to ≈-1.0 or ≈+1.0; with `R8G8B8A8_UNORM`, `0x00`→0.0 and `0xFF`→1.0 (remapped to -1/+1 by shader logic). The engine computes `binormal = cross(normal, tangent) * sign`.
>
> **Tangent vector**: The last 4 bytes of each vertex are the tangent vector (3 signed bytes) plus a redundant handedness sign byte. Present when `MODL.flags & 0x1` (tangents computed) is set. Tangent vectors are not guaranteed to be perpendicular to normals — they represent the UV-gradient direction. The binormal is derived via cross product at runtime.

UV coordinates are stored as `int16`. For REGN v5+, use the region's `uvMultiply` and `uvOffset` fields: `float_uv = i16_uv * uvMultiply + uvOffset`. For earlier REGN versions, divide by 2048.0: `float_uv = i16_uv / 2048.0`.

9.3 DIV_ — Mesh Divisions

Tag: `DIV_` | **Version**: v2 = 52 bytes

Container referencing the index buffer, regions, batches, and bounding shapes.

```
struct DIV {
    Ref<U16_>   faces;                  // triangle indices
    Ref<REGN>   regions;                // submesh regions
    Ref<BAT_>   batches;                // draw calls
    Ref<MSEC>   msec;                   // bounding shapes
    u32         instances;              // Instance count (always 1)
};
```

> The bone lookup table is referenced from `MODL.boneLookup`, not from DIV_.

9.3.1 MSEC — Mesh Section Bounds

Tag: `MSEC` | **Version**: v1 = 72 bytes

One per region, providing animated bounding data.

```
struct MSEC {
    u32                 nodeIndex;      // Region/node index
    AnimRef<Extent>     bounds;         // Animated bounding sphere
};
```

9.4 REGN — Region / Submesh

Tag: `REGN` | **Versions**: v3=36, v4=40, v5=48 bytes

Each region defines a submesh within the vertex/index buffers.

```
struct REGN {
    u32     id;                         // Region ID
    u32     unknown01;                  // Always 0
    u32     firstVertex;                // Offset into vertex array
    u32     vertexCount;                // Number of vertices
    u32     firstIndex;                 // Offset into face index array
    u32     indexCount;                 // triangles = indexCount/3
    u16     boneCount;                  // Bones for this region
    u16     firstBoneLookup;            // Bone lookup table start
    u16     boneLookupCount;            // Bone lookup entries
    u16     padding;
    u8      vertexLookupsUsed;          // Vertex lookups used
    u8      unknown04;                  // Default 1
    u16     rootBone;                   // Root bone for this region
    if (version >= 4) {
        Flag    flags;                  // Region flags
    }
    if (version >= 5) {
        f32     uvMultiply;             // UV scale (default 16.0)
        f32     uvOffset;               // UV offset (default 0.0)
    }
};
```

REGN v4+ Flags:

Mask	Name	Description
0x1	hidden	Region is hidden
0x2	placeholder	Placeholder region
0x4	clothSimulated	Cloth-simulated
0x8	clothInfluenced	Cloth-influenced

9.5 BAT_ — Batch / Draw Call

Tag: `BAT_` | **Version**: v1 = 14 bytes

Maps a submesh region to a material.

```
struct BAT {
    u16     flags;                      // Always 0
    u16     priorityPlane;              // Always 0
    u16     regionIndex;                // Index into REGN
    u16     boundsIndex;                // Index into MSEC
    u16     colorIndex;                 // Color override index
    u16     materialRefIndex;           // Index into MATM
    i16     bone;                       // Controlling bone (-1 = none)
};
```

9.6 IREF — Inverse Bind Pose

Tag: `IREF` | **Version**: v0 = 64 bytes

One per bone — the inverse bind-pose matrix.

```
struct IREF {
    Matrix44f matrix;
};
```

9.7 ATT_ — Attachment Point

Tag: `ATT_` | **Version**: v1 = 20 bytes

Named points attached to bones (for effects, weapons, etc.).

```
struct ATT {
    i32         id;                         // Attachment ID (default -1)
    Ref<CHAR>   name;
    u32         boneIndex;                  // Attached bone index
};
```

---

10. Reading Geometry

10.1 Data Sources

Data	Source
Vertex positions, normals, UVs, weights	`MODL.vertices` → U8__ blob
Triangle indices	`DIV_.faces` → U16_ array
Submesh definitions	`DIV_.regions` → REGN array
Material assignments	`DIV_.batches` → BAT_ array
Bone index remapping	`MODL.boneLookup` → U16_ array

10.2 Step-by-Step

1. **Determine vertex stride** from `MODL.vertexFlags`:

   ```
   u32 vf = modl.vertexFlags;
   bool hasColor = (vf >> 9) & 1;  // bit 10 (1-based)
   int numUVs = ((vf >> 17) & 1) + ((vf >> 18) & 1) +
                ((vf >> 19) & 1) + ((vf >> 20) & 1) + ((vf >> 29) & 1);
   int stride = 24 + (hasColor ? 4 : 0) + (numUVs * 4) + 4;
   ```

2. **Read the vertex blob** via `MODL.vertices` → U8__ chunk.

3. **Read the index buffer** via `DIV_.faces` → U16_ chunk.

4. **Iterate over regions**:

```
for (auto& region : regions) {
    for (u32 v = 0; v < region.vertexCount; v++) {
        u32 byteOffset = (region.firstVertex + v) * stride;
        // position: 12 bytes at offset 0
        // boneWeights: 4 × u8 at offset 12 (divide by 255.0)
        // boneIndices: 4 × u8 at offset 16
        // normal: 3 × i8 at offset 20 (divide by 127.0)
        // sign: i8 at offset 23 (tangent handedness)
        // vertexColor (if present): 4 bytes at offset 24
        // UVs: i16 pairs starting at offset 24 or 28
        // tangent: 3 × i8 + 1 × i8 sign at end of vertex
    }
    for (u32 i = 0; i < region.indexCount; i += 3) {
        u16 i0 = indexBuffer[region.firstIndex + i + 0];
        u16 i1 = indexBuffer[region.firstIndex + i + 1];
        u16 i2 = indexBuffer[region.firstIndex + i + 2];
    }
}
```

5. **Remap bone indices** (region-local → global):

```
u16* boneLookup = readRef<u16>(modl.boneLookup);
u32 globalBone = boneLookup[region.firstBoneLookup + localBoneIdx];
```

6. **Convert UVs**: For REGN v5 use `uvMultiply` and `uvOffset`; earlier versions divide by 2048.0.

7. **Resolve materials** via BAT_ → MATM:

```
MATM& matMap = materialMaps[batch.materialRefIndex];
// matMap.materialType → which material array
// matMap.materialIndex → index into that array
```

10.3 Skinning

Each vertex has up to 4 bone influences. Weights are unsigned bytes (divide by 255.0). Bone indices are region-local — remap through the bone lookup table.

```
Vector3f skinnedPos = {0, 0, 0};
for (int i = 0; i < 4; i++) {
    float weight = boneWeights[i] / 255.0f;
    if (weight == 0.0f) continue;
    u32 globalBone = boneLookup[region.firstBoneLookup + boneIndices[i]];
    Matrix44f skinMatrix = boneWorldTransform[globalBone] * iref[globalBone].matrix;
    skinnedPos += weight * (skinMatrix * vertex.position);
}
```

---

11. Materials

11.1 MATM — Material Map

Tag: `MATM` | **Version**: v0 = 8 bytes

```
struct MATM {
    u32 materialType;       // Material type enum
    u32 materialIndex;      // Index into the respective material array
};
```

Material Type Enum:

Value	Type	Tag
1	Standard	MAT_
2	Displacement	DIS_
3	Composite	CMP_
4	Terrain	TER_
5	Volume	VOL_
6	Volume Noise	VON_
7	Creep	CREP
8	*(unused / not observed)*	—
9	Splat Terrain Bake	STBM
10	Reflection	REF_
11	Lens Flare	LFLR
12	Data-Driven Material	MADD

11.2 MAT_ — Standard Material

Tag: `MAT_` | **Versions**: v15=268, v16=280, v17=280, v18=280, v19=340, v20=352 bytes

The primary material type used for most rendering.

```
struct MAT {
    Ref<CHAR>           name;
    Flag                additionalFlags;
    Flag                flags;
    u32                 blendMode;
    i32                 priority;
    u32                 rttChannels;
    f32                 specularExponent;
    f32                 depthBlendFalloff;
    u32                 alphaTestThreshold;      // 0–255
    f32                 hdrSpecularMultiplier;   // default 1.0
    f32                 hdrEmissiveMultiplier;   // default 1.0
    if (version >= 20) {
        f32             hdrEnvironmentConstant;  // default 1.0
        f32             hdrEnvironmentDiffuse;   // default 0.0
        f32             hdrEnvironmentSpecular;  // default 0.0
    }
```

```
    // Texture Layers (Ref<LAYR> to LAYR chunks)
    Ref<LAYR>           diffuseLayer;
    Ref<LAYR>           decalLayer;
    Ref<LAYR>           specularLayer;
    if (version >= 16) {
        Ref<LAYR>       glossLayer;
    }
    Ref<LAYR>           emissiveLayer1;
    Ref<LAYR>           emissiveLayer2;
    Ref<LAYR>           environmentLayer;
    Ref<LAYR>           environmentMaskLayer;
    Ref<LAYR>           alphaLayer1;
    Ref<LAYR>           alphaLayer2;
    Ref<LAYR>           normalLayer;
    Ref<LAYR>           heightLayer;
    Ref<LAYR>           lightMapLayer;
    Ref<LAYR>           ambientOcclusionLayer;
    if (version >= 19) {
        Ref<LAYR>       normalBlend1MaskLayer;
        Ref<LAYR>       normalBlend2MaskLayer;
        Ref<LAYR>       normalBlend1Layer;
        Ref<LAYR>       normalBlend2Layer;
    }

    u32                 materialClass;
    u32                 layerBlendMode;
    u32                 emissiveBlendMode1;
    u32                 emissiveBlendMode2;
    u32                 specularMode;
    AnimRef<f32>        parallaxHeight;
    AnimRef<f32>        motionBlurAmount;
    if (version >= 19) {
        Ref<SR32>       normalBlendFactors;
    }
};
```

MAT_ Additional Flags:

Mask	Name	Description
0x1	depthBlendFalloff	Enable depth blend falloff
0x4	vertexColor	Uses vertex color
0x8	vertexAlpha	Uses vertex alpha

MAT_ Flags:

Mask	Name	Description
0x00000001	vertexColor	Enable vertex color
0x00000002	vertexAlpha	Enable vertex alpha
0x00000004	unfogged	Not affected by fog
0x00000008	twoSided	Two-sided rendering
0x00000010	unshaded	Unlit / unshaded

Mask	Name	Description
0x00000020	noShadowsCast	Does not cast shadows
0x00000040	noHitTest	Excluded from hit testing
0x00000080	noShadowsReceive	Does not receive shadows
0x00000100	depthPrepass	Z-fill pre-pass
0x00000200	terrainHDR	Terrain HDR mode
0x00000800	simulateRoughness	Simulate roughness
0x00001000	pixelForwardLighting	Pixel forward lighting
0x00002000	depthFog	Depth-based fog
0x00004000	transparentShadows	Transparent shadows
0x00008000	decalLighting	Decal lighting mode
0x00010000	transparentDepthEffects	Transparent depth effects
0x00020000	transparentLocalLights	Transparent local lights
0x00040000	disableSoft	Disable soft blending
0x00080000	doubleLambert	Double Lambert shading
0x00100000	hairLayerSorting	Hair layer sorting
0x00200000	acceptSplats	Accept splat projections
0x00400000	decalLowRequired	Decal low LOD required
0x00800000	emisLowRequired	Emissive low LOD required
0x01000000	specLowRequired	Specular low LOD required
0x02000000	acceptSplatsOnly	Accept splats only
0x04000000	backgroundObject	Background object
0x10000000	depthPrepassLowRequired	Depth prepass low LOD
0x20000000	noHighlighting	Disable highlighting
0x40000000	clampOutput	Clamp output
0x80000000	geometryVisible	Geometry visible (v17+)

Layer Indices (v19/v20, 18 layers):

Index	Layer	Shader Name	Common Texture
0	Diffuse	Diffuse	`*_Diff.dds`
1	Decal	Decal	
2	Specular	Specular	`*_Spec.dds`
3	Gloss	SpecularExponent	(v15 skips this)
4	Emissive	Emissive	`*_Emis.dds`

Index	Layer	Shader Name	Common Texture
5	Emissive 2	Emissive2	
6	Environment	Envio	
7	Environment Mask	EnvioMask	
8	Alpha	AlphaMask	
9	Alpha 2	AlphaMask2	
10	Normal	Normal	`*_Norm.dds`
11	Height	Heightmap	
12	Light Map	LightMap	(v19+)
13	Ambient Occlusion	AO	(v19+)
14	Normal Blend 1 Mask	NormalBlendMask	(v19+)
15	Normal Blend 2 Mask	NormalBlendMask2	(v19+)
16	Normal Blend 1	NormalBlendNormal	(v19+)
17	Normal Blend 2	NormalBlendNormal2	(v19+)

> **Note**: The "Shader Name" column shows the token used in HLSL shaders (e.g., `PSMaterialLayer_Envio`). "Envio" (Environment) and "EnvioMask" layers support cubemap textures via `texCUBE()` sampling in addition to standard 2D textures.

BlendMode (`blendMode` field):

Value	Name	Description
0	Opaque	Fully opaque rendering
1	AlphaBlend	Standard alpha blending
2	Add	Additive blending
3	AlphaAdd	Alpha-modulated additive
4	Mod	Multiplicative blending
5	Mod2x	Double multiplicative

MaterialClass (`materialClass` field):

Value	Name	Description
0	Unit	Unit material (characters, heroes)
1	Building	Building/structure material
2	Doodad	Doodad/prop material
3	SpecialFX	Special effect material

LayerBlendOp (`layerBlendMode`, `emissiveBlendMode1`, `emissiveBlendMode2` fields):

Value	Name	Description
0	Mod	Multiply: `base * layer`
1	Mod2x	Double multiply: `base * layer * 2`
2	Add	Add: `base + layer`
3	Lerp	Linear interpolate by layer alpha
4	TeamColorEmissiveAdd	Team color emissive add
5	TeamColorDiffuseAdd	Team color diffuse add
6	AddNoAlpha	Add ignoring alpha channel

SpecularMode (`specularMode` field):

Value	Name	Description
0	RGB	Use RGB channels for specularity
1	AlphaOnly	Use alpha channel only for specularity

11.3 LAYR — Texture Layer

Tag: `LAYR` | **Versions**: v22=356, v25=468, v26=464 bytes

Each texture layer defines a texture source and its UV animation parameters.

```
struct LAYR {
    u32                 id;
    Ref<CHAR>           texturePath;
    AnimRef<ColorBGRA>  color;              // Tint color
    Flag                flags;
    u32                 uvMapping;
    u32                 colorType;
    AnimRef<f32>        rgbMultiply;
    AnimRef<f32>        rgbAdd;
    u32                 pocTexture;
    if (version >= 24) {
        f32             noiseAmplitude;
        f32             noiseFrequency;
    }
    u32                 textureSource;
    u32                 aviFrameRate;
    u32                 aviStart;
    u32                 aviStop;
    u32                 aviLoop;
    u32                 aviSync;
    AnimRef<u32>        aviPlay;
    AnimRef<u32>        aviRestart;
    u32                 flipbookRows;
    u32                 flipbookColumns;
    AnimRef<u16>        currentFrame;
    AnimRef<Vector2f>   uvOffset;
    AnimRef<Vector3f>   uvAngle;
    AnimRef<Vector2f>   uvTiling;
    AnimRef<f32>        wOffset;
    AnimRef<f32>        wTiling;
```

```
    AnimRef<f32>        mapAlpha;
    if (version >= 23) {
        AnimRef<Vector3f> triplanarOffset;
        AnimRef<Vector3f> triplanarScale;
    }
    u32                 uvSourceRelated;
    u32                 fresnelMode;
    f32                 fresnelExponent;
    f32                 fresnelMin;
    f32                 fresnelMax;
    if (version >= 25) {
        Vector3f        fresnelTranslation;
        Vector3f        fresnelMask;
        Vector2f        fresnelRotation;
    }
    u32                 uvDensity;
};
```

LAYR Flags:

Mask	Name	Description
0x0004	uvWrapX	Wrap texture in U
0x0008	uvWrapY	Wrap texture in V
0x0010	colorInvert	Invert color
0x0020	colorClamp	Clamp to [0,1]
0x0040	colorAdd	Additive blending
0x0080	colorMultiply	Multiplicative blending
0x0100	particleUVFlipbook	Flipbook UVs for particles
0x0200	video	Video texture
0x0400	color	Solid color (no texture)
0x0800	replaceTextureSource	Override texture source
0x4000	fresnelTransform	Fresnel-based UV transform
0x8000	fresnelNormalize	Normalize fresnel values

UVMapping (`uvMapping` field):

Value	Name	Description
0	ExplicitUV0	Use UV coordinate set 0
1	ExplicitUV1	Use UV coordinate set 1
2	ReflectCubicEnvio	Cubic environment reflection mapping
3	ReflectSphericalEnvio	Spherical environment reflection mapping
4	PlanarLocalZ	Planar local UVs (Z plane)
5	PlanarWorldZ	Planar world UVs (Z plane)

Value	Name	Description
6	ParticleFlipbook	Particle flipbook UVs
7	CubicEnvio	Cubic environment mapping
8	SphericalEnvio	Spherical environment mapping
9	ExplicitUV2	Use UV coordinate set 2
10	ExplicitUV3	Use UV coordinate set 3
11	PlanarLocalX	Planar local UVs (X plane)
12	PlanarLocalY	Planar local UVs (Y plane)
13	PlanarWorldX	Planar world UVs (X plane)
14	PlanarWorldY	Planar world UVs (Y plane)
15	ScreenSpace	Screen-space UVs
16	TriPlanarLocal	Tri-planar blending (local space)
17	TriPlanarWorld	Tri-planar blending (world space)
18	TriPlanarWorldLocalZ	Tri-planar world with local Z

ColorChannelSelect (`colorType` field):

Value	Name	Description
0	RGB	Use RGB channels (alpha set to 1)
1	RGBA	Use all RGBA channels
2	Alpha	Use alpha channel only (splat to all)
3	Red	Use red channel only (splat to all)
4	Green	Use green channel only (splat to all)
5	Blue	Use blue channel only (splat to all)

FresnelMode (`fresnelMode` field):

Value	Name	Description
0	None	No fresnel effect
1	Standard	Standard fresnel (edge glow)
2	Inverted	Inverted fresnel (center glow)

11.4 DIS_ — Displacement Material

Tag: `DIS_` | **Version**: v4 = 68 bytes

```
struct DIS {
    Ref<CHAR>       name;
    u32             unknown;
    AnimRef<f32>    strength;
    Ref<LAYR>       normalMap;
```

```
    Ref<LAYR>       strengthMap;
    Flag            flags;
    u32             priority;
};
```

> **Note**: The `normalMap` layer provides the displacement direction normal, and `strengthMap` controls the displacement magnitude. Despite the field names, these are displacement-specific layers — not the same as the standard material normal map.

11.5 CMP_ — Composite Material

Tag: `CMP_` | **Version**: v2 = 28 bytes

```
struct CMP {
    Ref<CHAR>   name;
    u32         priority;
    Ref<CMS_>   sections;
};
```

CMS_ Sub-structure:

```
struct CMS {
    u32             materialIndex;  // Index into MATM
    AnimRef<f32>    mapMultiplier;  // Blend weight
};
```

11.6 TER_ — Terrain Material

Tag: `TER_` | **Version**: v1 = 28 bytes

```
struct TER {
    Ref<CHAR>   name;
    Ref<LAYR>   terrainMap;
    u32         unknown;
};
```

11.7 VOL_ — Volume Material

Tag: `VOL_` | **Version**: v0 = 84 bytes

```
struct VOL {
    Ref<CHAR>       name;
    u32             blendMode;
    u32             falloffType;
    AnimRef<f32>    density;
    Ref<LAYR>       colorMap;
    Ref<LAYR>       noiseMap1;
    Ref<LAYR>       noiseMap2;
    u32             alphaThreshold;
```

```
    Flag            flags;
};
```

> **Note**: VOL_ and VON_ both use `SHADINGMODE_VOLUME` (shading mode 4) in the engine shader system. They are distinguished by a secondary toggle: `VOLUME_TYPE_UNIFORM` = 0 (VOL_) vs `VOLUME_TYPE_NOISY` = 1 (VON_).

Volume Falloff Type (`falloffType` field):

Value	Name	Description
0	Linear	Linear density falloff
1	Exponential	Exponential density falloff

11.8 VON_ — Volume Noise Material

Tag: `VON_` | **Version**: v0 = 268 bytes

```
struct VON {
    Ref<CHAR>       name;
    u32             falloffType;
    u32             drawTransparency;
    AnimRef<f32>    density;
    AnimRef<f32>    nearPlane;
    AnimRef<f32>    falloff;
    Ref<LAYR>       colorMap;
    Ref<LAYR>       noiseMap1;
    Ref<LAYR>       noiseMap2;
    AnimRef<Vector3f> scrollRate;
    AnimRef<Vector3f> position;
    AnimRef<Vector3f> scale;
    AnimRef<Vector3f> rotation;
    u32             alphaThreshold;
    Flag            flags;
};
```

Volume Falloff Type (`falloffType` field — same enum as VOL_):

Value	Name	Description
0	Linear	Linear density falloff
1	Exponential	Exponential density falloff

Camera Position Mode (`drawTransparency` field):

Value	Name	Description
0	Outside	Camera is outside the volume
1	Inside	Camera is inside the volume

VON_ Flags (`flags` field):

Mask	Name	Description
0x1	DrawAfterTransparency	Draw in a separate pass after transparency

11.9 CREP — Creep Material

Tag: `CREP` | **Version**: v1 = 28 bytes

```
struct CREP {
    Ref<CHAR>   name;
    Ref<LAYR>   maskMap;
    u32         creepLow;
};
```

11.10 STBM — Splat Terrain Bake Material

Tag: `STBM` | **Version**: v0 = 48 bytes

```
struct STBM {
    Ref<CHAR>   name;
    Ref<LAYR>   diffuseMap;
    Ref<LAYR>   normalMap;
    Ref<LAYR>   specularMap;
};
```

11.11 REF_ — Reflection Material

Tag: `REF_` | **Versions**: v1=84, v2=156, v3=160 bytes

```
struct REF {
    Ref<CHAR>       name;
    u32             unknown;
    AnimRef<f32>    reflectionStrength;
    AnimRef<f32>    displacementStrength;
    if (version >= 2) {
        AnimRef<f32> reflectionOffset;
        AnimRef<f32> blurAngle;
        AnimRef<f32> blurDistanceMax;
    }
    Ref<LAYR>       reflectionMap;
    Ref<LAYR>       displacementMap;
    if (version >= 2) {
        Ref<LAYR>   blurMap;
    }
    Flag            flags;
    if (version >= 3) {
        u32         unknown2;
    }
};
```

REF_ Flags (`flags` field):

Mask	Name	Description
0x1	UseReflectionMap	Enable the reflection map layer
0x2	UseDisplacementMap	Enable the displacement map layer
0x4	RenderInTransparentPass	Render in the transparent pass
0x8	Blurring	Enable blurring effect
0x10	UseBlurMap	Enable the blur map layer (v2+)

11.12 LFLR — Lens Flare Material

Tag: `LFLR` | **Versions**: v2=80, v3=152 bytes

```
struct LFLR {
    Ref<CHAR>           name;
    Ref<LAYR>           flareMap;
    Ref<LAYR>           maskMap;
    Ref<LFSB>           subFlares;
    u32                 columns;
    u32                 rows;
    f32                 distanceFade;
    AnimRef<f32>        intensity;
    if (version >= 3) {
        Ref<CHAR>       libName;
        AnimRef<ColorBGRA> color;
        AnimRef<f32>    hdr;
        AnimRef<f32>    size;
    }
};
```

LFSB Sub-Flare (per entry, 56 bytes):

```
struct SubFlare {
    u32     index;
    f32     position;
    Vector2f sizeXY;
    Vector2f scaleXY;
    Vector2f fadeIn;
    Vector2f fadeOut;
    ColorBGRA colorAlpha;
    u32     faceCenter;
    Vector2f offset;
};
```

> **Note**: The lens flare shader has two modes controlled by whether a `maskMap` is present. Mode 0 (no mask): uses only the `flareMap` atlas. Mode 1 (with mask): multiplies the atlas color by a "dirty lens" mask texture.

11.13 MADD — Data-Driven Material

Tag: `MADD` | **Versions**: v1=140, v2=152, v3=160 bytes

The engine's own name for this chunk is **`SDataDrivenMaterialData`**, taken from the chunk registration table in the Heroes client (`Heroes.decrypted`, macOS x86-64), which registers it as `(tag='DDAM', size=0xA0, version=3)`.

It is not "additional" data. At load the Heroes renderer converts every `StandardMaterial` (MATM type 1), `DisplacementMaterial` (2) and `ReflectionMaterial` (10) in a model into one of these and rewrites the MATM entry to type 12, so this is the only material representation the renderer consumes. See §11.15.

Version layout:

- **v1** (140 bytes): 4 references + 48 reserved bytes + 11 scalars
- **v2** (152 bytes): +`extraHashes` at offset 24
- **v3** (160 bytes): +2 appended u32 fields

```
struct MADD {
    Ref<CHAR>   materialName;       // 0
    Ref<U32_>   fragmentHashes;     // 12  crc32 of each shader fragment name, in link
order
    if (version >= 2) {
        Ref<U32_> extraHashes;      // 24
    }
    Ref<CHAR>   propertyBlob;       // 36  binary; the dictionary below
    Ref<SCHR>   texturePaths;       // 48  array of Ref<CHAR>; indexed by the Tex*
properties

    u8          reserved[48];       // zero in all 2582 corpus records, every version

    f32         unknown108;         // 1.0 / 1.5 / 2.0
    f32         unknown112;         // 1.0 in every known record
    f32         unknown116;
    u32         effectNameHash;     // crc32 of the shader permutation name; 0 = compute at
load
    u32         unknown124;
    u32         padding128;         // always 0
    i32         unknown132;
    u32         unknown136;         // packed bit field
    u32         unknown140;
    u32         unknown144;
    u8          unknown148;
    u8          alphaFresnelFlags;  // derived cache the loader recomputes
    u8          shaderType;         // 0 Material, 1 Medium, 2 Simple, 3 Particle, 4 Splat
    u8          unknown151;
    if (version >= 3) {
        u32     effectNameHash2;    // 0xFFFFFFFF = none
        u32     effectNameHash3;    // 0xFFFFFFFF = none
    }
};
```

The engine's per-record fixup resolves exactly **five** references — at 0, 12, 24, 36 and 48 — and never touches bytes 60–107. Earlier revisions of this document modelled those 48 bytes as four more `Reference`s; both readings sum to the same element size, but the loader treats only five as references.

The property blob

`propertyBlob` is a self-describing two-level dictionary keyed by **`zlib` CRC32** of unprefixed names (`crc32("Diffuse")`, `crc32("DiffuseUVTransform")`). All offsets are absolute within the blob and 64-bit.

```
Level 1 — fragment map
    u32  count
    u64  ofsKeys            always 20
    u64  ofsGroupOffsets    always 20 + 4*count
    u32  key[count]         crc32(<FragmentName>)
    u64  groupOffset[count]

Level 2 — one property group per key
    u32  count              CAPACITY, not population
    u64  ofsHashes          always groupOffset + 36
    u64  ofsSizes           ofsHashes + 4*count
    u64  ofsTypes           ofsSizes  + 2*count
    u64  ofsValueOffsets    ofsTypes  + count
    u32  hash[count]        crc32(<PropertyName>)
    u16  size[count]
    u8   type[count]        1 == live slot
    u64  valueOffset[count]
```

> ⚠️ `count` **is the array's capacity, not its population, and the writer never clears the tail.** Stale slots carry plausible-looking hashes (really leftover u16 index pairs), sizes like 29282, and value offsets near 2^{63}. **The live-slot test is** `type == 1`, which holds for all 32,554 real slots corpus-wide and nothing else. A live-typed slot whose `valueOffset + size` leaves the blob is stale too and must be dropped individually — rejecting the whole record discards 141 good ones.

`fragmentHashes` duplicates the level-1 key list, in the same order.

Value shapes by `size`:

size	shape
4	scalar — `f32`, a `u32` enum, or a BGRA colour, depending on the property
8	`Tex<X>`: `{u32 index into texturePaths, u32 texture source}`
12	3 floats (tri-planar offset / scale)
16	4 floats
20	`<X>ChannelSelection`: `{u32 ColorChannelSelect, f32 multiply, f32 add, f32, u16 flags}` — the trailing field is a **u16**; the two bytes after it are uninitialised
32	`<X>UVTransform`: `{f32 offsetU/V, tilingU/V, angleU/V/W, u32 flags}`
48	`FresnelParams<X>`: `{u32 FresnelMode, f32 exponent, min, max, rotation, mask}`

`DataDrivenMaterial::decodeProperties()` returns this decoded, with names resolved against a table of 399 verified CRC32 → name pairs (387 of which were harvested from string literals in the Heroes client itself). Over the HotS corpus it names 34,981 of 35,026 properties.

11.14 Runtime Material System (engine behaviour, informative)

> This subsection documents how the StarCraft II engine *consumes* M3 materials at render time. It is not part of the file format, but it explains how the type arrays above are actually used. Reverse-engineered from the retail client (`base71061`, macOS x86-64); function names are RE labels.

Model → runtime material objects. At load, each per-type material array (`MAT_`/`DIS_`/`CMP_`/`TER_`/`VOL_`/`CREP`/`VON_`/`STBM`/`LFLR`) is instantiated into a concrete C++ material object deriving from a

common `CBaseMaterial` (RTTI-confirmed classes: `CMaterial`, `CDisplacementMaterial`, `CCompositeMaterial`, `CCreepMaterial`, `CVolumeMaterialBase` → `CVolumeMaterialUniform`/`CVolumeMaterialNoisy`, `CSplatTerrainBakeMaterial`, `CReflectionMaterial`, `CLensFlareMaterial`, plus the aux `CCloakingMaterial`). The `MATM (materialType, materialIndex)` map is flattened into a **per-ref material-object list** (56-byte stride) indexed directly by a batch's `materialRefIndex`.

Dispatch is virtual, not a switch. Rendering a mesh batch resolves its material object and calls the object's `ApplyForDraw` virtual — **vtable slot 10 (offset `+0x50`)**. Each concrete class overrides that slot with its type-specific setup, and every override funnels into `CBaseMaterial::ApplyForDraw → Material_ResolveShaderStateForDraw`, which lazily resolves/compiles the VS+PS permutation via the shared shader pipeline (see the material→shader path: permutation-flag mask at `material+748` → resource-budget check → shader-pair cache/compile). Composite materials (`CMP_`) select an active sub-material and **re-dispatch** to *its* `ApplyForDraw`.

Draw call chain (per mesh batch):

```
CModel::DrawMeshBatchMaterial            (CModel vtable slot 25, +0xC8)
  → reads BAT_ materialRefIndex + region, uploads node transforms to shader scratch
  → Model_DispatchMaterialByRef          (material = matList + 56*materialRefIndex)
      → <Type>Material::ApplyForDraw      (vtable slot 10 / +0x50)   ← per-type setup
          → CBaseMaterial::ApplyForDraw → Material_ResolveShaderStateForDraw
              → ResolveMaterialShaderPermutation → GetOrCreateShaderPair (compile VS+PS)
          → Material_UploadLayerUVTransform   (per-LAYR UV/rotation → shader scratch)
          → CMaterial_BindStandardLayers      (binds the LAYR texture samplers)
          → Material_BindShaderParamProvider  (resolves shader input registers)
          → Material_CommitAndDraw            (commit constants + issue draw)
```

MATM type	Class	`ApplyForDraw` configures
1 Standard	`CMaterial`	blend/pass flags from batch layer type; binds the 12 shader-used LAYR layers + standard param providers
2 Displacement	`CDisplacementMaterial`	displacement strength + normal layer
3 Composite	`CCompositeMaterial`	selects active sub-material, re-dispatches to it
5 Volume	`CVolumeMaterialUniform`	uniform-volume flags/layers
6 Volume Noise	`CVolumeMaterialNoisy`	3 volume UV layers + noise setup
7 Creep	`CCreepMaterial`	creep LAYR/UV binds
9 Splat Terrain Bake	`CSplatTerrainBakeMaterial`	base permutation resolve only
10 Reflection	`CReflectionMaterial`	reflection layers
11 Lens Flare	`CLensFlareMaterial`	occlusion params

> **Terrain (type 4)** is *not* driven through `CModel::DrawMeshBatchMaterial`. The `CTerrain3Material*` family (Solid/Ground/Cliff/Creep/HeightSample) lives in a separate terrain-render subsystem (`CActorTerrain`/`CTerrain3`) with its own splat draw loop; only its shader warmup (`terrain blender shader`, `Creep noise`) shares the common pipeline.

Startup shader warmup. All built-in engine materials — `Model`/`Cliff`/`Ribbon`/`Particle`/`Splat`/`Light` (M3 model set), `terrain blender shader`/`Creep noise`, and the UI/post-process materials — are precompiled across 5 quality levels by `PrecompileAllBuiltinMaterialShaders` so the per-draw path only hits the shader cache, never a stall.

11.15 Data-Driven Material Pipeline (Heroes of the Storm, informative)

> Heroes replaced the per-type virtual dispatch of §11.14 with a single data-driven path. Reverse-engineered from `Heroes.decrypted` (macOS x86-64); function names are RE labels. Not part of the file format.

Everything is converted at load. A model-level pass walks the MATM table and rewrites the fixed-function materials into MADD records:

MATM type	chunk	stride	converter	result
1 Standard	`MAT_`	340	`sub_1027A2380`	MATM rewritten to `{12, newIndex}`
2 Displacement	`DIS_`	68	`sub_10279D220`	MATM rewritten to `{12, newIndex}`
10 Reflection	`REF_`	160	`sub_1027A0560`	type kept; `REF_[i]+156` links the record
3 Composite	`CMP_`	28	—	indirection, resolved first
12 Data-driven	`MADD`	160	—	already data-driven

Each converter also receives whether a particle system, ribbon, projector or composite material uses this material; those change which fragments are emitted (a projector adds `SplatPS`, for instance).

The fragment list is the shader permutation name. To resolve a record to a compiled effect, the renderer seeds a CRC32 with the `shaderType` token and accumulates each fragment name in `fragmentHashes` order:

```
u32 h = record.effectNameHash;
if (h == 0 || h == 0xFFFFFFFF) {
    h = crc32(0, shaderTypeToken(record.shaderType));   // "Material", "MaterialParticle",
...
    for (i = 0; i < record.fragmentHashes.count; i++)
        h = crc32(h, fragmentRegistry[record.fragmentHashes[i]].name);
}
record.effectIndex = lookup(effectRegistry, h);
```

Because `crc32(crc32(A), B) == crc32(A||B)`, the accumulated value is exactly `crc32("<ShaderType><fragment0><fragment1>…")`. The client's own registered permutation names decompose on that grammar, e.g.

```
Material         + AlphaMask + AlphaMask2 + AlphaTest                              + Halo
MaterialParticle + AlphaMask + FresnelSetup + FresnelAlpha + AlphaTest             + ZFill
```

Fragment order is fixed and part of the name. Across the 1,903 corpus records that use only fixed-function fragments, the 67 fragments yield 891 ordered pairs with zero contradictions — no two records ever disagree on which of two fragments comes first. (Those pairs constrain only 40% of the possible orderings, so the data proves a consistent order exists without fully determining it.)

Two registries, both built at runtime, 32-byte stride: a *fragment* registry keyed by `crc32(name)` and an *effect* registry keyed by `crc32(permutation name)`. On load the callback rewrites `fragmentHashes` **in place**, replacing each hash with its index in the fragment registry — so a live in-memory record holds indices where the file holds hashes.

Records authored directly as data-driven (229 of 2,582 in the corpus) use fragments outside the fixed-function vocabulary, carry a pre-baked `effectNameHash`, and do not follow the fixed order.

11.16 Shader-Graph Materials (Heroes of the Storm, informative)

Not every MADD record was converted from a fixed-function material. 278 of the 2,582 in the reference corpus were authored directly in the node editor, and use a separate fragment vocabulary — `Rail*`, `Surface*`, `GradientColorize` — in place of the layer fragments of §11.15. The two vocabularies are disjoint apart from pipeline fragments (`Fog`, `FOW`, `MaterialFinalize`, `FogAmount`, `SplatPS`, `SplatMaterialAttenuation`), so the presence of a `Rail*` or `Surface*` fragment is an exact test for one.

`extraHashes` is the node label array. It runs parallel to the fragment list — one entry per node, holding `crc32` of the name the artist gave that node — and is `0` for pipeline fragments, which are not nodes. It is populated only for shader-graph records; every fixed-function record leaves it empty. Node names default to the node type, so `RailTex` and `RailLighting` appear as labels.

The graph's *edges* are not stored. A record is a node list, not a graph, so the topology cannot be recovered from the file.

`DataDrivenMaterial::approximateStandardMaterial()` produces a best-effort `StandardMaterial` for these anyway, assigning each sampled texture to a layer slot from the node type, the node label, and the texture filename, in that order. It forwards fixed-function records to `toStandardMaterial()` unchanged, and its result always carries an `approximated from a shader graph` entry in `lossy`. 146 of the 278 approximate; the rest are procedural effects with no texture that denotes a surface role.

---

12. Particle Systems

12.1 PAR_ — Particle Emitter

Tag: `PAR_` | **Versions**: v10=1300, v11=1304, v12=1316, v14=1356, v17=1460, v18=1464, v19=1464, v21=1464, v22=1484, v23=1492, v24=1496 bytes

The most complex chunk type. Fully defines a particle system with emission, physics, noise, collision, per-particle animation, and rendering parameters.

```
struct PAR {
    // Always-present base
    u32                 boneIndex;          // Emitter bone index
    u32                 materialIndex;      // Material reference index

    if (version >= 17) {
        u32             additionalFlags;    // ParticleAdditionalFlag (v17+)
    }

    // Initial velocity channels
    AnimRef<f32>        initialSpeed;
    AnimRef<f32>        initialSpeedRandom;
    if (version <= 14) {
        u32             emitSpeedRandomize; // Legacy flag before additionalFlags
    }
    AnimRef<f32>        initialYaw;
    AnimRef<f32>        initialPitch;
    AnimRef<f32>        initialHorizontal;
    AnimRef<f32>        initialVertical;

    // Lifetime
    AnimRef<f32>        lifetime;
    AnimRef<f32>        lifetimeRandom;
    if (version <= 14) {
        u32             lifespanRandomize;  // Legacy flag before additionalFlags
    }
```

```
// Distance and gravity
f32                 killRadius;
u32                 gravityX;
u32                 gravityY;
f32                 gravity;

if (version >= 12) {
    // Midpoint timing
    f32             sizeMidTime;
    f32             colorMidTime;
    f32             alphaMidTime;
    f32             rotationMidTime;
}

if (version >= 14) {
    // Midpoint hold timing
    f32             sizeMidHoldTime;
    f32             colorMidHoldTime;
    f32             alphaMidHoldTime;
    f32             rotationMidHoldTime;
}

// Curves
AnimRef<Vector3f>   sizeAnimation;
if (version <= 11) {
    f32             sizeMidTimeLegacy;  // Old location (v11-)
}
AnimRef<Vector3f>   rotationAnimation;
if (version <= 11) {
    f32             rotationMidTimeLegacy; // Old location (v11-)
}
AnimRef<ColorBGRA>  colorStart;
AnimRef<ColorBGRA>  colorMid;
AnimRef<ColorBGRA>  colorEnd;
if (version <= 11) {
    f32             colorMidTimeLegacy; // Old location (v11-)
    f32             alphaMidTimeLegacy; // Old location (v11-)
}

// Physics
f32                 drag;
if (version <= 14) {
    u32             massRandomizeLegacy;
}
f32                 mass;
f32                 massRandom;
if (version >= 12) {
    f32             massSizeMultiplier;
}
if (version <= 14) {
    u32             worldSpaceLegacy;
}

// Force channels
u16                 localForces;
u16                 worldForces;
u16                 localForcesFallback;
u16                 worldForcesFallback;
```

```
    if (version >= 24) {
        f32             worldForcesMassMultiplier;
    }

    // Noise
    f32                 noiseAmplitude;
    f32                 noiseFrequency;
    f32                 noiseCoherence;
    f32                 noiseEdge;
    if (version <= 11) {
        f32             noiseSmoothness;
    }
    if (version >= 11) {
        u32             indexPlusLength;
    }

    // Emission
    u32                 maxParticles;
    AnimRef<f32>        emissionRate;
    u32                 emitterShape;
    AnimRef<Vector3f>   shapeOuter;
    AnimRef<Vector3f>   shapeInner;
    AnimRef<f32>        outerRadius;
    AnimRef<f32>        innerRadius;
    if (version >= 14) {
        Ref<U32_>       shapeRegions;
    }

    // Randomization
    u32                 velocityType;
    u32                 sizeRandomEnable;
    AnimRef<Vector3f>   sizeRandomAnimation;
    u32                 rotationRandomEnable;
    AnimRef<Vector3f>   rotationRandomAnimation;
    u32                 colorRandomEnable;
    AnimRef<ColorBGRA>  colorStartRandom;
    AnimRef<ColorBGRA>  colorMidRandom;
    AnimRef<ColorBGRA>  colorEndRandom;
    u32                 alphaRandomEnable;

    // Squirt and flipbook
    AnimRef<u16>        squirtAmount;
    u8                  flipbookStartInitIndex;
    u8                  flipbookStartStopIndex;
    u8                  flipbookEndInitIndex;
    u8                  flipbookEndStopIndex;
    f32                 flipbookMidTime;
    u16                 flipbookColumns;
    u16                 flipbookRows;
    if (version >= 12) {
        f32             flipbookColumnFraction;
        f32             flipbookRowFraction;
    }

    // Collision
    f32                 bounce;
    f32                 friction;
    i32                 collisionSpawnIndex;
    u32                 collisionSpawnMin;
```

```
    u32                 collisionSpawnMax;
    f32                 collisionSpawnChance;
    f32                 collisionSpawnEnergy;
    u32                 collisionDieBounce;

    // Instance
    u32                 instanceType;
    f32                 tailLength;
    Vector3f            instanceAngle;
    if (version >= 17) {
        f32             instanceDistance;
    }

    // Variation channels (fixed order)
    u32                 pitchType;
    AnimRef<f32>        pitchAmplitude;
    AnimRef<f32>        pitchFrequency;
    u32                 yawType;
    AnimRef<f32>        yawAmplitude;
    AnimRef<f32>        yawFrequency;
    u32                 speedType;
    AnimRef<f32>        speedAmplitude;
    AnimRef<f32>        speedFrequency;
    u32                 sizeType;
    AnimRef<f32>        sizeAmplitude;
    AnimRef<f32>        sizeFrequency;
    u32                 alphaType;
    AnimRef<f32>        alphaAmplitude;
    AnimRef<f32>        alphaFrequency;
    u32                 colorType;
    AnimRef<f32>        colorAmplitude;
    AnimRef<f32>        colorFrequency;
    u32                 rotationType;
    AnimRef<f32>        rotationAmplitude;
    AnimRef<f32>        rotationFrequency;
    u32                 horizontalType;
    AnimRef<f32>        horizontalAmplitude;
    AnimRef<f32>        horizontalFrequency;
    u32                 verticalType;
    AnimRef<f32>        verticalAmplitude;
    AnimRef<f32>        verticalFrequency;

    // Parent velocity and phase
    AnimRef<f32>        particleVelocity;
    if (version >= 22) {
        AnimRef<f32>    phaseShift;
    }

    // Flags
    Flag                flags;
    if (version >= 18) {
        u32             rotationFlags;
    }

    if (version >= 14) {
        // Smoothing (InterpolationMode enum, see below)
        InterpolationMode colorSmoothing;
        InterpolationMode sizeSmoothing;
        InterpolationMode rotationSmoothing;
```

```
    }

    if (version >= 17) {
        // UV screen-space controls
        AnimRef<f32>    alphaThreshold;
        AnimRef<Vector2f> uvOffset;
        AnimRef<Vector3f> uvAngle;
        AnimRef<Vector2f> uvTiling;
    }

    // Always present
    Ref<SVC3>           splatLineData;
    f32                 windMultiplier;
    u32                 lodReduce;
    u32                 lodCut;
    AnimRef<f32>        lowerBound;
    AnimRef<f32>        upperBound;
    i32                 trailLinkIndex;
    f32                 trailChance;
    AnimRef<f32>        trailEmissionRate;
    i32                 splatProjectionIndex;
    f32                 splatChance;
    Ref<SCHR>           modelPaths;
    Ref<U32_>           copyIndices;

    if (version >= 23) {
        f32             spawnRibbonOnBounceChance;
        i32             ribbonLinkIndex;            // index into RIB_ (-1 = none)
    }
};
```

Emitter Shape Enum:

Value	Shape
0	Point
1	Plane
2	Sphere
3	Cube (Box)
4	Cylinder
5	Disc
6	Mesh
7	Spline

Particle Instance Type Enum (maps 1:1 to `b_iInstanceType` in `Particle.fx`):

Value	Name	Description
0	Billboard	Camera-facing billboard quad
1	Tail	Velocity-stretched quad; length from `tailLength`
2	FaceTravelDir	Quad oriented along instantaneous velocity

Value	Name	Description
3	FaceWorldDir	Quad oriented along a fixed world direction
4	SingleAxis	Billboard locked to a single rotation axis
5	TerrainOriented	Quad projected onto terrain normal
6	TerrainDirOriented	Terrain-oriented + velocity-stretched
7	EmitterOriented	Quad uses the emitter bone's orientation
8	PhysicsOriented	Quad oriented by physics simulation
9	Pinned	Stretch between spawn origin and current position
10	Trail	Like Tail but offset by one tail-length

LOD Enum:

Value	Level
0	None
1	Low
2	Medium
3	High
4	Ultra

Main Flags (`flags` field):

Mask	Name	Description
0x1	Sort	Sort by distance
0x2	CollideTerrain	Collide with terrain
0x4	CollideObjects	Collide with objects
0x8	CollideEmit	Emit on collision
0x10	EmitShapeCutout	Emit from shape cutout
0x20	InheritEmitParams	Inherit emission parameters
0x40	InheritParentVelocity	Inherit parent velocity
0x80	SortHeight	Sort by height
0x100	SortReverse	Reverse sort order
0x200	OldRotationSmooth	Legacy rotation smoothing
0x400	OldRotationBezier	Legacy rotation bezier
0x800	OldSizeSmooth	Legacy size smoothing
0x1000	OldSizeBezier	Legacy size bezier
0x2000	OldColorSmooth	Legacy color smoothing
0x4000	OldColorBezier	Legacy color bezier

Mask	Name	Description
0x8000	LitParts	Lit particles
0x10000	RandomFlipbookStart	Random flipbook start
0x20000	MultiplyGravityByMass	Multiply gravity by mass
0x40000	ClampTailLength	Clamp tail length
0x80000	SpawnTrailingParticles	Spawn trailing particles
0x100000	FixTailLengthOnCreation	Fix tail length on creation
0x200000	UseVertexAlpha	Use vertex alpha
0x400000	ModelParticles	Use model particles
0x800000	SwapYZOnModelParticles	Swap Y/Z on model particles
0x1000000	ScaleTimeByParent	Scale time by parent
0x2000000	UseLocalTime	Use local time
0x4000000	SimulateInit	Simulate on initialization
0x8000000	Copy	Copy emitter
0x10000000	RequiresGpuSim †	Part of the `b_useProceduralPosition` trigger mask (`0x10480003`) — see *Shader Permutation Flags* below
0x40000000	ShaderPerm30 †	Toggles a particle shader permutation flag in `SetupShaderFlags` (exact role TBD)
0x80000000	ForceProceduralPosition †	Forces the GPU procedural-position path (`b_useProceduralPosition`)

† Bits ≥ `0x10000000` are consumed by the engine's `CParticleSystem::SetupShaderFlags` (reverse-engineered from SC2 build 71061) but are **not** part of the community-documented flag set — names above are provisional.

Additional Flags (`additionalFlags` field, v17+):

Mask	Name	Description
0x1	EmitSpeedRandomize	Randomize emission speed
0x2	LifespanRandomize	Randomize lifespan
0x4	MassRandomize	Randomize mass
0x8	WorldSpace	World-space coordinates

Rotation Flags (`rotationFlags` field, v18+):

Mask	Name	Description
0x2	Relative	Relative rotation
0x4	AlwaysSet	Always set

Interpolation Mode Enum (shared by `colorSmoothing`, `sizeSmoothing`, `rotationSmoothing` in PAR_ v14+ and RIB_ v8+; maps to `RibbonParticleCommon.fx`):

Value	Name	Description
0	Linear	Linear interpolation between keyframes
1	LinearSmooth	Linear interpolation with smoothed transitions
2	Bezier	Bezier curve interpolation
3	LinearWithHold	Linear interpolation with hold at keyframes
4	BezierWithHold	Bezier curve interpolation with hold at keyframes

Shader Permutation Flags (`Particle.fx` / `VSEmitters.fx`)

Each emitter compiles a shader permutation. The engine's `CParticleSystem::SetupShaderFlags` (RE'd from SC2 base71061) derives these compile-time flags from `PAR_`; they select which `Particle.fx` code paths are compiled (validated against the SC2 shader source):

`PAR_` source	Shader permutation flag	Effect
`instanceType`	`b_iInstanceType`	Geometry path; values are the *Instance Type* enum (`PARTICLE_BILLBOARD`=0 … `PARTICLE_TRAIL`=10)
`colorSmoothing`	`b_iParticleColorInterpolation`	Color keyframe interpolation (*Interpolation Mode* enum → `ITERPOLATION_*`)
`sizeSmoothing`	`b_iParticleSizeInterpolation`	Size interpolation
`rotationSmoothing`	`b_iParticleRotationInterpolation`	Rotation interpolation
`additionalFlags & WorldSpace(0x8)`	`b_localSpace` (**inverted**)	When set, the VS multiplies particle position by the emitter world transform
`flags & RandomFlipbookStart(0x10000)`	`b_randomFlipBookStart`	Randomizes the flipbook start cell
`flags & ClampTailLength(0x40000)` and `instanceType != Pinned(9)`	`b_clampedTailLength`	Clamps tail length to the swept distance
`flags & FixTailLengthOnCreation(0x100000)`	`b_fixedTailLength`	Fixed (creation-time) tail length vs. speed-scaled
`flags & LitParts(0x8000)`	lit pixel-shader variant	Selects the lit shading path
emitter complexity (below)	`b_useProceduralPosition`	VS runs `CalculatePositionAndVelocity` (GPU gravity/drag/velocity integration) instead of using CPU-computed positions

`b_useProceduralPosition` is enabled when the emitter needs runtime physics — `instanceType` ∈ {`TerrainOriented`(5), `TerrainDirOriented`(6)}, any bit of `flags & 0x10480003` (`Sort` | `CollideTerrain` | `SpawnTrailingParticles` | `ModelParticles` | `0x10000000`), nonzero force channels, or nonzero `windMultiplier` / `killRadius` / `noiseAmplitude` / `flags & 0x80000000`.

Other particle shader flags not driven by a single `PAR_` bit: `b_useModelInstancing` (model-geometry particles), `b_iUVMapping[8]` / `b_UVRandomOffsetEnable[8]` / `b_iUVEmitterCount` (per-layer UV emission, incl. `UVMAP_PARTICLE_FLIPBOOK`=6), `b_splatTextureProjectorEnabled`, `b_enableVertexWarps`, `b_usePackedUVEmitter`.

12.2 PARC — Particle Emitter Copy

Tag: `PARC` | **Version**: v0 = 40 bytes

Overrides emission rate and squirt amount of a referenced PAR_.

```
struct PARC {
    AnimRef<f32>    emissionRate;
    AnimRef<u16>    squirtAmount;
    u32             boneIndex;
};
```

12.3 RIB_ — Ribbon Emitter

Tag: `RIB_` | **Versions**: v4=744, v5=748, v6=748, v8=756, v9=760 bytes

Ribbon/trail particle systems that generate connected strips.

```
struct RIB {
    // Identification
    u16                 boneIndex;
    u16                 boneIndexFallback;
    u32                 materialIndex;
    if (version >= 8) {
        u32             additionalFlags;    // RibbonAdditionalFlag (v8+)
    }

    // Initial velocity
    AnimRef<f32>        initialSpeed;
    AnimRef<f32>        initialSpeedRandom;
    if (version <= 6) {
        u32             speedRandomize;
    }

    // Order differs by version: v6- = pitch->yaw, v8+ = yaw->pitch
    AnimRef<f32>        initialYaw;
    AnimRef<f32>        initialPitch;
    AnimRef<f32>        initialHorizontal;
    AnimRef<f32>        initialVertical;

    // Lifetime
    AnimRef<f32>        lifetime;
    AnimRef<f32>        lifetimeRandom;
    if (version <= 6) {
        u32             lifespanRandomize;
    }
    u32                 killRadius;

    // Gravity
    f32                 gravityX;
    f32                 gravityY;
    f32                 gravity;
```

```
if (version >= 6) {
    // Midpoint timing
    f32             sizeMidTime;
    f32             colorMidTime;
    f32             alphaMidTime;
    f32             rotationMidTime;
}
if (version >= 8) {
    // Midpoint hold timing
    f32             sizeMidHoldTime;
    f32             colorMidHoldTime;
    f32             alphaMidHoldTime;
    f32             rotationMidHoldTime;
}

// Curves
AnimRef<Vector3f>   sizeAnimation;
if (version <= 5) {
    f32             sizeMidTimeLegacy;
}
AnimRef<Vector3f>   rotationAnimation;
if (version <= 5) {
    f32             rotationMidTimeLegacy;
}
AnimRef<ColorBGRA>  colorStart;
AnimRef<ColorBGRA>  colorMid;
AnimRef<ColorBGRA>  colorEnd;
if (version <= 5) {
    f32             colorMidTimeLegacy;
    f32             alphaMidTimeLegacy;
}

// Physics
f32                 drag;
if (version <= 6) {
    u32             massRandomize;
}
f32                 mass;
f32                 massRandom;
f32                 massSizeMultiplier;
if (version <= 6) {
    u32             worldSpace;
}

// Forces
u16                 localForces;
u16                 worldForces;
u16                 localForcesFallback;
u16                 worldForcesFallback;
if (version >= 9) {
    f32             worldForcesMassMultiplier;
}

// Noise
f32                 noiseAmplitude;
f32                 noiseFrequency;
f32                 noiseCoherence;
f32                 noiseEdge;
```

```
    if (version >= 5) {
        u32             indexPlusLength;
    }

    // Shape
    u32                 emitterShape;
    RibbonType          ribbonType;         // RibbonType enum (see below)
    f32                 divisions;
    u32                 edges;
    f32                 innerRadius;
    AnimRef<f32>        maxLength;
    if (version <= 6) {
        i32             unknown3fbae7d6;
    }

    // References
    Ref<SRIB>           splineRibbons;
    AnimRef<u32>        active;

    // Flags
    Flag                flags;
    if (version >= 8) {
        InterpolationMode sizeSmoothing;    // InterpolationMode enum
        InterpolationMode colorSmoothing;   // InterpolationMode enum
    }

    // Collision and LOD
    f32                 friction;
    f32                 bounce;
    u32                 lodReduce;
    u32                 lodCut;

    // Order differs by version: v6- = pitch->yaw, v8+ = yaw->pitch
    u32                 yawType;
    AnimRef<f32>        yawAmplitude;
    AnimRef<f32>        yawFrequency;
    u32                 pitchType;
    AnimRef<f32>        pitchAmplitude;
    AnimRef<f32>        pitchFrequency;

    // Length / scale / alpha variation
    u32                 speedType;
    AnimRef<f32>        speedAmplitude;
    AnimRef<f32>        speedFrequency;
    u32                 sizeType;
    AnimRef<f32>        sizeAmplitude;
    AnimRef<f32>        sizeFrequency;
    u32                 alphaType;
    AnimRef<f32>        alphaAmplitude;
    AnimRef<f32>        alphaFrequency;

    // Tail controls
    AnimRef<f32>        particleVelocity;
    AnimRef<f32>        overlay;
};
```

RIB_ Main Flags:

Mask	Name	Description
0x02	collideTerrain	Collide with terrain
0x04	collideObjects	Collide with objects
0x08	edgeFalloff	Fade edges
0x10	inheritParentVelocity	Inherit parent velocity
0x20	smoothSize	Smooth size
0x40	bezierSmoothSize	Bezier smooth size
0x80	useVertexAlpha	Use vertex alpha
0x100	scaleTimeByParent	Scale time by parent
0x200	forceCPUSim	Force CPU simulation
0x400	localTime	Use local time
0x800	simulateInit	Simulate on init
0x1000	useLengthAndTime	Use length and time
0x2000	accurateGPUTangents	Accurate GPU tangents
0x4000	yawFromSpeed	Derive yaw from speed
0x8000	useLocator	Use locator node

RIB_ Additional Flags (`additionalFlags` field, v8+):

Mask	Name	Description
0x1	speedRandomize	Randomize emission speed
0x2	lifespanRandomize	Randomize lifespan
0x4	massRandomize	Randomize mass
0x8	worldSpace	World-space coordinates

Ribbon Type Enum (`ribbonType` field; maps to `b_iRibbonType` in `Ribbon.fx`):

Value	Name	Description
0	Billboard	Camera-facing ribbon strip
1	Planar	Flat/planar ribbon
2	Cylinder	Cylindrical cross-section
3	Star	Star-shaped cross-section

Spline Ribbon (SRIB, 272 bytes):

```
struct SplineRibbon {
    Vector3f        emissionOffset;
    Vector3f        emissionVector;
    AnimRef<f32>    velocity;
    u32             reserved;            // always 0 (padding/reserved)
```

```
    u32             boneIndex;
    AnimRef<f32>    velocityBaseFactor;
    AnimRef<f32>    velocityEndFactor;
    u32             yawType;
    AnimRef<f32>    yawAmplitude;
    AnimRef<f32>    yawFrequency;
    u32             pitchType;
    AnimRef<f32>    pitchAmplitude;
    AnimRef<f32>    pitchFrequency;
    u32             velocityType;
    AnimRef<f32>    velocityAmplitude;
    AnimRef<f32>    velocityFrequency;
    AnimRef<f32>    yaw;
    AnimRef<f32>    pitch;
    f32             emissionVectorNormFactor; // precomputed ≈ 0.01/|emissionVector|
    f32             velocityNormFactor;       // precomputed ≈ 0.01/velocity.initValue
};
```

RIB_ v9 — runtime-verified byte offsets (informative). Reverse-engineered from the retail client (`base71061`): the loader parses `RIB_` (v9) as a 760-byte record and the runtime `CRibbon` stores the pointer at `CRibbon+616` (`pEmitterData`). The offsets below were confirmed against the loader (`M3_ResolveRefs_RIB`) and the `CRibbon` simulation code (IDA structs `SRibbonData` / `SplineRibbonData`). `AnimRef<f32>` = 20 B, `AnimRef<Vector3f>` = 36 B, `AnimRef<ColorBGRA>` = 20 B, `Ref` = 12 B.

Offset	Field	Used by CRibbon for
0x008 (8)	`additionalFlags`	`&8` = world-space (segment dragging / local-space force eval)
0x0B0–0x0B8 (176–184)	`gravityX/Y/gravity`	per-segment acceleration (Simulate_Type4)
0x160 (352)	`drag`	`dragCoeff = drag·dt` velocity damping
0x194 (404)	`ribbonType`	cross-section shape (0=Billboard,1=Planar,2=Cylinder,3=Star); loop gate
0x198 (408)	`divisions`	emission-rate density (segments/unit)
0x1A4 (420)	`maxLength` (AnimRef)	arc-length sample → segment re-spacing
0x1B8 (440)	`splineRibbons` (Ref<SRIB>)	spline-ribbon control data
0x1C4 (452)	`active` (AnimRef)	per-frame on/off emission gate (UpdateEmit)
0x1D8 (472)	`flags`	collide (0x02/0x04), yawFromSpeed (0x4000), inheritParentVelocity (0x10)
0x1DC/0x1E0 (476/480)	`sizeSmoothing`/`colorSmoothing`	batch-compatibility key
0x1EC/0x1F0 (492/496)	`lodReduce`/`lodCut`	quality/detail table indices

The `CRibbon` runtime object also **caches** several of these into its own fields at first use (e.g. the sampled emission rate and emit period), which is why the per-frame emit path reads `CRibbon+364`/`+924` rather than re-reading `RIB_` every frame. The parsed `RIB_` pointer lives at `CRibbon+616` (`pEmitterData`), and the resolved **`SRIB` (spline-ribbon) data pointer lives at `CRibbon+624`** (`splineData`) — the spline simulation path reads its `yawType` (+92), `pitchType` (+136),

`velocityType` (+180) and the precomputed `emissionVectorNormFactor` (+264) / `velocityNormFactor` (+268) directly from that `SplineRibbonData` record.

12.4 PROJ — Projector

Tag: `PROJ` | **Versions**: v4=388, v5=388 bytes

Projected textures / decals. Contains no `Ref<T>` fields.

```
struct PROJ {
    u32             projectionType;         // ProjectionType enum (see below)
    u32             boneIndex;
    u32             materialReferenceIndex;
    AnimRef<Vector3f> offset;               // 36 bytes — animated 3D offset
    AnimRef<f32>    pitch;                  // animated pitch angle
    AnimRef<f32>    yaw;                    // animated yaw angle
    AnimRef<f32>    roll;                   // animated roll angle
    AnimRef<f32>    fieldOfView;
    AnimRef<f32>    aspectRatio;
    AnimRef<f32>    near;                   // near clip distance
    AnimRef<f32>    far;                    // far clip distance
    AnimRef<f32>    boxOffsetZBottom;       // box projection -Z
    AnimRef<f32>    boxOffsetZTop;          // box projection +Z
    AnimRef<f32>    boxOffsetXLeft;         // box projection -X
    AnimRef<f32>    boxOffsetXRight;        // box projection +X
    AnimRef<f32>    boxOffsetYFront;        // box projection -Y
    AnimRef<f32>    boxOffsetYBack;         // box projection +Y
    f32             falloff;
    f32             alphaInit;              // alpha at start
    f32             alphaMid;               // alpha at midpoint
    f32             alphaEnd;               // alpha at end
    f32             lifetimeAttack;         // fade-in start
    f32             lifetimeAttackTo;       // fade-in end
    f32             lifetimeHold;           // hold start
    f32             lifetimeHoldTo;         // hold end
    f32             lifetimeDecay;          // fade-out start
    f32             lifetimeDecayTo;        // fade-out end
    f32             attenuationDistance;    // distance attenuation
    AnimRef<u32>    active;                 // alive/active toggle
    u32             layer;
    u32             lodReduce;
    u32             lodCut;
    Flag            flags;                  // ProjectorFlag bitfield (see below)
};
```

Projection Type Enum (`projectionType` field):

Value	Name
0	Orthographic
1	Perspective

Projector Flags (`flags` field):

Mask	Name	Description
0x1	static	Static position

---

13. Physics

13.1 FOR_ — Force

Tag: FOR_ | **Versions**: v1=104, v2=104 bytes

```
struct FOR {
    u32             forceType;
    u32             forceShape;
    u32             unknown;
    u32             boneIndex;
    Flag            flags;
    u32             channels;
    AnimRef<f32>    strength;
    AnimRef<f32>    width;
    AnimRef<f32>    height;
    AnimRef<f32>    length;
};
```

FOR_ Flags:

Mask	Name	Description
0x01	falloff	Distance falloff
0x02	heightGradient	Height gradient
0x04	unbounded	Unbounded range

13.2 WRP_ — Warp

Tag: WRP_ | **Version**: v1 = 132 bytes

```
struct WRP {
    u32             warpType;
    u32             boneIndex;
    u32             unknown;
    AnimRef<f32>    radius;
    AnimRef<f32>    height;
    AnimRef<f32>    strength;
    AnimRef<f32>    angular;
    AnimRef<f32>    axial;
    AnimRef<f32>    radial;
};
```

13.3 PHRB — Rigid Body

Tag: PHRB | **Versions**: v2=104, v3=56, v4=80 bytes

```
struct PHRB {
    if (version <= 2) {
        // Legacy Havok-era layout (80-byte base + post-ref fields)
        f32             density;            // Mass density
        f32             friction;           // Surface friction
        f32             restitution;        // Bounce / elasticity
        f32             linearDamping;      // Linear velocity damping
        f32             angularDamping;     // Angular velocity damping
        f32             gravityScale;       // Gravity multiplier
        f32             inertiaTensor[3][3];// 3x3 symmetric inertia tensor (36 bytes)
        u16             parentBoneIndex;    // Parent/anchor bone
        u16             boneIndex;          // Legacy bone index (typically ==
parentBoneIndex)
        u32             reserved[4];        // Always 0 (16 bytes)
    }
    if (version >= 3) {
        u16             simulationType;     // Simulation mode
        u16             parentBoneIndex;    // Parent/anchor bone
        u32             physicsType;        // Engine-specific rigid body type
        f32             density;            // Mass density
        f32             friction;           // Surface friction
        f32             restitution;        // Bounce / elasticity
        f32             linearDamping;      // Linear velocity damping
        f32             angularDamping;     // Angular velocity damping
        f32             gravityScale;       // Gravity multiplier
    }
    if (version >= 4) {
        AnimRef<u32>    dynamicState;       // Animated dynamic/static state
        f32             dynamicBlendOut;    // Dynamic state blend-out factor
    }
    // Present in all versions
    Ref<PHSH>           rigidBodyShape;     // Reference to physics shape
    Flag                flags;              // RigidBodyFlag bitmask
    u16                 localForces;        // Local force channel bitmask
    u16                 worldForces;        // World force channel bitmask
    u32                 priority;           // Solver/update priority
};
```

Parser behavior summary:

- `version <= 2`: reads legacy Havok-era layout with 80-byte base (including inertia tensor) + 12-byte Ref + 12-byte post-ref fields = **104 bytes total**
- `version == 3`: reads modern layout without `dynamicState`/`dynamicBlendOut` = **56 bytes**
- `version >= 4`: reads full v4+ layout with animated dynamic state = **80 bytes**

PHRB Flags:

Mask	Name	Description
0x0001	collidable	Can collide
0x0002	walkable	Walkable surface
0x0004	stackable	Can be stacked
0x0008	simulateCollision	Simulate collisions
0x0010	ignoreLocalBodies	Ignore local bodies

Mask	Name	Description
0x0020	alwaysExists	Always present
0x0040	unknown6	Unknown (16.2% of entries have this set)
0x0080	noSimulation	Disable simulation
0x0200	unknown9	Unknown (5.3% of entries have this set)

Most common flag combinations: 0x0021 (37.4%), 0x0025 (22.2%), 0x0061 (16.2%), 0x0065 (8.0%), 0x0221 (5.3%). 37 distinct flag values observed in the corpus.

PHRB World Forces:

Mask	Name
0x01	wind
0x02	explosion
0x04	energy
0x08	blood
0x10	magnetic
0x20	grass
0x40	brush
0x80	trees

Rigid Body Shape (PHSH): v1=132, v2=292, v3=300 bytes

Parser-aligned versioned layout (`visit(PhysicsShape&)`):

- v1 and below use a legacy-only branch
- v2+ use the modern layout with hull/mesh sections
- v3+ mesh references are interleaved with the hull section
- v2 mesh references follow `meshTolerance`, replacing the v3+ positions

```
struct PHSH {
    Matrix44f   transform;

    if (version <= 1) {
        // Legacy v1 layout
        f32         collisionMargin;    // Havok convex radius
        u8          shapeType;          // Shape type enum
        u8          padding[3];
        Ref<VEC3>   legacyVertices;
        Ref<U8__>   unknown0;
        Ref<U16_>   faceIndices;
        Ref<VEC4>   planeEquations;
        Vector3f    halfExtents;
    }

    if (version >= 2) {
        // Modern v2+ layout
        u8          shapeType;          // 0=box, 1=sphere, 2=capsule, 3=cylinder,
```

```
4=convex_hull, 5=mesh
        u8          padding[3];
        Vector3f    oldSizes;           // v1 leftover, usually zero

        // Convex/simple-shape section
        Ref<?>      reserved0;          // Always empty
        Vector3f    shapeDimensions;    // Shape dimensions for types 0-3, zero for 4-5
        Ref<VEC3>   hullFaceNormals;    // Per-face normals (shapeType=4)
        Ref<VEC4>   hullVertexPositions;// Vertex positions (w=0)
        Ref<DMSE>   hullHalfEdges;      // Half-edge connectivity
        Ref<U8__>   hullVertexFaceIndices;// One face index per vertex
        Vector3f    hullCenter;         // Hull centroid
        u32         hullFaceNormalCount;
        u32         hullVertexCount;
        u32         hullHalfEdgeCount;
        f32         hullUnknown0;
        f32         hullUnknown1;
    }

    if (version >= 3) {
        // v3+ mesh section
        Ref<DMMN>   meshBvhNodes;       // k-DOP BVH tree nodes (see §14.9b)
        Ref<VEC4>   meshVertexPositions;// Vertex positions in mesh-local space (w=0)
        Ref<MT16>   meshFaceIndices16;  // 16-bit triangle indices (or empty)
        Ref<MT32>   meshFaceIndices32;  // 32-bit triangle indices (or empty)
    }

    if (version >= 2) {
        // Mesh AABB / quantization grid
        Vector3f    meshBoundsCenter;   // AABB center in model space (quantization grid
origin)
        Vector3f    meshBoundsExtent;   // AABB half-extents (defines quantization range)
        Vector3f    meshTolerance;      // Per-axis quantization step (= extent / 32767)
    }

    if (version == 2) {
        // v2 mesh section (entry ends at 292 bytes, not 300)
        Ref<DMMN>   meshBvhNodes;
        Ref<VEC3>   meshVertexPositions;// VEC3, not VEC4
        Ref<DMMT>   unknown;            // Physics mesh triangles
        Ref<DMME>   unknown2;           // Physics mesh edges
        // 6-dword v2 tail, mapped per the SC2 client's version-upgrade
        // copier (M3_ProcessChunks): the vertex/face counts land in the v3
        // count slots (faces in the 32-bit slot), tree depth keeps its
        // meaning, and the client never reads the three unknown dwords.
        u32         tailUnknown0;
        u32         meshVertexCount;
        u32         meshFaceCount;      // → v3 meshFaceIndex32Count on upgrade
        u32         tailUnknown1;
        u32         tailUnknown2;
        u32         meshTreeDepth;
    }

    if (version >= 3) {
        // v3 mesh counters
        u32         meshNormalCount;    // DMMN entry count (= 2 * n_leaves - 1, always
odd)
        u32         meshVertexCount;
        u32         meshFaceIndex16Count;// MT16 face count (0 when MT32 used)
```

```
        u32         meshFaceIndex32Count;// MT32 face count (0 when MT16 used)
        u32         meshUnknown1;
        u32         meshReserved;       // Always 0
        u32         meshTreeDepth;      // BVH tree height (root-to-leaf path length, 1–12)
        f32         meshCollisionMargin;// MT16: small float; MT32: 0.0
    }
};
```

> **Mesh face indices**: exactly one of `meshFaceIndices16` (MT16) or `meshFaceIndices32` (MT32) is populated — never both. In the corpus, 536 of 558 mesh shapes use MT32 at offset 220; the remaining 22 use MT16 at offset 208. Each entry is one triangle stored as 7 values (u32 for MT32, u16 for MT16): three vertex indices followed by three adjacency / edge-welding values and one flags word (typically 0).
>
> The scalar fields at offsets 276–296 differ systematically by index type:
>
> - **MT32 meshes**: `meshFaceIndex16Count`=0, `meshFaceIndex32Count`=N, `meshCollisionMargin`=0.0
> - **MT16 meshes**: `meshFaceIndex16Count`=N, `meshFaceIndex32Count`=0, `meshUnknown1`=N+3, `meshCollisionMargin`=small positive float
>
> **Simple shape dimensions** (corpus-validated, 19,026 entries): the `unused0` field at offset 68 is always (0,0,0) in v3. Actual dimensions are stored at offset 92 as 3 floats overlapping the `shapeDimensions Ref<T>` slot.
>
> **Shape type distribution** (23,931 PHRB → PHSH entries): capsule 61.8%, convex_hull 18.2%, box 14.4%, mesh 2.3%, sphere 2.2%, cylinder 1.1%. Dynamic (simType=2) rigid bodies are rare (1.3%) and most commonly use convex_hull or box shapes.

13.4 PHYJ — Physics Joint

Tag: `PHYJ` | **Version**: v0 = 180 bytes

```
struct PHYJ {
    u32         jointType;
    u32         boneIndex1;
    u32         boneIndex2;
    Matrix44f   matrixBody1;
    Matrix44f   matrixBody2;
    u32         enableLimits;
    f32         limitMin;
    f32         limitMax;
    f32         coneAngle;
    u32         enableFriction;
    f32         friction;
    f32         dampingRatio;
    f32         angularFrequency;
    f32         breakThreshold;
    u8          enableShape;
};
```

13.5 PHCL — Cloth Physics

Tag: `PHCL` | **Versions**: v2=128, v4=192 bytes

```
struct PHCL {
    u32             clothMeshCount;
```

```
    u32             skinBoneCount;
    Ref<U16_>       skinBones;
    Ref<U8__>       simEnabled;
    Ref<U32_>       vertexBones;
    Ref<U32_>       vertexWeights;
    Ref<PHCC>       colliders;
    Ref<PHAC>       proxies;
    f32             density;
    f32             tracking;
    f32             stretchStiffness;
    f32             horizontalStiffness;
    f32             bendingStiffness;
    f32             damping;
    f32             friction;
    f32             gravity;
    f32             explosionScale;
    f32             windScale;
    f32             shearStiffness;
    f32             dragFactor;
    if (version >= 4) {
        f32             liftFactor;
        f32             sphereStiffness;
        u32             flatten;
        AnimRef<u32>    active;
        u32             useSkinCollision;
        f32             skinOffset;
        f32             skinExponent;
        f32             skinStiffness;
        u32             localChannels;
        Vector3f        localWind;
    }
};
```

Cloth Collider (PHCC, v0=76 bytes):

```
struct PHCC {
    Matrix44f   transform;
    f32         radius;
    f32         height;
    u32         padding;
};
```

Cloth Proxy (PHAC, v0=32 bytes):

```
struct PHAC {
    u32         proxyIndex;
    u32         clothIndex;
    Ref<U64_>   proxyVertices;
    Ref<U32_>   proxyWeights;
};
```

13.6 Runtime Physics System (engine behaviour, informative)

> How the engine *simulates* M3 physics. Reverse-engineered from the retail client (`base71061`); function names are RE labels. The physics engine is **Domino** — Blizzard's in-house Box2D/Box3D-lineage impulse solver (class prefix `dm`), **not Havok** (fully replaced). Domino supports both rigid bodies and soft bodies (`dmCloth`).

Chunk → Domino object mapping. At load, a model's physics chunks are instantiated into Domino objects by `CModelPhysics_Build`:

M3 chunk	Domino object	Notes
`PHRB` rigid body	`dmBody` + fixtures	one body per PHRB, bound to its bone
`PHSH` shape (`shapeType`)	`dmShape` subclass	0=box / 3=cylinder / 4=hull → `dmPolytope`; 1=sphere → `dmSphere`; 2=capsule → `dmCapsule`; 5=mesh → `dmTreeMesh` (BVH)
`PHYJ` joint (`jointType`)	`dmJoint`	0=spherical, 1=revolute, 2=shoulder/cone-twist, 3=weld
`PHCL` / `PHCC` / `PHAC`	`dmCloth` + collider capsules/spheres + anchors	mass-spring / position-based-dynamics cloth

Each rigid body's fixture carries a collision filter (category/mask/group) derived from the PHRB flags and the body's dynamic/kinematic/static type.

Lifecycle.

1. **Build** — `CModel_EnsurePhysics` → `Model_CreatePhysicsInstance` → `CModelPhysics_Build` iterates `rigidBodies` (PHRB), `physicsJoints` (PHYJ), and `clothPhysics` (PHCL). A `dmWorld` is shared per scene (`CModelPhysicsScene_EnsureWorld`).
2. **Per-frame sync** — `CModelPhysicsWorld_Update` runs, per instance: pre-step sync (animated/kinematic bones → `dmBody`), `dmWorld::Step` × N substeps, then **`CModelPhysics_PostStepSyncToBones`** — the ragdoll step: it reads each dynamic `dmBody`'s position + orientation and writes the simulated world transform straight into the M3 **bone/node matrix**, so the animated skeleton is driven by the physics solve. Environment forces (wind/fluid/force volumes) are applied via `dmBody::ApplyForce`, and cloth colliders follow the bones.
3. **Teardown** — `Model_DestroyPhysicsInstance` → `CModelPhysics_DestroyAll` destroys all joints, bodies, and cloth.

A per-bone state machine (`M3Physics_UpdateBodyDrivenState`) chooses, from the PHRB flags and an animated blend value, whether each bone is **dynamic** (physics drives the bone) or **kinematic** (animation drives the body), switching the `dmBody` type accordingly — this is how a unit blends from animation into a ragdoll on death. The gameplay impact/damage layer (`CActorMsgPhysicsImpact`, `CActorTermPhysicsImpact*`) applies impulses to these bodies.

13.6.1 Cloth solver (`dmCloth`, informative)

`PHCL`/`PHCC`/`PHAC` are instantiated into a **`dmCloth`** object — a **Position-Based Dynamics** (PBD) solver, not a rigid body. `CModelPhysics_Build` gathers the authored cloth data (particles, triangles, colliders, anchors) plus the scene gravity into an `SPhysicsClothData` block and hands it to `dmCloth_Create`, which allocates one arena and precomputes per-particle weights (`dmCloth_InitParticleParams`): inverse mass scaled by `dt²`, constraint rest lengths, and a `powf`-curve **tether** look-up table.

The solver stores, per particle, a current position, previous position, velocity, inverse mass, a **pin bit** (kinematic particles the solver never moves), and a render normal/tangent frame. Particles `[0, firstDynamicParticle)` are pinned and driven directly by the model's world transform; the rest are simulated.

The cloth mesh drives three constraint families, all built from the triangle list by `dmCloth_BuildTriangles` (dedup edges → distance constraints from edges → bend constraints from adjacent-triangle pairs → island reorder):

Constraint	Endpoints	Enforces
Distance	2	edge rest length (structural stretch/shear)
Bend	4 (two triangles)	rest **dihedral angle** between adjacent faces
Tether	1 + anchor	maximum distance from an anchor (long-range attachment)

`dmCloth_Step` runs each frame in stages: **(1)** refresh world transforms → **(2)** `dmCloth_PreSolve` predictor (pinned particles snap to anchor/tether targets through the world SQT; dynamic particles integrate gravity + the inertia from the previous→current transform delta + velocity damping) → **(3)** `dmCloth_SolveConstraints` PBD projection sweeps (distance → bend → distance) → **(4)** `dmCloth_SolveTethersAndAttachments` (clamp tether distances, apply hard attachments) → **(5)** `dmCloth_ApplyAerodynamics` (per-triangle wind + drag) → **(6)** `dmCloth_CollideCapsulesAndSpheres` (project particles out of the collider and world-space capsules/spheres with friction, AABB-broadphased) → **(7)** `dmCloth_ApplyRestPose` (blend toward the rest curvature) → **(8)** derive each particle's velocity from `(current − previous) / dt`. A separate `dmCloth_UpdateParticleNormals` rebuilds the render normal/tangent frame from the triangle face normals. The mesh can also be queried (`dmCloth_SweptSphereQuery`) and kicked by explosions (`dmCloth_ApplyRadialImpulse`).

---

14. Miscellaneous Chunks

14.1 LITE — Light

Tag: `LITE` | **Version**: v7 = 212 bytes

```
struct LITE {
    u16                 lightType;          // 0=directional, 1=point, 2=spot
    u16                 boneIndex;
    Flag                flags;
    u32                 lodCut;
    u32                 shadowLodCut;
    AnimRef<Vector3f>   diffuseColor;
    AnimRef<f32>        intensityMultiplier;
    AnimRef<Vector3f>   specularColor;
    AnimRef<f32>        specularMultiplier;
    AnimRef<f32>        decay;
    f32                 attenuationEnd;
    AnimRef<f32>        attenuationStart;
    AnimRef<f32>        hotSpot;            // Radians
    AnimRef<f32>        falloff;
};
```

LITE Flags:

Mask	Name	Description
0x01	shadows	Casts shadows
0x02	specular	Specular component
0x04	ambientOcclusion	AO influence
0x08	lightOpaque	Lights opaque objects
0x10	lightTransparent	Lights transparent objects

Mask	Name	Description
0x20	teamColor	Uses team color

14.2 CAM_ — Camera

Tag: `CAM_` | **Versions**: v2=144† (β), v3=180, v4=220, v5=264 bytes

```
struct CAM {
    u32                 boneIndex;
    Ref<CHAR>           name;

    if (version >= 2) {
        AnimRef<f32>    fieldOfView;        // Radians (e.g. π/4 = 45°)
        u32             useVerticalFOV;     // Boolean (0 or 1)
    }

    if (version == 2) {
        // Beta v2 only: farClip/nearClip are plain f32 (promoted to AnimRef in loader)
        f32             farClip;            // Static far clip distance
        f32             nearClip;           // Static near clip distance
    }

    if (version >= 5) {
        u32             dofType;            // Default 3 when absent
    }

    if (version >= 3) {
        // v3+: farClip/nearClip became animatable
        AnimRef<f32>    farClip;
        AnimRef<f32>    nearClip;
    }

    if (version >= 2) {
        AnimRef<f32>    shadowClipDistance;
        AnimRef<f32>    focusDistance;      // DOF focal point distance
        AnimRef<f32>    farFocusRange;      // DOF far focus range
        AnimRef<f32>    nearFocusRange;     // DOF near focus range
    }

    if (version >= 4) {
        AnimRef<f32>    nearFalloffStart;
        AnimRef<f32>    nearFalloffEnd;
    }

    if (version >= 2) {
        AnimRef<f32>    dofAmount;          // DOF strength (0 = disabled)
    }

    if (version >= 5) {
        AnimRef<f32>    bokehFStop;         // Bokeh f-stop value
        AnimRef<f32>    bokehMaxCoCDiameter;// Max circle of confusion diameter
    }
};
```

Parser Flow (`visit(Camera&, version)`):

1. Always reads `boneIndex` (u32) and `name` (Ref)
2. `version >= 2`: reads `fieldOfView` (AnimRef) and `useVerticalFOV` (u32)
3. `version == 2`: reads plain `farClip` (f32) and `nearClip` (f32), promotes them to AnimRef in-memory
4. `version >= 5`: reads `dofType` (u32)
5. `version >= 3`: reads animatable `farClip` and `nearClip` (AnimRef)
6. `version >= 2`: reads DOF parameters: `shadowClipDistance`, `focusDistance`, `farFocusRange`, `nearFocusRange`
7. `version >= 4`: reads `nearFalloffStart` and `nearFalloffEnd`
8. `version >= 2`: reads `dofAmount`
9. `version >= 5`: reads `bokehFStop` and `bokehMaxCoCDiameter`

CAM_ v2 layout (MD33 only, 144 bytes with 8-byte SmallRef):

Offset	Size	Field	Typical values
0	4	boneIndex	Camera bone
4	8	name (SmallRef<CHAR>)	Camera name string
12	20	AnimRef<f32> fieldOfView	init: 0.27–0.79 rad (15–45°)
32	4	useVerticalFOV	0 or 1
36	4	farClip (f32)	2.8–95.6
40	4	nearClip (f32)	0.001–13.1
44	20	AnimRef<f32> shadowClipDistance	init: 1.5–40.0
64	20	AnimRef<f32> focusDistance	init: 0.0–6.0
84	20	AnimRef<f32> farFocusRange	init: 0.0–30.0
104	20	AnimRef<f32> nearFocusRange	init: 0.0–2.0
124	20	AnimRef<f32> dofAmount	init: 0.0–1.0

> **Note**: In v2 (beta), `AnimRef<T>.unused` is consistently 0 rather than the typical −1 seen in MD34 files. The v2→v3 upgrade promoted `farClip` and `nearClip` from plain `f32` values to `AnimRef<f32>` fields, adding animation support. Validated against 162 cameras across 18 SC2 beta model files.

14.3 EVNT — Event

Tag: `EVNT` | **Versions**: v1=104, v2=108 bytes

```
struct EVNT {
    Ref<CHAR>       name;
    i32             id;                 // Default -1
    i16             bone;               // Bone index (-1 = none)
    u16             boneFallback;       // Always 0
    Matrix44f       transform;
    u32             eventType;          // Default 4
    Ref<CHAR>       payload;            // actor/model name
    if (version >= 1) {
        u32         rttChannelIndex;
    }
    if (version >= 2) {
        u32         extraParameter;
```

```
    }
};
```

14.4 SSGS — Hit Test Shape

Tag: SSGS | **Version**: v1 = 108 bytes

```
struct SSGS {
    u32         shapeType;          // 0=box, 1=sphere, 2=capsule, 3=cylinder, 4=mesh
    u16         boneIndex;
    u16         padding;
    Matrix44f   transform;
    Ref<VEC3>   vertexPositions;
    Ref<U16_>   faceIndices;
    f32         sizeX;
    f32         sizeY;
    f32         sizeZ;
};
```

14.4.1 ATVL — Attachment Volume

Tag: ATVL | **Version**: v0 = 116 bytes

Similar to SSGS but with two additional bone indices.

```
struct ATVL {
    u32         bone1;
    u32         bone2;
    u32         shapeType;
    u16         boneIndex;
    u16         padding;
    Matrix44f   transform;
    Ref<VEC3>   vertexPositions;
    Ref<U16_>   faceIndices;
    f32         sizeX;
    f32         sizeY;
    f32         sizeZ;
};
```

14.5 TRGD — Trigger Data

Tag: TRGD | **Version**: v0 = 24 bytes

```
struct TRGD {
    Ref<U32_>   dataIndices;
    Ref<CHAR>   name;
};
```

14.6 PATU — Turret Behavior

Tag: PATU | **Versions**: v1=112 bytes, v4=152 bytes

Contains no `Ref<T>` fields.

```
struct PATU {
    Matrix44f   transform;
    if (version >= 4) {
        Vector4f unknown1;
        Vector4f unknown2;
    }
    u16         boneIndex;
    u8          useAsMainTurret;
    u8          turretGroupId;
    u32         yawLimited;
    f32         yawMin;
    f32         yawMax;
    if (version >= 4) {
        f32     yawWeight;
    }
    u32         pitchLimited;
    f32         pitchMin;
    f32         pitchMax;
    if (version >= 4) {
        f32     pitchWeight;
    }
    f32         unknown3;
    f32         unknown4;
    Vector3f    mainBoneOffset;
};
```

14.7 BBSC — Billboard Behavior

Tag: `BBSC` | **Version**: v0 = 48 bytes

```
struct BBSC {
    Ref<U16_>   dependents;
    u16         boneIndex;
    u8          billboardType;
    u8          cameraLookAt;
    Quaternion  up;
    Quaternion  forward;
};
```

14.8 IKJT — IK Joint

Tag: `IKJT` | **Version**: v0 = 32 bytes

```
struct IKJT {
    Ref<U16_>   dependents;
    u16         boneIndex1;
    u16         boneIndex2;
    f32         raycastUp;
    f32         raycastDown;
    f32         maxSpeed;
```

```
    f32         goalThreshold;
};
```

14.9 SHBX — Shadow Box

Tag: `SHBX` | **Version**: v0 = 64 bytes

```
struct SHBX {
    Matrix44f matrix;
};
```

14.9b DMMN — Physics Mesh BVH Node

Tag: `DMMN` | **Versions**: v0 = 12 bytes, v1 = 8 bytes

DMMN entries form a linearized **k-DOP Bounding Volume Hierarchy** for concave mesh collision detection (PHSH `shapeType = 5`).

```
// v0 — slab normal direction only (Havok-era, no quantized bounds)
struct DMMN_v0 {
    Vector3f    normal;             // Slab normal direction (unit vector)
};

// v1 — octahedral-encoded normal + quantized slab bounds (Domino physics)
struct DMMN_v1 {
    i16         octX;               // Octahedral-mapped normal X (snorm16)
    i16         octY;               // Octahedral-mapped normal Y (snorm16)
    u16         slabMin;            // Quantized bounding-slab min distance
    u16         slabMax;            // Quantized bounding-slab max distance (0 = leaf
sentinel)
};
```

v0 vs v1: v0 stores only the slab normal direction as a plain `Vector3f` (12 bytes). No precomputed slab bounds are present — the runtime must project vertices against `meshBoundsCenter`/`meshBoundsExtent` to compute slab distances on the fly. The tree topology is identical to v1. Only 3 corpus files use v0 (all paired with PHSH v2). v1 compresses the normal into an octahedral encoding and adds precomputed quantized slab bounds, enabling fast rejection without per-query projection.

Octahedral decoding (v1):

```
x = octX / 32767.0
y = octY / 32767.0
z = 1.0 - |x| - |y|
if z < 0:  x = (1 - |y|) * sign(x),  y = (1 - |x|) * sign(y)
normalize(x, y, z)
```

Tree structure — right-skewed binary tree stored in DFS preorder:

- The array has n = 2 × n_leaves − 1 entries (always odd).
- Layout: (INT_0, $LEAF_1$), (INT_2, $LEAF_3$), ..., LEAF_{n−1}
- Even indices 0 ... n−3: **internal nodes** (v1: slabMax ≠ 0).

- Odd indices 1 … n−2: **leaf nodes** (v1: slabMax = 0 as sentinel).
- Last index n−1: **leaf node** (v1: may have slabMax ≠ 0 despite being a leaf).
- Each internal node at index 2k has: left child = 2k+1 (leaf), right child = 2k+2 (internal or final leaf).

Each internal node defines one bounding slab (a pair of parallel planes along the node's normal direction). Its paired leaf uses a **different** normal direction, forming a 2-DOP bound per primitive group. Most trees (391/468 in corpus) use multiple distinct slab normals across internal levels for tighter culling.

Quantization (v1 only) — universally confirmed across the corpus (468 files):

- `meshTolerance = meshBoundsExtent / 32767` (per-axis quantization step)
- Projected step: `tol_proj = dot(meshTolerance, |normal|)`
- Slab distances quantized as: `q = round(projection / tol_proj)`
- Root node slab range approaches [−32767, +32767] (the full AABB span)

PHSH `meshTreeDepth` gives the tree height (longest root-to-leaf path length in nodes, range 1–12). This is NOT the leaf count; `n = 2 × meshTreeDepth − 1` holds only for perfectly right-skewed trees (91/468 files).

Corpus: 561 files — 3 files have v0, 558 files have v1.

14.10 DMMT — Physics Mesh Triangle

Tag: `DMMT` | **Version**: v0 = 28 bytes

Winged-edge triangle for physics meshes.

```
struct DMMT {
    u32     vertexIndex0;
    u32     vertexIndex1;
    u32     vertexIndex2;
    u32     edgeIndex0;             // v0⇔v1 (index into DMME)
    u32     edgeIndex1;             // v1⇔v2
    u32     edgeIndex2;             // v2⇔v0
    u16     reserved;               // Always 0
    u16     flags;                  // Material / break-group ID
};
```

14.11 DMME — Physics Mesh Edge

Tag: `DMME` | **Version**: v0 = 20 bytes

Winged-edge topology edge. Boundary edges use `faceB = 0xFFFFFFFF`.

```
struct DMME {
    u32     edgeType;               // 0–2: interior; 3: boundary
    u32     vertexA;
    u32     vertexB;
    u32     faceA;                  // Index into DMMT
    u32     faceB;                  // 0xFFFFFFFF for boundary
};
```

Edge type values:

Value	Meaning
0	Interior (classification A)
1	Interior (classification B)
2	Interior (classification C)
3	Boundary edge

14.12 DMSE — Convex Hull Half-Edge

Tag: `DMSE` | **Version**: v0 = 4 bytes

Half-edge connectivity table for convex hull physics shapes (PHSH `shapeType = 4`). Each entry encodes one half-edge of the hull's surface triangulation. Entries are stored in consecutive twin pairs: even entries carry `type = 0x01` (forward) and odd entries carry `type = 0xFF` (reverse), representing the two directed half-edges of the same undirected edge.

The `nextAroundVertex` field forms closed per-vertex ring cycles: following the chain `entry[i].nextAroundVertex → entry[j].nextAroundVertex → …` visits every half-edge that shares the same target vertex and returns to the starting entry.

```
struct DMSE {
    u8      type;               // 0x01 = forward, 0xFF = reverse (twin direction)
    u8      faceIndex;          // face this half-edge borders (0-based)
    u8      vertexIndex;        // target vertex of this half-edge (0-based)
    u8      nextAroundVertex;   // index of next half-edge sharing the same vertex
};
```

Structural invariants (verified across corpus):

- Entry count = 2 × edge count.
- Euler's formula holds: V − E + F = 2 (where V = max `vertexIndex` + 1, F = max `faceIndex` + 1, E = count / 2).
- `nextAroundVertex` is a permutation of 0 … count−1; every cycle contains entries with the same `vertexIndex`.
- Each twin pair has distinct `faceIndex` and distinct `vertexIndex` values.
- Faces are typically triangles (3 half-edges) but polygons with more edges are valid; one corpus specimen has a pentagonal face (5 half-edges).

Corpus: 4 files — all have PHSH v3 with `shapeType = 4` (convex hull).

File	Corpus	V	E	F	DMSE entries	Notes
`Storm_Hero_D3WitchDoctorM_Base.m3`	HotSM3	8	18	12	36	All tris
`Storm_Hero_Zeratul_HighTemplar.m3`	HotSM3	15	36	23	72	Mixed 3/4-gons
`Ghost_Heavens_COOP_DeathRagdoll.m3`	Sc2M3	15	39	26	78	All tris
`test.m3`	StarM3	8	16	10	32	9 tris + 1 pentagon

14.13 VVOL — View Volume

Tag: `VVOL` | **Version**: v0 = 40 bytes

Visibility volume attached to a bone for camera culling.

```
struct VVOL {
    u32                 nodeIndex;      // Index into BONE
    AnimRef<Vector3f>   size;           // Animated half-extents
};
```

---

15. Primitive Data Chunks

Arrays of a single primitive type, referenced from other chunks via `Ref<T>` fields.

Tag	Version	Element Size	Description
`CHAR`	0	1 byte	Null-terminated ASCII string
`U8__`	0	1 byte	Unsigned 8-bit (vertex blobs, weights)
`U16_`	0	2 bytes	Unsigned 16-bit (indices, bone lookups)
`U32_`	0	4 bytes	Unsigned 32-bit (animation IDs, flags)
`U64_`	0	8 bytes	Unsigned 64-bit
`I16_`	0	2 bytes	Signed 16-bit
`I32_`	0	4 bytes	Signed 32-bit
`VEC2`	0	8 bytes	Vector2f
`VEC3`	0	12 bytes	Vector3f
`VEC4`	0	16 bytes	Vector4f
`QUAT`	0	16 bytes	Quaternion
`REAL`	0	4 bytes	f32
`\0COL`	0	4 bytes	ColorBGRA (LE bytes: `4C 4F 43 00`)
`FLAG`	0	4 bytes	Flag (u32)
`BNDS`	0	28 bytes	Extent
`MT32`	0	28 bytes	Transform (7 × u32)
`MT16`	0	14 bytes	Transform (7 × u16)

---

16. Animation Data Blocks

Keyframe data is stored in typed AnimBlock structs:

```
struct AnimBlock {
    Ref<I32_> frames;           // frame times in animation ticks
    u32       flags;
    u32       endFrame;         // Last frame number
    Ref<T>    keys;             // see below
};
```

Key type mapping:

Tag	Key Type	Key Size	Description
SDEV	REAL (f32)	4	Scalar float keyframes
SD2V	VEC2	8	2D vector keyframes
SD3V	VEC3	12	3D vector keyframes
SD4Q	QUAT	16	Quaternion keyframes
SDCC	COL	4	Color keyframes
SDR3	VEC3	12	Rotation vector keyframes
SDS6	I16_	2	Signed 16-bit keyframes
SDU6	U16_	2	Unsigned 16-bit keyframes
SDU3	U32_	4	Unsigned 32-bit keyframes
SDFG	FLAG	4	Flag keyframes
SDMB	BNDS	28	Bounding sphere keyframes

The SD tag does **not** strictly determine the key data type — the `keys` reference can point to any primitive chunk. Parsers should read the actual index table entry tag to determine the key data type.

Additional typed channels:

Tag	Type	Size	Description
SVC3	AnimRef<Vector3f>	36	Static Vector3 channel
SR32	AnimRef<f32>	20	Static scalar channel
SCHR	Ref<CHAR>	12	Array of CHAR references

See §8 for the full AnimRef resolution procedure.

---

17. Complete Version/Size Table

Every chunk type with all known versions and struct sizes in bytes. Sizes are MD34 sizes unless marked with † (MD33 size — see §C.2 for MD34 conversion: add `4 × N_refs`). Versions marked **(β)** are Beta (MD33) only.

Tag	Version (hex)	Size (bytes)	Notes
MD34	0x0B	32	Header index entry
MODL	0x14	560 †	**(β)** No viewVolumes/trailing/m3a; see §C.4
	0x15	560 †	**(β)** +trailingModels/m3aAnimHash; see §C.4
	0x17	784	
	0x18	796	+ikCCD
	0x19	808	+volumeNoiseMaterials
	0x1A	820	+stbMaterials
	0x1C	844	+reflectionMaterials, +clothPhysics

Tag	Version (hex)	Size (bytes)	Notes
	0x1D	856	+lensFlareMaterials
	0x1E	868	+materialAddData
SEQS	0x01	96	2 refs
	0x02	92	
STC_	0x04	204	16 refs
STG_	0x00	24	2 refs
STS_	0x00	28	1 ref
BONE	0x01	160	1 ref
REGN	0x02	28	**(β)** No refs
	0x03	36	
	0x04	40	+flags
	0x05	48	+UV scale/offset
BAT_	0x01	14	No refs
DIV_	0x02	52	4 refs
MSEC	0x01	72	No refs
IREF	0x00	64	No refs
ATT_	0x01	20	1 ref
LITE	0x07	212	No refs
CAM_	0x02	144 †	**(β)** 1 ref; farClip/nearClip as plain f32
	0x03	180	1 ref; farClip/nearClip promoted to AnimRef
	0x04	220	+nearFalloff
	0x05	264	+DOF type, bokeh
MATM	0x00	8	No refs
MAT_	0x0F	268	14 refs
	0x10	280	15 refs
	0x11	280	15 refs
	0x12	280	15 refs
	0x13	340	20 refs
	0x14	352	20 refs + env multipliers
MADD	0x01	140	4+4 refs
	0x02	152	+extraHash ref
	0x03	160	+extraId0/1

Tag	Version (hex)	Size (bytes)	Notes
LAYR	0x14	352 †	**(β)** 1 ref; pre-v22 layer
	0x15	352 †	**(β)** 1 ref; pre-v22 layer
	0x16	356	1 ref
	0x17	428	+triplanar
	0x18	436	+noise
	0x19	468	+extended fresnel
	0x1A	464	Full featured
DIS_	0x04	68	3 refs
CMP_	0x02	28	2 refs
CMS_	0x00	24	No refs
TER_	0x00	24	2 refs
	0x01	28	2 refs
VOL_	0x00	84	3 refs
CREP	0x00	24	2 refs
	0x01	28	2 refs
VON_	0x00	268	3 refs
STBM	0x00	48	4 refs
REF_	0x01	84	Minimal
	0x02	156	
	0x03	160	Extended
LFLR	0x02	80	Basic
	0x03	152	+color, HDR, size
PAR_	0x0A	1300	**(β)** 3 refs; beta particle emitter
	0x0B	1304	**(β)** 3 refs; +indexPlusLength
	0x0C	1316	3 refs; +midTimes, massSizeMultiplier, flipbook fractions
	0x0E	1356	+holdTimes, emissionMesh, smoothings
	0x11	1460	+additionalFlags, instanceDistance, uvScreenSpace
	0x12	1464	+rotationFlags
	0x13	1464	
	0x15	1464	
	0x16	1484	+phaseShift
	0x17	1492	+spawnRibbonOnBounceChance, ribbonLinkIndex

Tag	Version (hex)	Size (bytes)	Notes
	0x18	1496	+worldForcesMassMultiplier
PARC	0x00	40	No refs
RIB_	0x04	744 †	**(β)** 1 ref; beta ribbon
	0x05	748 †	**(β)** 1 ref; beta ribbon
	0x06	748	1 ref
	0x08	756	
	0x09	760	+worldForcesMassMultiplier
SRIB	0x00	272	No refs
PROJ	0x04	388	No refs
	0x05	388	
FOR_	0x01	104	No refs
	0x02	104	
WRP_	0x01	132	No refs
PHRB	0x02	104	Legacy (80-byte base + Ref + 12-byte post-ref)
	0x03	56	Modern: simulationType, physicsType, Ref, flags, forces
	0x04	80	+dynamicState (AnimRef), +dynamicBlendOut
PHYJ	0x00	180	No refs
PHCL	0x02	128	Legacy
	0x04	192	Full
TRGD	0x00	24	2 refs
PATU	0x01	≤112	**(β)** No refs; smaller layout
	0x04	152	No refs
BBSC	0x00	48	1 ref
IKJT	0x00	32	1 ref
EVNT	0x00	92 †	**(β)** 2 refs; beta event
	0x01	104	2 refs
	0x02	108	
SSGS	0x01	108	2 refs
ATVL	0x00	116	2 refs
LFSB	0x02	56	No refs
PHSH	0x01	132	Legacy: 1 ref (transform, collisionMargin, shapeType, 4 refs, halfExtents)
	0x02	292	Modern: 10 refs (hull section + v2 mesh refs)

Tag	Version (hex)	Size (bytes)	Notes
	0x03	300	Modern: 10 refs (hull section + v3+ mesh refs); see §13.3
DMMN	0x00	12	Vector3f slab normal
	0x01	8	k-DOP BVH node: octahedral normal (i16×2) + quantized slab bounds (u16×2); see §14.9b
PHCC	0x00	76	No refs
PHAC	0x00	32	2 refs
DMSE	0x00	4	Convex hull half-edge
SHBX	0x00	64	No refs
DMMT	0x00	28	No refs
DMME	0x00	20	No refs
VVOL	0x00	40	No refs
SCHR	0x00	12	1 ref
SDEV	0x00	32	2 refs
SD2V	0x00	32	2 refs
SD3V	0x00	32	2 refs
SD4Q	0x00	32	2 refs
SDCC	0x00	32	2 refs
SDR3	0x00	32	2 refs
SDS6	0x00	32	2 refs
SDU6	0x00	32	2 refs
SDU3	0x00	32	2 refs
SDFG	0x00	32	2 refs
SDMB	0x00	32	2 refs
SVC3	0x00	36	No refs — referenced by PAR_ (splatLineData)
SR32	0x00	20	No refs
MT32	0x00	28	
MT16	0x00	14	

Appendix A: Reading Algorithm (Pseudocode)

```
void readM3(const char* path) {
    File file(path);

    // 1. Read MD34 header
    u8 headerBuf[32];
```

```
    file.read(headerBuf, 0, 32);
    u32 magic = *(u32*)headerBuf;
    assert(magic == 0x4D443334);  // "MD34" in LE

    u32 indexOffset = *(u32*)(headerBuf + 4);
    u32 indexCount  = *(u32*)(headerBuf + 8);

    // Model reference at offset 12 (12-byte Ref<MODL>)
    u32 modlEntries = *(u32*)(headerBuf + 12);
    u32 modlIdx     = *(u32*)(headerBuf + 16);

    // 2. Read index table
    std::vector<IndexEntry> index(indexCount);
    file.read(index.data(), indexOffset, indexCount * 16);
    for (auto& entry : index) {
        std::reverse(entry.tag, entry.tag + 4);
    }

    // 3. Read MODL (0-based index)
    auto& modlEntry = index[modlIdx];
    file.seek(modlEntry.offset);
    MODL modl = readMODL(file, modlEntry.version);

    // 4. Follow references to read sub-chunks
    if (modl.bones.entries > 0) {
        auto& boneEntry = index[modl.bones.index];
        file.seek(boneEntry.offset);
        for (u32 i = 0; i < modl.bones.entries; i++) {
            BONE bone = readBONE(file, boneEntry.version);
        }
    }
    // ... repeat for all Ref<T> fields
}
```

Appendix B: Computing AnimRef<T> Size

```
constexpr size_t animRefSize(size_t valueSize) {
    return 2 + 2 + 4 + valueSize + valueSize + 4;
    //     ^   ^   ^      ^           ^        ^
    //   ipt flg aid  initVal     nullVal    idx
}
// AnimRef<float>:    animRefSize(4)  = 20
// AnimRef<Vector3f>: animRefSize(12) = 36
// AnimRef<Quat>:     animRefSize(16) = 44
// AnimRef<ColorBGRA>:animRefSize(4)  = 20
```

Appendix C: MD33 (Beta) Format

The **MD33** header revision was used during the StarCraft II beta. It shares the same chunk-based layout, 16-byte alignment, and index table structure as MD34, with two key differences:

C.1 SmallRef<T> (8 bytes)

MD33 files use a compact typed reference without the flags field:

```
template<typename T>
struct SmallRef {                // MD33 only
    u32 entries;                 // Number of T elements
    u32 index;                   // 0-based index into Index Table
};
```

C.2 Impact on Struct Sizes

Every chunk struct that embeds *N* `Ref<T>` fields is `4 × N` bytes smaller in an MD33 file (each 12-byte `Ref<T>` becomes an 8-byte `SmallRef<T>`). AnimRef<T> fields are unaffected — they do not contain `Ref<T>` fields.

C.3 MD33 Header

Tag: `MD33` | **Size**: 32 bytes (20 meaningful + 12 padding)

```
struct MD33Header {
    char      magic[4];          // "MD33" (LE uint32 0x4D443333)
    u32       indexOffset;
    u32       indexCount;
    u32       modelEntries;      // Always 1
    u32       modelIndex;        // 0-based, typically 1
    u8        padding[12];       // 0xAA fill
};
```

C.4 Beta-Only MODL Versions

Version	Size	Notes
0x14 (v20)	560 bytes	No viewVolumes, trailingModels, m3aAnim fields
0x15 (v21)	560 bytes	Adds trailingModels + m3aAnimHash

C.5 Beta-Only MODL Offset Map (8-byte SmallRef<T>)

Field	v20	v21	v23
attachmentPoints	0x0B0	0x0B0	0x0B0
attachmentPointAddons	0x0B8	0x0B8	0x0B8
lights	0x0C0	0x0C0	0x0C0
shadowBoxes	0x0C8	0x0C8	0x0C8
cameras	0x0D0	0x0D0	0x0D0
camerasAddons	0x0D8	0x0D8	0x0D8
materialMaps	0x0E0	0x0E0	0x0E0
standardMaterials	0x0E8	0x0E8	0x0E8
displacementMaterials	0x0F0	0x0F0	0x0F0

Field	v20	v21	v23
compositeMaterials	0x0F8	0x0F8	0x0F8
terrainMaterials	0x100	0x100	0x100
volumeMaterials	0x108	0x108	0x108
hairMaterials	0x110	0x110	0x110
creepMaterials	0x118	0x118	0x118
particleEmitters	0x120	0x120	0x120
particleEmitterCopies	0x128	0x128	0x128
ribbonEmitters	0x130	0x130	0x130
projections	0x138	0x138	0x138
forces	0x140	0x140	0x140
warps	0x148	0x148	0x148
viewVolumes	—	—	0x150
rigidBodies	0x150	0x150	0x158
physicsConstraints	0x158	0x158	0x160
physicsJoints	0x160	0x160	0x168
ikTwoJoints	0x168	0x168	0x170
ikJoints	0x170	0x170	0x178
oneBoneSolvers	0x178	0x178	0x180
turretBehaviors	0x180	0x180	0x188
triggerData	0x188	0x188	0x190
initialReference	0x190	0x190	0x198
(inline hit-test)	0x198	0x198	0x1A0
fuzzyHitTestObjects	0x1FC	0x1FC	0x204
attachmentVolumes	0x204	0x204	0x20C
attachmentVolumesAddon0	0x20C	0x20C	0x214
attachmentVolumesAddon1	0x214	0x214	0x21C
billboardBehaviors	0x21C	0x21C	0x224
trailingModels	—	0x224	0x22C
m3aAnimHash	—	0x22C	0x234
m3aAnimHashes	—	—	0x238

C.6 Beta-Only Chunk Versions

The following chunk versions are found **exclusively** in MD33 files. Sizes listed are the MD33 element size.

Tag	Version	MD33 Size	Notes
MODL	0x14	560	No viewVolumes/trailing/m3a fields
MODL	0x15	560	+trailingModels, +m3aAnimHash
REGN	0x02	28	Smaller than v3; no refs
CAM_	0x02	144	Pre-v3 camera; farClip/nearClip are plain f32
LAYR	0x14	352	Pre-v22 layer
LAYR	0x15	352	Pre-v22 layer
PAR_	0x0A	1300	Beta particle emitter; 3 refs
PAR_	0x0B	1304	Beta particle emitter; 3 refs
RIB_	0x04	744	Beta ribbon; 1 ref
RIB_	0x05	748	Beta ribbon; 1 ref
EVNT	0x00	92	Beta event; 2 refs
TER_	0x00	16	Name ref only
PATU	0x01	112	No refs (smaller layout)

C.7 MD33 Size Reference for Shared Chunk Versions

For chunk versions that exist in both MD33 and MD34 files, the MD33 size is the MD34 size minus `4 × N` where *N* is the number of `Ref<T>` fields:

Tag	Version	MD34 Size	Refs	MD33 Size
MODL	0x17	784	52	576
SEQS	0x01	96	2	88
SEQS	0x02	92	2	84
STC_	0x04	204	16	140
STG_	0x00	24	2	16
STS_	0x00	28	1	24
BONE	0x01	160	1	156
DIV_	0x02	52	4	36
ATT_	0x01	20	1	16
MAT_	0x0F	268	14	212
CREP	0x00	24	2	16
TER_	0x00	24	2	16
TRGD	0x00	24	2	16
BBSC	0x00	48	1	44
IKJT	0x00	32	1	28
EVNT	0x01	104	2	96

Tag	Version	MD34 Size	Refs	MD33 Size
PAR_	0x0C	1316	3	1304
All SDXX	0x00	32	2	24
SCHR	0x00	12	1	8
TMD_	0x01	72	1	68

Appendix D: Corpus Observations

Statistical observations from the SC2/HotS game file corpus (~50,700 `.m3` files and ~447 `.m3a` files).

D.1 M3A File Statistics

- **0/447** `.m3a` files contain vertex data or mesh batches — strictly animation-only
- **447/447** contain BONE, SEQS, STC_ — always have skeleton and animation data
- **442/447** contain MAT_ — materials present in nearly all (for particle effects)
- MODL versions observed in `.m3a`: v23 (196), v25 (80), v26 (13), v28 (7), v29 (151)

M3A naming conventions:

Suffix	Count	Description
`_Dialogue*`	110	Cinematic dialogue animations
`_RequiredAnims`	94	Mandatory animation supplements
`_SwarmAnims`	41	Heart of the Swarm animations
`_FacialAnims`	29	Facial animation overlays
`OptionalAnims`	10	Optional animation extensions
`_VoidAnims`	5	Legacy of the Void animations
Other	158	Various per-model extensions

M3A linking observations:

- When a `.m3` file's `m3aAnimHashes` array is populated, the corresponding `.m3a` file's `m3aAnimHash` appears in that array
- Many older `.m3` files have empty `m3aAnimHashes` even when matching `.m3a` files exist — the engine may rely on filename conventions
- A single model can have 5+ `.m3a` files (e.g. Marauder: SwarmAnims, VoidAnims, DLC2Anims, etc.)
- Different `.m3a` files can share the same `m3aAnimHash` when providing equivalent animation sets for different skins
- The `.m3a` file's MODL version often differs from its parent `.m3`

D.2 Unused Animation Slots

STC_ animation data slots 6 (SDU8) and 9 (SDS3) are defined by the engine but never observed in the corpus.

D.3 Rare Chunks

Chunk	Occurrences	Context
VVOL	1 file	`SM_HyperionBridgeHolomap.m3`

Chunk	Occurrences	Context
DMMT/DMME	11 files	SC2 destructible models (bridges, walls)
DMSE	1570 files	Convex hull half-edge connectivity (PHSH shapeType 4)
TMD_	1 file	`Infestor.m3` (SC2 Beta only)
BSET	0	Defined in m3studio, never observed
IK2J	0	Defined in m3studio, never observed
IKCC	0	Defined in m3studio, never observed
PHCT	0	Defined in m3studio, never observed
PAOB	0	Defined in m3studio, never observed
VON_	0	Defined in m3studio, never observed in corpus
HAI_	0	Defunct — defined in m3studio, never observed

D.4 Rare Feature Specimens — Corpus Deep Dive

The following is a curated index of specific files in the SC2, HotS, and SC2 Beta corpuses (56,139 `.m3` files total) that exercise uncommon or unique features of the M3 format. All counts and file names were produced by automated binary scanning of every file in the corpus.

D.4.1 VVOL — View Volume (1 file)

The rarest observed chunk in the entire corpus.

File	Corpus	MODL	Notes
`SM_HyperionBridgeHolomap.m3`	Sc2M3	v23	Sole specimen; camera clipping volume

VVOL v0 is 40 bytes, no references. Since only one file uses this feature, parsers should treat it as optional and log a diagnostic when encountered.

D.4.2 TMD_ — Trailing Model (1 file)

File	Corpus	MODL	Notes
`Infestor.m3`	Sc2BetaM3	v23	Sole specimen; defunct trailing model

See Appendix E.7 for full struct layout. This file is also MD33 format, making it the only MD33+TMD_ combination.

D.4.3 WRP_ — Warp Effect (3 files)

All warp effects in the corpus are gravitational distortion VFX.

File	Corpus	MODL	Notes
`SingularityAreaImpact.m3`	Sc2BetaM3	v23	Neural Parasite / Gravity effect
`SingularityAreaImpact.m3`	Sc2M3	v23	Same effect, release build
`Taldarim_Mothership_Black_Hole.m3`	Sc2M3	v29	Black Hole ability VFX

WRP_ v1 is 132 bytes with no references. The `Taldarim_Mothership_Black_Hole.m3` specimen is the only WRP_ in a late-version (v29) file.

D.4.4 SDU3 — Unsigned 32-bit Animation Keyframes (3 unique files)

The rarest animation data block type. STC_ slot index 10, mapping to `SDU3` chunks.

File	Corpus	MODL	Notes
`SM_Terran01Props.m3`	Sc2M3	v23	Terran storymode props (single SDU3 chunk)
`SM_Terran05Props.m3`	Sc2M3	v23	Terran storymode props (single SDU3 chunk)
`UI_Screen_MainMenu_VideoTakeOver.m3`	Sc2M3	v29	Main menu video (two SDU3 chunks)

For comparison, STC_ slot 6 (`SDU8`) and slot 9 (`SDS3`) remain completely absent from the corpus. SDU3 is the only "near-zero" animation data type actually used.

D.4.5 DMMT / DMME — Physics Mesh (11 files)

The 11 files with destruction mesh data are exclusively SC2 destructible environment models. All use MODL v25 and DMMT/DMME v0.

File	Description
`DefensiveWall90Inner_00.m3`	Defensive wall corner segment
`DefensiveWall90Inner_01.m3`	Defensive wall corner variant
`IceBridge_00.m3`	Ice bridge destructible
`IceBridge_01.m3`	Ice bridge variant
`KorhalGateDeath.m3`	Korhal city gate destruction
`ProtossBridge_00.m3`	Protoss bridge destructible
`ProtossBridge_01.m3`	Protoss bridge variant
`SpaceElevatorDoodad.m3`	Space elevator doodad
`UmojanLabPlatformCellMassive_00.m3`	Umojan lab platform
`ZerusChrysalisEggBase.m3`	Zerus chrysalis egg
`ZerusLogBridge.m3`	Zerus log bridge

Every DMMT file also has a matching DMME chunk. These 11 files are the only test cases for physics mesh face winding and edge adjacency in the entire corpus.

D.4.5a DMSE — Convex Hull Half-Edge (1570 files)

DMSE chunks are present in 1,570 files across HotSM3 and Sc2M3, with 4,370 total index entries — far more common than previously documented. Any PHSH v3 shape with `shapeType = 4` (convex hull) references a DMSE chunk encoding a half-edge table for face adjacency and vertex ring linkage. The following are notable specimens with verified topology:

File	Corpus	MODL	V	E	F	Entries	Notes
`Storm_Hero_D3WitchDoctorM_Base.m3`	HotSM3	v29	8	18	12	36	All tris
`Storm_Hero_Zeratul_HighTemplar.m3`	HotSM3	v29	15	36	23	72	Mixed 3/4-gons

File	Corpus	MODL	V	E	F	Entries	Notes
Ghost_Heavens_COOP_DeathRagdoll.m3	Sc2M3	v29	15	39	26	78	All tris
test.m3	StarM3	v29	8	16	10	32	9 tris + 1 pentagon

Most hulls are fully triangulated, but non-triangular faces are valid—test.m3 contains a pentagonal face (5 half-edges). The nextAroundVertex field chains half-edges into closed vertex rings, enabling efficient support-point queries for GJK/EPA collision detection.

D.4.6 MT16 — Half-precision Transform Keyframes (14 unique files)

Rare animation data type using 7×u16 packed transforms (14 bytes per key).

File	Corpus	MODL	Notes
Storm_Doodad_KingsCrest_Ramp.m3	HotSM3	v29	Map doodad; single MT16
Storm_Doodad_KingsCrest_RavenCourt_WatchTowerPillar_Broken.m3	HotSM3	v29	Broken pillar doodad
Aiur_Hologram_Bridge.m3	Sc2M3	v26	Holographic bridge
ExtendingBridgeWideDirty_00.m3	Sc2M3	v26	Extending bridge (6 MT16 chunks)
ExtendingBridgeWide_00.m3	Sc2M3	v26	Extending bridge (4 MT16 chunks)
Korhal_Runway_00.m3 – _07.m3	Sc2M3	v26	8 runway segment variants
Shakuras_LightBridge.m3	Sc2M3	v26	Shakuras light bridge

ExtendingBridgeWideDirty_00.m3 is notable for having 6 separate MT16 chunks — the highest count of any file. All Sc2M3 specimens are MODL v26, suggesting MT16 was introduced around that version.

D.4.7 SR32 — Static Float AnimRef Channel (8 unique files)

Static scalar channel data stored as AnimRef<f32> (20 bytes per entry). All specimens are SC2 cinematic/portrait models at MODL v29.

File	Corpus	SR32 Count	Notes
Commander_Stetmann_COOP_Portrait.m3	Sc2M3	1	Co-op commander portrait
GeneralDavis_MoebiusLab_Portrait.m3	Sc2M3	4	Cinematic character
SMX3_Cutscene_MP03_Credits_Asteroid_00.m3	Sc2M3	1	Credits cinematic
SMX3_Cutscene_MP03_Credits_Asteroid_01.m3	Sc2M3	1	Credits cinematic
SMX3_GeneralDavis.m3	Sc2M3	6	General Davis cinematic model
SMX3_GeneralDavis_Low.m3	Sc2M3	5	Low-detail variant
SMX3_Stetmann_COOP_Commander.m3	Sc2M3	1	Co-op commander model

File	Corpus	SR32 Count	Notes
StetmannPortrait_Mist.m3	Sc2M3	1	Portrait mist effect

SR32 is referenced from MAT_ normalBlendFactors (v19+). The GeneralDavis and Stetmann models are the best test cases for this channel type.

D.4.8 REF_ — Reflection Material (34 files)

Reflection materials are used exclusively for UI floors and cinematic environments.

Version	Files	Context
v1 (84 bytes)	5	SC2 campaign cutscene rooms (SMX2_Bridge_00, SMX2_Hallway_*, SMX2_ShipRoom_*); MODL v28
v2 (156 bytes)	28	HotS hero selection floors + SC2 cinematics; MODL v29
v3 (160 bytes)	1	Storm_UI_HeroAbout_Platform_Summer20.m3; MODL v30

REF_ v3 has a single specimen — the Summer 2020 HotS hero about platform. This is the only file exercising the extended 160-byte layout. REF_ v1 is found exclusively in SC2 LotV campaign ship rooms.

D.4.9 IKJT — IK Joint (41 files, 1 base model: Colossus)

Every single IKJT file in the corpus is a Colossus variant. No other unit uses IK.

Base Model	Corpus	Skin Variants
Colossus.m3	Sc2BetaM3	Base only
Colossus.m3	Sc2M3	Base + Golden, Ihanrii, Purifier, Taldarim skins
DarkColossus.m3	Sc2M3	Campaign dark variant
Hologram_Skin_Colossus.m3	Sc2M3	Holographic skin

Including death/ragdoll/collection variants, this produces 41 files total. The Colossus uses IKJT for its quadruped gait with 4 IK legs. Parsers needing to test IKJT only need a single Colossus file.

D.4.10 VOL_ — Volume Material (190 files)

Volume materials appear almost exclusively in portrait models for atmospheric fog.

Corpus	Count	Context
Sc2M3	189	SC2 unit/building portraits (e.g. ArtanisPortrait.m3, ArchonPortrait.m3)
Sc2BetaM3	1	ImmortalPortrait.m3
HotSM3	0	—

HotS does not use volume materials at all. All 190 specimens are MODL v23–v29.

D.4.11 SHBX — Shadow Box (269 files)

Shadow boxes are used exclusively for buildings and large doodads.

Corpus	Count	Context
Sc2M3	258	Buildings (`Barracks*`, `Nexus*`, `Hatchery*`), large doodads
Sc2BetaM3	11	Beta buildings
HotSM3	0	—

HotS does not use shadow boxes (its renderer handles shadows differently). SHBX v0 is 64 bytes — a raw 4×4 matrix defining an oriented box volume.

D.4.12 CREP — Creep Material (325 files)

Corpus	Count	Version
Sc2M3	184	v0, v1
Sc2BetaM3	139	v0
HotSM3	2	v1

Creep materials are predominantly Zerg assets. The two HotSM3 files (`CreepMaterial.m3` and one other) are the only HotS specimens.

D.4.13 PHCL — Physics Cloth (382 files)

Cloth physics has two distinct version populations:

Version	Size	Files	Context
v2	128 bytes	19 (Sc2M3 only)	Simpler legacy cloth — banners, capes
v4	192 bytes	363 (HotSM3 only)	Full constraint config — hero capes, tabards

This means **PHCL v2 is SC2 only** and **PHCL v4 is HotS only** — they never overlap in the same corpus. Files with PHCL almost always have PHAC (cloth proxy) and PHCC (cloth constraint) chunks as well.

D.4.14 LFLR — Lens Flare Material (497 files)

Version	Size	Files	Context
v2	80 bytes	~20	Basic lens flares (early SC2)
v3	152 bytes	~477	Extended — animated intensity, HDR, size

Lens flares only appear in Sc2M3 (481) and HotSM3 (16). Each LFLR references LFSB sub-flare entries (700 LFSB chunks total). The 16 HotSM3 specimens are all v3 and exclusively light/glow effects.

D.4.15 MADD — Data-Driven Material (686 files, HotS only)

MADD is **exclusively a HotS feature** — not a single SC2 or Beta file contains it. All 686 files are MODL v30.

Version	Size	Files	Notes
v1	140 bytes	(uncommon)	Base MADD
v2	152 bytes	(common)	+12 bytes
v3	160 bytes	(common)	+8 extra bytes

MADD was the final material feature added to the M3 format (MODL v30 = 0x1E).

D.4.16 PATU — Turret Behavior (1,024 files)

Corpus	Count
HotSM3	628
Sc2M3	381
Sc2BetaM3	15

Two versions exist: v1 (112 bytes, SC2 Beta only) and v4 (152 bytes, SC2/HotS). SC2 specimens include tanks, bunkers, battlecruisers, and turret buildings. HotS specimens are primarily hero models that have auto-targeting abilities.

D.4.17 TRGD — Trigger Data (1,046 files)

Corpus	Count
HotSM3	639
Sc2M3	392
Sc2BetaM3	15

TRGD is common in HotS hero models and SC2 campaign units. It pairs with the engine's trigger/scripting system to fire events at animation keyframes.

D.4.18 PHYJ — Physics Joints (1,078 files)

Corpus	Count
HotSM3	764
Sc2M3	314
Sc2BetaM3	0

PHYJ is absent from the SC2 Beta corpus entirely — joints were added with MODL v24. HotS hero ragdoll models are the primary consumers.

D.4.19 FOR_ — Force (1,288 files)

Version	Files	Context
v1	~older	Basic force
v2	~newer	Extended force

Forces interact with PHRB rigid bodies and PHCL cloth. They define wind, explosion forces, and other environmental influences on physics objects.

D.4.20 MODL Version Distribution

Version	Hex	Total	HotSM3	Sc2M3	Sc2BetaM3	Introduced Feature
v20	0x14	1,160	—	—	1,160	SC2 Beta (MD33 only)

Version	Hex	Total	HotSM3	Sc2M3	Sc2BetaM3	Introduced Feature
v21	0x15	412	—	—	412	SC2 Beta (MD33 only)
v23	0x17	11,585	—	8,487	3,098	SC2 WoL launch
v24	0x18	65	—	65	—	+ikCCD
v25	0x19	5,811	6	5,805	—	+volumeNoiseMaterials
v26	0x1A	3,871	1	3,870	—	+stbMaterials
v28	0x1C	2,088	228	1,860	—	+reflectionMaterials
v29	0x1D	30,461	17,488	12,973	—	+lensFlareMaterials
v30	0x1E	686	686	—	—	+materialAddData (HotS only)

Key observations:

- **v29 dominates** (54% of all files) — the vast majority of production SC2/HotS.
- **v24 is the rarest release version** (65 files) — `+ikCCD` was briefly current before v25 superseded it.
- **v20/v21 are MD33 only** — never appear in MD34 files.
- **v30 is HotS-exclusive** — SC2 never adopted it.
- **v27 does not exist** — the version jumped from v26 (0x1A) to v28 (0x1C).

D.4.21 MD33 Beta Format (4,670 files)

All 4,670 MD33 files reside in Sc2BetaM3. MODL versions observed in MD33:

MODL Version	Files	Notes
v20	1,160	Earliest beta build
v21	412	Later beta build
v23	3,098	Final beta / early release equivalent

MD33 files use `SmallRef<T>` (8 bytes) instead of `Ref<T>` (12 bytes), causing all struct sizes to shrink by `4 × N` where N is the `Ref<T>` count per struct. The `Infestor.m3` (MD33, v23) is the sole TMD_ specimen, making it a dual-rarity test case.

D.4.22 High-Complexity Specimens

Files exercising the most simultaneous rare features (7+ rare chunk types):

File	Corpus	MODL	Rare Chunks
`Storm_Hero_Alexstrasza_Cowl.m3`	HotSM3	v30	MADD, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD
`Storm_Hero_Artanis_Craft20.m3`	HotSM3	v30	MADD, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD
`Storm_Hero_Chogall_Base.m3`	HotSM3	v30	MADD, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD
`Storm_Hero_D3WitchDoctorM_Base.m3`	HotSM3	v29	DMSE, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD
`Storm_Hero_Tyrande_Base.m3`	HotSM3	v29	PATU, PHAC, PHCC, PHCL, PHYJ, SVC3, TRGD
`Storm_Hero_Zeratul_HighTemplar.m3`	HotSM3	v29	DMSE, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD
`Ghost_Heavens_COOP_DeathRagdoll.m3`	Sc2M3	v29	DMSE, PATU, PHAC, PHCC, PHCL, PHYJ, TRGD

These files are the best comprehensive test cases for parser implementors as they exercise cloth physics (PHCL+PHAC+PHCC), rigid body joints (PHYJ), turret behaviors (PATU), trigger data (TRGD), and data-driven materials (MADD) simultaneously.

Files with the most unique chunk tags overall (any tag, not just rare):

File	Corpus	Unique Tags
`XelNaga_ObeliskConstruct.m3`	Sc2M3	55
`Coop_Node_Doorlock.m3`	Sc2M3	54
`XelNaga_Node_Doorlock.m3`	Sc2M3	54
`Purifier_Central_CoreMatrix.m3`	Sc2M3	52
`VoidThrasherEx2_Portrait.m3`	Sc2M3	52
`Void_Thrasher.m3`	Sc2M3	52

D.4.23 Chunk Frequency Summary

Complete chunk occurrence counts across the full 56,139 file corpus, highlighting the rarity tiers:

Ultra-rare (< 20 files):

Chunk	Count	Context
VVOL	1	`SM_HyperionBridgeHolomap.m3` (Sc2M3)
TMD_	1	`Infestor.m3` (Sc2BetaM3, MD33)
WRP_	3	Singularity/Black Hole effects
SDU3	3	Storymode props + main menu
DMMT	11	Physics mesh faces — bridges/walls (Sc2M3)
DMME	11	Physics mesh edges (always paired with DMMT)
MT16	14	Bridges/runways with half-precision transforms
SR32	8	Cinematic normal-blend channels

Rare (20–100 files):

Chunk	Count	Context
REF_	34	Reflection materials (UI floors, cinematics)
IKJT	41	IK joints (Colossus only)

Uncommon (100–500 files):

Chunk	Count	Context
VOL_	190	Volume materials (SC2 portraits)
PHAC	216	Cloth proxies
U64_	216	64-bit unsigned data
SHBX	269	Shadow boxes (SC2 buildings)

Chunk	Count	Context
CREP	325	Creep materials (Zerg)
PHCC	375	Cloth constraints
PHCL	382	Physics cloth (v2=SC2, v4=HotS)
LFLR	497	Lens flare materials

Moderately uncommon (500–2,000 files):

Chunk	Count	Context
MT32	528	PHSH mesh triangle entries (7×u32: 3 vertices + 3 adjacency + flags)
PARC	537	SC2 particle copy data
DMMN	561	k-DOP BVH tree nodes for PHSH mesh collision (v0: Vector3f normal, v1: octahedral + slab bounds); see §14.9b
TER_	656	Terrain objects
MADD	686	Data-driven material (HotS only)
LFSB	700	Lens flare sub-elements
PATU	1,024	Turret behaviors
TRGD	1,046	Trigger data
PHYJ	1,078	Physics joints
FOR_	1,288	Force objects
SVC3	1,323	Static Vector3 channels

Appendix E: Undocumented / Rare Chunk Types

Chunk types defined in the m3studio structures.xml but rarely or never observed in game files. Field layouts are derived from m3studio unless otherwise noted.

E.1 BSET — Bone Animation Set

Tag: `BSET` | **Version**: v0 = 32 bytes

```
struct BSET {
    Flag        flags;
    u16         animationSequenceIndex;
    u16         fallbackSequenceIndex;
    Ref<CHAR>   name;
    Ref<U16_>   splitItems;
};
```

E.2 IK2J — Two-Joint IK Solver

Tag: `IK2J` | **Version**: v0 = 48 bytes

```
struct IK2J {
    Ref<U16_>   dependents;
    u16         boneBase;
    u16         boneTarget;
    u16         boneEnd;
    u16         padding;
    Vector3f    hingeAxis;
    f32         maxAngleInner;
    f32         maxAngleOuter;
    f32         searchUp;
    f32         searchDown;
};
```

E.3 IKCC — CCD IK Solver

Tag: `IKCC` | **Version**: v0 = 24 bytes

```
struct IKCC {
    Ref<U16_>   dependents;
    u16         boneBase;
    u16         boneTarget;
    f32         searchUp;
    f32         searchDown;
};
```

E.4 PAOB — One-Bone Solver

Tag: `PAOB` | **Version**: v0 = 24 bytes

```
struct PAOB {
    Ref<U16_>   dependents;
    u16         bone;
    u16         boneFallback;
    Flag        flags;
    f32         maxAngle;
};
```

E.5 PHCT — Physics Constraint

Tag: `PHCT` | **Version**: v0 = 24 bytes

```
struct PHCT {
    Ref<U16_>   dependents;
    u16         rigidBody1;
    u16         rigidBody2;
    Flag        flags;
    f32         breakForce;
};
```

E.6 HAI_ — Hair Material (Defunct)

Tag: `HAI_` | **Version**: v0 = 116 bytes

Defunct hair material system. Has no corresponding MATM material type value.

```
struct HAI_ {
    Ref<CHAR>       name;
    Ref<LAYR>       layerBase;
    Ref<LAYR>       layerSpecShift;
    Ref<LAYR>       layerSpecNoise;
    Ref<LAYR>       layerAO;
    f32             shiftPrimary;
    f32             shiftSecondary;
    AnimRef<Color>  colorDiffuse;
    AnimRef<Color>  colorSpec;
    f32             specExponent0;
    f32             specExponent1;
};
```

E.7 TMD_ — Trailing Model (Defunct)

Tag: `TMD_` | **Version**: v1 = 72 bytes (MD34) / 68 bytes (MD33)

Trailing model system — defunct. Found in only 1 file (`Infestor.m3`, SC2 Beta). Layout reverse-engineered from that single file.

```
struct TMD_ {
    Ref<VEC3>       vectors;
    f32             param0;             // Observed 5.0
    f32             param1;             // Observed 1.0
    AnimRef<f32>    animFloat0;         // init 0.5
    AnimRef<f32>    animFloat1;         // init 1.0
    u32             flag;               // Observed 1
    u32             reserved0;
    u32             reserved1;
};
```

Appendix F: Effects

This appendix provides a higher-level view of the M3 effect systems described in §12.1, §12.3, and §12.4. In practice, these systems all follow the same broad pattern:

- an effect is usually attached to a `boneIndex`
- visual appearance is driven by a material reference
- time-varying behavior is expressed through `AnimRef<T>` channels
- LOD controls decide when the effect is reduced or disabled

The difference is mainly in **how geometry is generated**: particles spawn discrete instances, ribbons generate connected strips, spline-driven effects follow authored paths, and projectors stamp materials onto nearby geometry.

F.1 Particle Emitters

`PAR_` is the largest and most complex chunk type in M3. Each instance fully describes a single particle system — spawn shape, emission budget, initial velocity, per-particle lifetime curves, physics, noise, collision, flipbook animation, rendering

instance type, and LOD. Optional lightweight overrides live in PARC.

The StarCraft II Art Tools documentation for the SC2 Particle node describes the same system from the artist's perspective. Tutorial — Basic Particle, Particle Key Interpolation, and Particle Overlays are useful companion reading.

F.1.1 Runtime data model

A minimal runtime particle representation needs roughly:

```
struct LiveParticle {
    float3   position;
    float3   velocity;
    float    age;          // seconds since spawn
    float    maxAge;       // resolved from lifetime + lifetimeRandom
    float    mass;         // resolved from mass ± massRandom
    float3   size;         // current interpolated size
    float3   rotation;     // current interpolated rotation (degrees)
    float4   color;        // current interpolated BGRA (unorm)
    uint     flipFrame;    // current flipbook frame index
};
```

Everything in PAR_ exists to either **initialize** those values at spawn or **drive** them during the particle's lifetime. The sections below walk through PAR_ in the order you would consume the fields when building an emitter.

F.1.2 Emitter setup

The first thing to resolve is where in the scene the emitter lives and what material will be used to render its output.

- boneIndex — index into the model's BONE array. Multiply by the bone's world-space transform each frame to get the emitter origin.
- materialIndex — index into MATM. Resolve through the material map to get the actual MAT_/DIS_/etc. chunk for texture binding and blend state.

emitterShape (EmitterShape enum: Point, Plane, Sphere, Box, Cylinder, Disc, Mesh, Spline) determines how spawn positions are generated within the emitter volume. The shape parameters are:

- shapeOuter / shapeInner — animated Vector3f extents for box-like shapes.
- outerRadius / innerRadius — animated f32 for sphere/disc/cylinder shapes.
- shapeRegions — for Mesh shapes, a Ref<U32_> listing region indices.
- splatLineData — for Spline shapes, a Ref<SVC3> providing authored control points.
- lowerBound / upperBound — animated bounds clamping for spline/mesh emission.

When flags & EmitShapeCutout is set, the inner shape is subtracted from the outer shape to create a shell.

F.1.3 Spawning

Each simulation tick your emitter needs to decide how many particles to create. The relevant fields are:

- emissionRate — AnimRef<f32>, particles per second. Resolve against the current animation state to get a rate, accumulate fractional particles across frames.
- maxParticles — hard cap on live particles. Skip spawning when the pool is full.
- squirtAmount — AnimRef<u16>, burst count for single-shot effects.
- killRadius — distance from the emitter origin beyond which particles are forcibly killed.

For each new particle you must generate initial conditions:

```
// Per spawn
float speed    = resolve(initialSpeed) + randPm(resolve(initialSpeedRandom));
float yaw      = resolve(initialYaw);
float pitch    = resolve(initialPitch);
float hSpread  = resolve(initialHorizontal);
float vSpread  = resolve(initialVertical);
float life     = resolve(lifetime) + randPm(resolve(lifetimeRandom));
float pmass    = mass + randPm(massRandom);
```

Whether the random channels are actually active depends on the enable bits. In v17+ chunks those live in `additionalFlags` (`EmitSpeedRandomize`, `LifespanRandomize`, `MassRandomize`). In older versions (≤ v14), dedicated per-field `u32` flags served the same purpose.

If `flags & InheritParentVelocity` is set, add `parentBoneVelocity * resolve(particleVelocity)` to the initial velocity.

F.1.4 Per-particle lifetime curves

Each living particle has a normalized age $t = \text{age}/\text{maxAge}$ in $[0, 1]$. The Art Tools model every visual property as a **Start → Mid → End** curve controlled by a midpoint time and optional hold time. Your renderer must evaluate these curves every frame.

Color and alpha. `colorStart`, `colorMid`, `colorEnd` are `AnimRef<ColorBGRA>` values (resolve them for the current animation state to get the base curve endpoints). Interpolate between them using `colorMidTime` and `colorMidHoldTime`.

The original engine shader (`RibbonParticleCommon.fx`) defines five interpolation modes that apply to all per-particle curves. Translated to Slang:

```
// Interpolation mode enum — matches the engine's b_iParticleColorInterpolation values
static const int INTERPOLATION_LINEAR             = 0;  // e_imLinear
static const int INTERPOLATION_LINEAR_SMOOTH      = 1;  // e_imLinearSmooth
static const int INTERPOLATION_BEZIER             = 2;  // e_imBezier
static const int INTERPOLATION_LINEAR_WITH_HOLD   = 3;  // e_imLinearWithHold
static const int INTERPOLATION_BEZIER_WITH_HOLD   = 4;  // e_imBezierWithHold

// Quadratic Bezier helper  (from Common.fx)
float bezierInterpolation(float v0, float v1, float v2, float t) {
    float invT = 1.0 - t;
    return invT * invT * v0 + 2.0 * t * invT * v1 + t * t * v2;
}

// Hermite smoothstep helper  (from Common.fx)
float smoothStep(float x) {
    return x * x * (3.0 - 2.0 * x);
}

// General-purpose 3-key interpolation — scalar overload
float interpolateValue(
    float age,
    float value0, float value1, float value2,
    float midTime, float inversedMidTime, float midTimeHold,
    int   interpolation
) {
    float ret = value0;
```

```
    if (interpolation == INTERPOLATION_BEZIER) {
        ret = bezierInterpolation(value0, value1, value2, age);
    } else if (interpolation == INTERPOLATION_LINEAR) {
        if (age < midTime) {
            float t = age * inversedMidTime;
            ret = lerp(value0, value1, t);
        } else {
            float t = (age - midTime) / (1.0 - midTime);
            ret = lerp(value1, value2, t);
        }
    } else if (interpolation == INTERPOLATION_LINEAR_SMOOTH) {
        if (age < midTime) {
            float t = smoothStep(age * inversedMidTime);
            ret = lerp(value0, value1, t);
        } else {
            float t = smoothStep((age - midTime) / (1.0 - midTime));
            ret = lerp(value1, value2, t);
        }
    } else if (interpolation == INTERPOLATION_LINEAR_WITH_HOLD) {
        float t0 = midTime - midTimeHold;
        float t1 = midTime + midTimeHold;
        if (age < t0)        ret = lerp(value0, value1, age / t0);
        else if (age < t1)   ret = value1;
        else                 ret = lerp(value1, value2, (age - t1) / (1.0 - t1));
    } else if (interpolation == INTERPOLATION_BEZIER_WITH_HOLD) {
        float t0 = midTime - midTimeHold;
        float t1 = midTime + midTimeHold;
        float a  = age;
        float v0 = value0, v2Out = value2;
        if (a < t0)   { a = a / t0;                      v2Out = value1; }
        else          { a = (a - t1) / (1.0 - t1);       v0 = value1;    }
        a = saturate(a);
        ret = bezierInterpolation(v0, value1, v2Out, a);
    }
    return ret;
}
```

The `float3` overload is **not** identical to the scalar version for `INTERPOLATION_BEZIER_WITH_HOLD`. The scalar version replaces either `value0` or `value2` with `value1` and evaluates a single two-piece quadratic Bezier with no explicit hold region. The `float3` overload instead uses a three-piece evaluation with an explicit hold region `[t0, t1]` where `ret = value1`, and shifts the Bezier mid-control points with `lerp(value0, value1, midTimeHold)` and `lerp(value1, value2, midTimeHold)` respectively:

```
// float3 overload — BEZIER_WITH_HOLD only (from RibbonParticleCommon.fx)
float t0 = midTime - midTimeHold;
float t1 = midTime + midTimeHold;
if (age < t0) {
    ret = bezierInterpolation(value0, lerp(value0, value1, midTimeHold), value1, age / t0);
} else if (age < t1) {
    ret = value1;
} else {
    ret = bezierInterpolation(value1, lerp(value1, value2, midTimeHold), value2,
                              (age - t1) / (1.0 - t1));
}
```

All other modes (Linear, LinearSmooth, Bezier, LinearWithHold) are identical between the scalar and `float3` overloads. Use `interpolateValue` for every per-particle curve (color, alpha, size, rotation).

Alpha is **not** stored as a separate channel — it is the `.a` byte of the BGRA color. The chunk does carry its own `alphaMidTime` and `alphaMidHoldTime`, so you should interpolate the alpha component on its own timeline. `colorSmoothing` selects one of the five interpolation modes (0–4) listed above. **Note:** in the particle vertex shader, the same specialization constant `b_iParticleColorInterpolation` is used for both color RGB and alpha — they share a single mode selector, not independent ones.

Size. `sizeAnimation` is an `AnimRef<Vector3f>` — the three components are {start, mid, end} packed into one vector. Evaluate the same Start → Mid → End ramp using `sizeMidTime` / `sizeMidHoldTime` / `sizeSmoothing`.

The vertex shader decompresses size the same way as the other curves:

```
// vInputSize was decompressed from the vertex: vertIn.vSize * (1.0 / 256.0)
float sizeNow = interpolateValue(
    normalizedAge,
    vInputSize.x, vInputSize.y, vInputSize.z,
    midKeyTimes[batchIdx].x,
    inversedMidKeyTimes[batchIdx].x,
    midKeyHoldTimes[batchIdx].x,
    particleSizeInterpolation   // b_iParticleSizeInterpolation
);
```

Rotation. Same pattern as size — `rotationAnimation` is `AnimRef<Vector3f>` {start, mid, end}. Use `rotationMidTime` / `rotationMidHoldTime` / `rotationSmoothing`. If `rotationFlags & Relative` is set, mid and end values are offsets from start rather than absolute.

The rotation value is used to build an axis-angle matrix in the vertex shader:

```
// Axis-angle rotation around an arbitrary axis  (from VSUtils.fx)
float3x3 makeRotation(float angle, float3 axis) {
    float s, c;
    sincos(angle, s, c);
    float oneC = 1.0 - c;
    float3x3 m;
    m[0][0] = oneC * axis.x * axis.x + c;
    m[1][1] = oneC * axis.y * axis.y + c;
    m[2][2] = oneC * axis.z * axis.z + c;
    float xy = axis.x * axis.y;
    float zs = axis.z * s;
    m[1][0] = oneC * xy + zs;    m[0][1] = oneC * xy - zs;
    float zx = axis.z * axis.x;
    float ys = axis.y * s;
    m[2][0] = oneC * zx - ys;    m[0][2] = oneC * zx + ys;
    float yz = axis.y * axis.z;
    float xs = axis.x * s;
    m[2][1] = oneC * yz + xs;    m[1][2] = oneC * yz - xs;
    return m;
}

float generateRotation(float age, float3 vRotation, int batchIdx) {
    return interpolateValue(
        age,
        vRotation.x, vRotation.y, vRotation.z,
```

```
        midKeyTimes[batchIdx].w,
        inversedMidKeyTimes[batchIdx].w,
        midKeyHoldTimes[batchIdx].w,
        particleRotationInterpolation  // b_iParticleRotationInterpolation
    );
}
```

Randomization. When `sizeRandomEnable`, `rotationRandomEnable`, or `colorRandomEnable` are non-zero, the corresponding random companion channel (`sizeRandomAnimation`, `rotationRandomAnimation`, `colorStartRandom` / `colorMidRandom` / `colorEndRandom`) provides a per-particle random offset. Generate the offset once at spawn and add it to the base curve each frame. `alphaRandomEnable` controls whether the alpha component of the random color channels is used.

F.1.5 Flipbook animation

If the bound material uses a sprite sheet, the flipbook fields drive which cell is displayed:

- `flipbookColumns`, `flipbookRows` — grid dimensions.
- `flipbookStartInitIndex` / `flipbookStartStopIndex` — frame range at birth.
- `flipbookEndInitIndex` / `flipbookEndStopIndex` — frame range at death.
- `flipbookMidTime` — crossover point between start and end ranges.
- `flipbookColumnFraction`, `flipbookRowFraction` (v12+) — sub-cell UV offsets.

When `flags & RandomFlipbookStart` is set, pick the initial frame randomly within the start range instead of always starting at `flipbookStartInitIndex`. The vertex shader (`Particle.fx`) resolves the flipbook cell like this:

```
// Inside EmitParticleUV — UVMAP_PARTICLE_FLIPBOOK path
int cell;
if (sizeAndAge.w <= flipbookMidKeyTime[batchIdx]) {
    float range  = flipbookFrames[batchIdx].y - flipbookFrames[batchIdx].x;
    float offset = range * (sizeAndAge.w / flipbookMidKeyTime[batchIdx]);
    cell = int(flipbookFrames[batchIdx].x + floor(offset + 0.5));
} else {
    float range  = flipbookFrames[batchIdx].z - flipbookFrames[batchIdx].y;
    float offset = range * ((sizeAndAge.w - flipbookMidKeyTime[batchIdx])
                            / (1.0 - flipbookMidKeyTime[batchIdx]));
    cell = int(flipbookFrames[batchIdx].y + floor(offset + 0.5));
}
if (randomFlipBookStart)
    cell += int(floor(noiseVector.w));

int cellX = cell % int(flipbookColumnCount[batchIdx]);
int cellY = cell / int(flipbookColumnCount[batchIdx]);
float2 uvOffset = float2(
    float(cellX) * flipbookCellSize[batchIdx].x,
    float(cellY) * flipbookCellSize[batchIdx].y
);
float2 uv = baseUV * flipbookCellSize[batchIdx] + uvOffset;
```

F.1.6 Physics update

Each simulation tick, integrate the standard Newtonian update for every live particle. The engine (`VSElementUtils.fx`) implements an analytic exponential-drag solver rather than naïve Euler integration. Translated to Slang:

```
// Analytic displacement and velocity under constant gravity + linear drag.
// Ported from VSElementUtils.fx — CalculateDisplacmentAndVelocity().
void calculateDisplacementAndVelocity(
    float  elapsedTime,
    float3 initialVelocity,
    float  mass,
    float  invMass,
    float  drag,
    float  invDrag,
    float  gravity,
    out float3 displacement,
    out float3 velocity
) {
    float3 gVec                   = float3(0.0, 0.0, gravity);
    float3 massGravity            = mass * gVec;
    float3 massGravityOverDrag    = massGravity * invDrag;
    float  expTerm                = exp(-drag * invMass * elapsedTime);

    // displacement
    float3 term0a = -(initialVelocity + massGravityOverDrag);
    float  term0b = mass * invDrag;
    float3 term0  = term0a * term0b * expTerm;
    float3 term1  = elapsedTime * massGravityOverDrag;
    float3 term2  = term0b * (initialVelocity + massGravityOverDrag);
    displacement  = term0 - term1 + term2;

    // instantaneous velocity
    float3 vTerm0 = invDrag * expTerm * (drag * initialVelocity + massGravity);
    velocity      = vTerm0 - massGravityOverDrag;
}
```

The vertex shader calls this per-particle when `useProceduralPosition` is set, passing the time delta from birth and the initial velocity stored in `interpolator1.xyz`. Unlike ribbons (which use per-batch uniform physics constants), the particle shader reads mass, drag, and gravity from **per-vertex** attributes: `interpolator1.w` = 1/mass, `birthDeathAndDrag.zw` = {drag, 1/drag}, and `interpolator2.w` = negated gravity.

`noiseAmplitude`, `noiseFrequency`, `noiseCoherence`, and `noiseEdge` parameterize a procedural noise displacement. A simple implementation evaluates a coherent noise function (e.g. simplex) seeded per-particle and scaled by amplitude/frequency.

Force channels are bitmasks. `localForces` selects which `FOR_` chunks in local (bone) space affect this emitter; `worldForces` selects world-space forces. `localForcesFallback` and `worldForcesFallback` are used when the primary channels are not available. Each set bit corresponds to a `FOR_` chunk's channel index.

If `additionalFlags & WorldSpace` is set, particles simulate in world space rather than relative to the bone. When `flags & UseLocalTime` is set, the emitter's clock does not advance with the global animation but with the parent model's local time. `flags & SimulateInit` tells the runtime to pre-roll the simulation when the emitter first activates, so particles do not visibly "pop in" from an empty state.

F.1.7 Collision

When `flags & CollideTerrain` or `flags & CollideObjects` is set, test particle positions against the collision surfaces each tick. On impact:

1. Reflect velocity, scale by `bounce` and reduce by `friction`.

2. If `collisionDieBounce` is set, kill the particle.
3. If `collisionSpawnIndex >= 0`, spawn `rand(collisionSpawnMin .. collisionSpawnMax)` child particles from `PAR_[collisionSpawnIndex]` with probability `collisionSpawnChance` and energy scale `collisionSpawnEnergy`.
4. If `splatProjectionIndex >= 0`, spawn a projector decal from `PROJ[splatProjectionIndex]` with probability `splatChance`.

F.1.8 Variation overlays

The nine groups of `*Type` / `*Amplitude` / `*Frequency` fields (for pitch, yaw, speed, size, alpha, color, rotation, horizontal spread, and vertical spread) apply a **per-particle procedural wave** on top of the base property. Think of them as LFO modulation:

```
float overlay = resolve(amplitude) * waveFunc(type, age * resolve(frequency) + phase);
```

`type` selects the wave shape (sine, sawtooth, etc.). `phaseShift` (v22+) offsets the phase uniformly across all overlay channels. These are the "Overlay" controls in the Art Tools (Appendix_ParticleOverlays).

F.1.9 Instance type and rendering

`instanceType` (`ParticleInstanceType` enum) determines the geometry you need to build per particle. The vertex shader (`Particle.fx`) branches on a `b_iInstanceType` specialization constant:

Value	Shader constant	Geometry
0	`PARTICLE_BILLBOARD`	Camera-facing billboard quad.
1	`PARTICLE_TAIL`	Velocity-oriented stretched quad; length from `tailLength`.
2	`PARTICLE_FACE_TRAVEL_DIR`	Quad oriented along instantaneous velocity.
3	`PARTICLE_FACE_WORLD_DIR`	Quad oriented along a fixed world direction.
4	`PARTICLE_SINGLE_AXIS`	Billboard locked to a single rotation axis.
5	`PARTICLE_TERRAIN_ORIENTED`	Quad projected onto terrain normal.
6	`PARTICLE_TERRAIN_DIR_ORIENTED`	Terrain-oriented + velocity-stretched.
7	`PARTICLE_EMITTER_ORIENTED`	Quad uses the emitter bone's orientation.
8	`PARTICLE_PHYSICS_ORIENTED`	Quad oriented by physics simulation.
9	`PARTICLE_PINNED`	Stretch between spawn origin and current position.
10	`PARTICLE_TRAIL`	Like Tail but offset by one tail-length.

The vertex input struct declared in `Particle.fx` supplies all per-particle data:

```
struct ParticleVertex {
    float4  position            : NORMAL;       // world-space position
    int4    size                : TEXCOORD0;    // {start, mid, end, extra} * 256
    half4   color0              : COLOR0;       // start color (BGRA)
    half4   color1              : COLOR1;       // mid color
    half4   color2              : TEXCOORD1;    // end color
    int4    rotation            : TEXCOORD2;    // {start, mid, end} * 32 + random UV in
.w
```

```
    float4  birthDeathAndDrag    : TEXCOORD3;    // {birthTime, deathTime, drag, 1/drag}
    uint4   batchIndex           : TEXCOORD4;    // batch ID for instanced draw
    half4   interpolator1        : TEXCOORD5;    // xyz = initial velocity, w = 1/mass
    half4   interpolator2        : TEXCOORD6;    // context-dependent; .xyz =
direction/tail/spawn-pos,
                                                 //   .w = negated gravity (for procedural
position)
    half4   noiseVector          : TEXCOORD7;    // xyz = noise offset, w = random seed
    int2    offset               : POSITION;     // quad corner {-1,+1}
};
```

For billboard and tail types, compute the quad from the current interpolated size, rotation, and color, then submit with the material resolved from `materialIndex`. Use `instanceAngle` to apply a fixed orientation offset, and `instanceDistance` (v17+) to space model instances.

The billboard path (most common) works as follows:

```
// PARTICLE_BILLBOARD path  (from Particle.fx → ParticleVertexMain)
float  angle   = generateRotation(normalizedAge, inputRotation, batchIdx);
float3 forward = cameraDirection;
float3 right   = billboardRight;
float3 up      = billboardUp;

if (useModelInstancing) {
    position = mul(float4(position, 1.0), particleInstanceTransform[batchIdx]).xyz;
}

float3x3 rot    = makeRotation(angle, forward);
float3   offset = quadCorner.x * right + quadCorner.y * up;
offset         *= elementScale[batchIdx];
position       += mul(sizeNow * offset, rot);

right    = mul(right, rot);
up       = mul(up, rot);
normal   = normalize(cross(right, up));
tangent  = right;
binormal = -up;
```

When `flags & Sort` is set, sort particles back-to-front by camera distance before drawing. `flags & SortHeight` uses world Z instead of camera distance. `flags & SortReverse` inverts the order (useful for additive blending). `flags & LitParts` means particles should receive scene lighting rather than being unlit. `flags & UseVertexAlpha` feeds the particle alpha into the vertex color attribute.

For model-particle mode (`flags & ModelParticles`), load the external models from `modelPaths`, instantiate them at each particle's transform, and if `flags & SwapYZOnModelParticles` is set, swap the Y and Z axes of the child model.

F.1.10 Screen-space UV and LOD

`alphaThreshold` (v17+) is an animated alpha-test cutoff — discard fragments below this value. `uvOffset`, `uvAngle`, and `uvTiling` (all v17+) let the material UVs be animated in screen space for scrolling / rotating overlay effects.

`lodReduce` and `lodCut` control quality scaling. A common implementation maps the LOD enum (0=None, 1=Low, 2=Medium, 3=High, 4=Ultra) to an emission-rate multiplier and a visibility cutoff respectively.

F.1.11 Trail spawning and `PARC`

When `flags & SpawnTrailingParticles` is set and `trailLinkIndex >= 0`, each particle periodically spawns child particles from `PAR_[trailLinkIndex]` at the rate given by `trailEmissionRate`, with `trailChance` controlling per-tick probability.

`PARC` (Particle Emitter Copy) is a 40-byte struct that overrides only `emissionRate`, `squirtAmount`, and `boneIndex` of an existing `PAR_`. The model root lists copies in `particleEmitterCopies`; `PAR_.copyIndices` links the base emitter back to its copies. This lets art reuse one expensive emitter setup across multiple bones without duplicating the full 1,300+ byte definition.

F.2 Ribbon Emitters

`RIB_` generates a connected polygon strip rather than a cloud of independent particles. Each "segment" of the ribbon is emitted at the bone origin once per tick, then simulates independently — drifting, colliding, fading — while its vertices remain stitched to the segments before and after it. The Art Tools documentation for the SC2 Ribbon node and SC2 Spline Ribbon node describe the same system from the artist's perspective.

F.2.1 Runtime data model

Unlike particles, the natural runtime unit for a ribbon is a **segment ring** rather than a point. A minimal representation looks like:

```
struct RibbonSegment {
    float3   position;      // center of this ring
    float3   velocity;      // current velocity
    float    age;           // seconds since emission
    float    maxAge;        // resolved lifetime (or infinite for length-based)
    float    distFromHead;  // cumulative distance from the emitter
    float    width;         // interpolated half-width at this segment
    float    twist;         // interpolated rotation (degrees) around the spine
    float4   color;         // interpolated color + alpha (unorm)
};
```

The ribbon vertex shader (`Ribbon.fx`) declares two separate vertex layouts selected by the simulation technique constant `b_iRibbonSimTech`:

```
static const int RIBBON_SIM_GPUONLY                   = 0;
static const int RIBBON_SIM_SPLINE_RIBBON             = 1;
static const int RIBBON_SIM_MIXED                     = 2;
static const int RIBBON_SIM_MIXED_PRECOMPUTED_TANGENT = 3;
static const int RIBBON_SIM_LEGACY                    = 4;

// Vertex layout for spline ribbons (minimal — positions come from uniforms)
struct SplineRibbonVertex {
    float4       position   : POSITION;     // .x=age, .y=contractedAge, .z=invAge,
.w=contractedInvAge
    half2        uv         : TEXCOORD0;
    half2        offset     : NORMAL;       // cross-section corner
    uint4        batchIndex : TEXCOORD6;
};

// Vertex layout for standard / GPU-only / legacy ribbons
struct RibbonVertex {
    float4       position       : POSITION;   // xyz = world position, w = birth time
    half4        size           : TEXCOORD1;  // {start, mid, end, unused}
```

```
    half4        color0         : COLOR0;     // start color (BGRA)
    half4        color1         : COLOR1;     // mid color
    half4        color2         : TEXCOORD2;  // end color
    half4        rotation       : TEXCOORD3;  // {start, mid, end, extra-U}
    half2        offset         : NORMAL;     // cross-section corner
    half3        up             : TEXCOORD4;  // emission-time up vector
    float4       initialVelocity: TEXCOORD5;  // xyz = velocity, w = death time
    uint4        batchIndex     : TEXCOORD6;
    // Conditional fields (depend on sim technique):
    // half2 uv             : TEXCOORD0;  // legacy only
    // half  extraU         : TEXCOORD7;  // GPU-only
    // float3 positionPrev  : TEXCOORD7;  // precomputed tangent / legacy
    // float3 positionNext  : TEXCOORD8;  // legacy only
};
```

The four cross-section shapes are selected by `b_iRibbonType`:

`b_iRibbonType`	**Geometry**
0 — `RIBBON_BILLBOARD`	One quad per segment, always camera-facing.
1 — `RIBBON_PLANAR`	One quad per segment, orientation locked at emission time.
2 — `RIBBON_CYLINDER`	Prism with `edges` sides extruded along the spine.
3 — `RIBBON_STAR`	Like Cylinder but with alternating inner/outer radii, creating star-shaped indents. `innerRadius` (0–1) controls indent depth.

`edges` specifies the polygon count of a cylindrical or star cross-section. `divisions` controls the tessellation density along the spine (higher = smoother but more expensive). `ribbonType` selects the ribbon cross-section shape (see **Ribbon Type Enum** above).

F.2.2 Emitter setup

- `boneIndex` (u16) — the bone that drives the emitter origin; `boneIndexFallback` is an alternate when the primary is unavailable.
- `materialIndex` — resolves through `MATM` like any other material reference.
- `active` — `AnimRef<u32>`, animatable on/off toggle. The Art Tools describe this as "step keys that behave as linear to the engine". In practice, treat any non-zero resolved value as "emitting".

F.2.3 Lifespan model — time-based vs. length-based

The Art Tools expose two culling modes:

- **Time-based**: each segment lives for `lifetime` seconds (± `lifetimeRandom`). Oldest segments are killed when they expire.
- **Length-based**: segments are culled when the total ribbon arc-length exceeds `maxLength`. The `flags & UseLengthAndTime` bit enables a hybrid where a length-based ribbon *also* enforces a maximum lifetime as a safety cap.

In either mode, `killRadius` works the same way as `PAR_`: segments beyond this distance from the emitter are forcibly removed.

F.2.4 Initial segment velocity

When a segment is born, compute its initial velocity from the same kinds of fields used in `PAR_`:

```
float speed   = resolve(initialSpeed) + randPm(resolve(initialSpeedRandom));
float yaw     = resolve(initialYaw);
float pitch   = resolve(initialPitch);
float hSpread = resolve(initialHorizontal);
float vSpread = resolve(initialVertical);
```

Whether the random terms are active is controlled by `additionalFlags` (v8+: `SpeedRandomize`, `LifespanRandomize`, `MassRandomize`). Older versions (≤ v6) used dedicated `u32` enable fields in the deprecated block.

If `flags & InheritParentVelocity` is set, add `parentBoneVelocity * resolve(particleVelocity)` to the initial velocity — the same pattern as `PAR_`.

F.2.5 Per-segment lifetime curves

Segment properties evolve exactly like particle properties — a Start → Mid → End ramp parameterized by `midTime` and `holdTime` per channel.

Color and alpha. `colorStart` / `colorMid` / `colorEnd` are `AnimRef<ColorBGRA>`. Alpha lives in the `.a` byte, with its own `alphaMidTime` / `alphaMidHoldTime`. `colorSmoothing` (v8+) selects the interpolation mode, shared between color and alpha — same modes as `PAR_` (see Particle Key Interpolation).

Size (width). `sizeAnimation` is `AnimRef<Vector3f>` {start, mid, end}. Use `sizeMidTime` / `sizeMidHoldTime` / `sizeSmoothing`. The result is the half-width of each ring.

Rotation (twist). `rotationAnimation` is `AnimRef<Vector3f>` {start, mid, end} in degrees. Twist rotates the cross-section around the ribbon spine.

For **color, alpha, and size**, the evaluation code is identical to the `PAR_` helper shown in §F.1.4. The ribbon vertex shader (`Ribbon.fx`) calls the same `interpolateValue` function from `RibbonParticleCommon`.

However, ribbon rotation uses a simple two-piece linear interpolation — it does **not** go through the 5-mode `interpolateValue()` function. There is no `b_iRibbonRotationInterpolation` constant in the shaders. From `Ribbon.fx`:

```
float3 segColor = interpolateValue(
    age, color0.rgb, color1.rgb, color2.rgb,
    midKeyTimes[batchIdx].y,
    inversedMidKeyTimes[batchIdx].y,
    midKeyHoldTimes[batchIdx].y,
    ribbonColorInterpolation  // b_iRibbonColorInterpolation
);
float segAlpha = interpolateValue(
    age, color0.a, color1.a, color2.a,
    midKeyTimes[batchIdx].z,
    inversedMidKeyTimes[batchIdx].z,
    midKeyHoldTimes[batchIdx].z,
    ribbonColorInterpolation
);
float segWidth = interpolateValue(
    age, size.x, size.y, size.z,
    midKeyTimes[batchIdx].x,
    inversedMidKeyTimes[batchIdx].x,
    midKeyHoldTimes[batchIdx].x,
```

```
    ribbonSizeInterpolation   // b_iRibbonSizeInterpolation
) * 0.5;   // engine multiplies by 0.5 to get half-width

// Ribbon twist — simple two-piece linear, NOT through interpolateValue()
float angle;
if (age < midKeyTimes[batchIdx].w)
    angle = lerp(rotation.x, rotation.y, age / midKeyTimes[batchIdx].w);
else
    angle = lerp(rotation.y, rotation.z,
                 (age - midKeyTimes[batchIdx].w) / (1.0 - midKeyTimes[batchIdx].w));
```

F.2.6 Physics update

Every segment simulates independently each tick. The integration uses the same analytic exponential-drag solver as `PAR_` (§F.1.6):

```
// From Ribbon.fx — proceduralPosition path
float3 offsetFromOrigin;
float3 instantaneousVelocity;

float elapsed = systemTime[batchIdx] - segment.birthTime;
calculateDisplacementAndVelocity(
    elapsed,
    segment.initialVelocity,
    physicsConstants[batchIdx].x,    // mass
    physicsConstants[batchIdx].y,    // 1/mass
    physicsConstants[batchIdx].w,    // drag
    1.0 / physicsConstants[batchIdx].w,  // 1/drag
    -physicsConstants[batchIdx].z,   // -gravity (downward)
    offsetFromOrigin,
    instantaneousVelocity
);
segment.position += proceduralScalar * offsetFromOrigin;
```

`mass` and `massRandom` work the same way; `massSizeMultiplier` scales effective mass by the segment's current width. Force channel bitmasks (`localForces`, `worldForces`, `localForcesFallback`, `worldForcesFallback`) interact with `FOR_` chunks identically to particles.

If `additionalFlags & WorldSpace` is set (or the legacy `worldSpace` field in ≤ v6), segments simulate in world coordinates — only the emitter origin follows the bone. Otherwise the entire ribbon transforms with the bone.

F.2.7 Noise

Noise is applied **after** the physics step as a post-simulation visual displacement. The Art Tools describe it as "seemingly random variation" that does not feed back into the simulation. The parameters are:

- `noiseAmplitude` — maximum displacement distance.
- `noiseFrequency` — spatial frequency of the noise field.
- `noiseCoherence` — speed at which the noise pattern scrolls over time.
- `noiseEdge` — mutes noise near the emitter. A value of 0 means full noise from birth; 0.5 means segments do not reach full amplitude until halfway through their lifetime.

Note that because noise is applied post-tangent-generation, high amplitudes can skew lighting normals. Enable `flags & AccurateGPUTangents` to regenerate tangents and normals every frame at the cost of extra GPU work.

F.2.8 Collision

When `flags & CollideTerrain` or `flags & CollideObjects` is set, test each segment against the collision surface. On impact:

1. Reflect velocity, scale normal component by `bounce`, tangent component by `1 - friction`.
2. Unlike `PAR_`, ribbons do not support spawn-on-collision children or splats — the collision response is purely physical.

`bounce = 1.0` is perfectly elastic; `friction = 0.0` keeps full tangent speed.

F.2.9 Variation overlays

Five overlay groups (compared to nine in `PAR_`):

Channel	Fields
Yaw	`yawType`, `yawAmplitude`, `yawFrequency`
Pitch	`pitchType`, `pitchAmplitude`, `pitchFrequency`
Speed	`speedType`, `speedAmplitude`, `speedFrequency`
Size	`sizeType`, `sizeAmplitude`, `sizeFrequency`
Alpha	`alphaType`, `alphaAmplitude`, `alphaFrequency`

These work identically to `PAR_` overlays — a per-segment procedural wave applied on top of the base property. `overlay` (`AnimRef<f32>`) provides a global phase offset (the Art Tools call it "Overlay Offset").

F.2.10 Rendering the strip

Once all segments have been updated, build the renderable strip:

1. Walk segments from head (newest) to tail (oldest).
2. At each ring, compute a right vector. The ribbon vertex shader (`Ribbon.fx`) handles the four cross-sections:

```
static const int RIBBON_BILLBOARD = 0;
static const int RIBBON_PLANAR    = 1;
static const int RIBBON_CYLINDER  = 2;
static const int RIBBON_STAR      = 3;

// Tangent — average of prev/next segment directions (legacy/mixed path)
float3 tangent = normalize(
    (position - positionPrev) + (positionNext - position)
);
tangent = safeNormalize(tangent, float3(1, 0, 0));

float3 normal, binormal;
if (ribbonType == RIBBON_BILLBOARD) {
    float3 cameraDir = cameraDirection;
    if (ribbonLocalSpace)
        cameraDir = mul(cameraDir, float3x3(invWorldTransform[batchIdx]));

    if (proceduralPosition && !precomputedTangent
        && simTech != RIBBON_SIM_MIXED) {
        // GPU / procedural path — flatten tangent onto camera plane,
        // then derive binormal perpendicular to the flattened segment.
```

```
            float3 segment = normalize(tangent - cameraDir * dot(tangent, cameraDirection));
            binormal = cross(segment, cameraDir);
        } else {
            // Standard / legacy / precomputed-tangent path
            normal   = -cameraDir;
            binormal = safeNormalize(cross(normal, tangent), float3(0, 1, 0));
        }
        normal = safeNormalize(cross(tangent, binormal), float3(0, 0, 1));
    } else {
        binormal = vertIn.up;
        normal   = safeNormalize(cross(tangent, binormal), float3(0, 0, 1));
    }

    // Apply per-segment twist rotation
    float angle;
    if (age < midKeyTimes[batchIdx].w)
        angle = lerp(rotation.x, rotation.y, age / midKeyTimes[batchIdx].w);
    else
        angle = lerp(rotation.y, rotation.z,
                     (age - midKeyTimes[batchIdx].w) / (1.0 - midKeyTimes[batchIdx].w));
    float3x3 rot = makeRotation(angle, tangent);
    binormal = mul(binormal, rot);
    normal   = mul(normal, rot);

    // Build vertex offset
    float3 vertexOffset;
    if (ribbonType == RIBBON_BILLBOARD || ribbonType == RIBBON_PLANAR) {
        vertexOffset = offset.y * binormal;
    } else { // RIBBON_CYLINDER or RIBBON_STAR
        float3x3 basis = float3x3(normal, tangent, binormal);
        vertexOffset = mul(float3(offset.x, 0.0, offset.y), basis);
        if (ribbonType == RIBBON_CYLINDER)
            vertexOffset = safeNormalize(vertexOffset, float3(0, 0, 1));
    }
    position += vertexOffset * halfWidth;
```

3. Connect consecutive rings with triangle strip indices.
4. Assign UVs: U typically goes 0→1 across the width; V goes 0→1 along the length from head to tail (or proportional to `distFromHead / maxLength` for length-based ribbons).
5. If `flags & EdgeFalloff` is set, fade alpha toward the strip edges (useful for soft-edged energy beams).
6. If `flags & UseVertexAlpha` is set, write per-vertex alpha to the vertex color attribute.

`flags & ForceCPUSim` forces the entire simulation and vertex generation onto the CPU — the Art Tools note that GPU ribbons can exhibit artifacts, and CPU mode is the most accurate fallback.

F.2.11 Timing and LOD

- `flags & LocalTime` — simulation clock is tied to the animation timeline rather than real time.
- `flags & SimulateInit` — pre-roll the simulation on first activation so the ribbon appears at full length immediately.
- `flags & ScaleTimeByParent` — simulation speed tracks the parent model's animation playback rate.
- `lodReduce` — the graphics quality level at which the ribbon begins emitting fewer segments (each step below this level culls ~25% of segments).
- `lodCut` — the quality level at which the ribbon is hidden entirely.

F.3 Spline Emitters

Spline behavior appears in two distinct forms inside the M3 effect system. There is no standalone `SPL_` chunk; instead, spline effects are authored as modified particles or as spline ribbons:

1. **Spline-shaped particle emission** — a `PAR_` record with `emitterShape = 7` (Spline), paired with `splatLineData` (`Ref<SVC3>`).
2. **Spline ribbons** — a `RIB_` record whose `splineRibbons` vector is populated with one or more `SRIB` control-point records.

The Art Tools document the two as separate node types: SC2 Ribbon (standard trail ribbons) and SC2 Spline Ribbon (Bezier-curve ribbons used for arcing lightning, dynamic tubes, and the stretchy spine crawler). This section focuses on the spline-ribbon form, which is the more complex of the two.

F.3.1 How a spline ribbon differs from a standard ribbon

A standard `RIB_` emits segments that trail behind a single bone, each simulating independently. A spline ribbon instead defines a **Bezier curve** between two or more bone-attached control points (`SRIB` records). The engine evaluates the curve every frame, subdivides it into segments, and renders the result as a polygon strip — the same cross-section geometry described in §F.2.1 (planar billboarded / planar / cylinder / star, with `edges` and `innerRadius`).

Because the curve is evaluated analytically, spline ribbons have a very different runtime profile from standard ribbons:

- Segments are **not** emitted over time — the strip is rebuilt every frame.
- There is no per-segment physics integration — positions come from the Bezier evaluation.
- Most of the `RIB_` per-element rollout fields (rotation, smoothing, CPU sim, GPU tangents) are **stubs** inherited from shared UI and have no effect.
- The `active` field, pre-pump, and inherit-parent-velocity flags are also stubs.

F.3.2 The `SRIB` control-point record

Each `SRIB` record (272 bytes, version 0) defines one control point on the Bezier curve:

```
struct SplineRibbon {
    float3      emissionOffset;       // local-space offset from the bone
    float3      emissionVector;       // tangent direction at this point
    AnimRef     velocity;             // speed along the spline at this point
    uint        reserved;             // always 0 (padding/reserved)
    uint        boneIndex;            // the bone this control point follows
    AnimRef     velocityBaseFactor;   // velocity weight near the base
    AnimRef     velocityEndFactor;    // velocity weight near the end
    // Overlay groups — procedural noise layered on top of base values
    uint        yawType;
    AnimRef     yawAmplitude;
    AnimRef     yawFrequency;
    uint        pitchType;
    AnimRef     pitchAmplitude;
    AnimRef     pitchFrequency;
    uint        velocityType;
    AnimRef     velocityAmplitude;
    AnimRef     velocityFrequency;
    AnimRef     yaw;                  // animated yaw offset (degrees)
    AnimRef     pitch;                // animated pitch offset (degrees)
    float       emissionVectorNormFactor; // precomputed ≈ 0.01/|emissionVector|
    float       velocityNormFactor;       // precomputed ≈ 0.01/velocity.initValue
};
```

A spline ribbon with two control points produces a single cubic Bezier segment; additional `SRIB` entries create a piecewise Bezier (one segment per consecutive pair).

F.3.3 Curve evaluation

The Art Tools description ("follows a bezier curve between the start and end points") implies a cubic Bezier where each `SRIB` provides a point **P** and a tangent **T**:

```
P0 = bone[srib[0].boneIndex].worldPos + srib[0].emissionOffset
T0 = srib[0].emissionVector * resolve(srib[0].velocity)
       * resolve(srib[0].velocityBaseFactor)
P1 = bone[srib[1].boneIndex].worldPos + srib[1].emissionOffset
T1 = srib[1].emissionVector * resolve(srib[1].velocity)
       * resolve(srib[1].velocityEndFactor)
```

The four cubic control points are then:

```
C0 = P0
C1 = P0 + T0
C2 = P1 - T1
C3 = P1
```

Subdivide the resulting $B(t) = (1-t)^3 C_0 + 3(1-t)^2 t \cdot C_1 + 3(1-t) t^2 \cdot C_2 + t^3 C_3$ into `divisions` equally-spaced rings. This gives you the same segment-ring array used for standard ribbon rendering (§F.2.10), minus the per-segment physics step.

Apply yaw and pitch overlays to rotate the tangent direction at each control point before computing `T0` / `T1`:

```
float yawDeg   = resolve(srib.yaw)
             + evalOverlay(srib.yawType, srib.yawAmplitude, srib.yawFrequency, t);
float pitchDeg = resolve(srib.pitch)
             + evalOverlay(srib.pitchType, srib.pitchAmplitude, srib.pitchFrequency, t);
// Rotate emissionVector by yaw around local X, pitch around local Y
```

F.3.4 Lifespan model

Spline ribbons support two lifespan modes, but the semantics differ from standard ribbons:

Mode	Behavior
Time-based	The entire ribbon appears, lives for `lifetime` seconds, then disappears. Individual segments are not aged independently.
Infinite	The ribbon exists indefinitely. This is the common case — dynamic tubes and tendrils that persist as long as the model is alive.

The `UseLengthAndTime` flag, `maxLength`, and `killRadius` fields are inherited from the parent `RIB_` structure but have no documented effect on spline ribbons.

F.3.5 Per-element interpolation along the curve

Unlike standard ribbons where color/alpha/size evolve **over time** per segment, spline-ribbon per-element values interpolate **along the length** of the curve (i.e. parameterized by `t` from 0 to 1):

- **Color**: `colorStart` → `colorMid` → `colorEnd` with `colorMidTime` controlling where along the curve the mid value is reached.
- **Alpha**: same pattern with `alphaMidTime`.
- **Size (width)**: `sizeAnimation` Start → Mid → End with `sizeMidTime`.

`holdTime` values for all channels, `smoothing`, `rotation`, `ForceCPUSim`, and `AccurateGPUTangents` are stubs — they exist in the binary because the `RIB_` struct is shared, but the Art Tools mark them as unused for spline ribbons.

Implementation-wise, evaluate the same `interpolateValue()` helper from §F.1.4, but pass `t` (position along the curve) instead of `age / maxAge`:

```
float4 c     = float4(interpolateValue(t, colorStart, colorMid, colorEnd, colorMidTime,
0.0), 1.0);
float  alpha = interpolateValue(t, alphaStart, alphaMid, alphaEnd, alphaMidTime, 0.0);
float  width = interpolateValue(t, sizeStart, sizeMid, sizeEnd, sizeMidTime, 0.0);
```

F.3.6 Physics — what actually works

The Art Tools explicitly mark most physics fields as stubs for spline ribbons. Only **gravity** is documented as functional:

Field	Status
`gravity`	**Active** — accelerates control points along the up-vector.
`mass`	Stub — no effect.
`drag`	Stub — no effect.
`massSizeMultiplier`	Stub — no effect.

Gravity is applied to the control-point positions (not to individual curve samples). In practice this means:

```
for (int i = 0; i < controlPointCount; i++) {
    controlPoints[i].velocity.z -= gravity * dt;
    controlPoints[i].position   += controlPoints[i].velocity * dt;
}
```

The result nudges the Bezier endpoints downward over time, producing a naturally sagging arc.

F.3.7 Noise

Noise works the same way as standard ribbons (§F.2.7) — it is a post-evaluation visual displacement applied to the subdivided segment positions:

- `noiseAmplitude` — maximum displacement.
- `noiseFrequency` — spatial frequency.
- `noiseCoherence` — scroll speed through the noise field.
- `noiseEdge` — mutes noise near the start of the ribbon.

This is one of the primary tools for making spline ribbons look organic rather than perfectly smooth. Lightning effects, for example, use high frequency and amplitude with rapid coherence scrolling.

F.3.8 Force fields

The Art Tools documentation includes a forcefield rollout for spline ribbons but explicitly warns: "Interactions between spline ribbons and forces are not well defined, and may cause the spline ribbon to not show up. Using this rollout is highly discouraged."

The `localForces` / `worldForces` channel masks are present in the binary (`RIB_` struct fields) and can be read, but a robust implementation should either ignore them or treat force field interaction as experimental.

F.3.9 Rendering

Once the curve has been evaluated and subdivided into rings, rendering follows the same strip-building process as standard ribbons (§F.2.10):

1. Build cross-section vertices at each ring using the `emitterShape` / `edges` / `innerRadius` from the parent `RIB_`.
2. Assign per-vertex color and alpha from the per-element interpolation (§F.3.5).
3. UV mapping: U spans the cross-section width; V goes 0→1 from start-point to end-point along the curve.
4. If `flags & EdgeFalloff`, fade alpha toward the strip edges.
5. LOD culling uses `lodReduce` / `lodCut` from the parent `RIB_` record.

The overlay offset (`overlay` AnimRef on the parent `RIB_`) provides a global phase shift for all procedural noise, useful when multiple spline ribbons with identical parameters need de-synchronized visual variation.

F.3.10 Spline-shaped particle emission (`PAR_` with `emitterShape = 7`)

The second form of spline behavior lives entirely inside `PAR_`. When `emitterShape` is set to `Spline` (7), particles are emitted along a spline curve defined by `splatLineData` (`Ref<SVC3>` — a vector of `AnimRef<Vector3f>` entries providing authored 3D positions).

This mode does not use `SRIB` records. The spline control points come from the `SVC3` data, and particles are spawned at positions sampled along the curve. All standard `PAR_` physics, lifetime curves, and rendering apply normally — the spline shape only affects *where* particles are born.

Detect this mode by checking `emitterShape == 7` on a `PAR_` record.

F.3.11 Detection and identification

When loading an M3, identify spline effects as follows:

```
// Spline-shaped particle emitters
for (int i = 0; i < particleEmitterCount; i++) {
    if (particleEmitters[i].emitterShape == EMITTER_SHAPE_SPLINE) {
        // Emission positions come from par.splatLineData (SVC3)
        // Everything else is standard PAR_ behavior
    }
}

// Spline ribbons
for (int i = 0; i < ribbonEmitterCount; i++) {
    if (ribbonEmitters[i].splineRibbonCount > 0) {
        // This is a spline ribbon — use SRIB control points
        // instead of trailing-segment emission
    }
}
```

There is no dedicated chunk tag to scan for. `SRIB` records are always reached through a parent `RIB_`'s `splineRibbons` reference.

F.4 Projectors

`PROJ` projects a material onto existing geometry — terrain, buildings, units — functioning as a decal or volumetric stamp rather than a spawned particle cloud. The Art Tools refer to these as "splats" and document them under the SC2 Projector node. They are used for spell ground indicators, blast marks, waypoints, selection circles, building shadows, terrain decoration, and any effect that paints onto a surface rather than floating above it.

`PROJ` is structurally the simplest effect chunk — it contains no nested `Ref<T>` fields, no sub-chunks, and no per-element simulation loop. All properties are either static floats or `AnimRef` curves.

F.4.1 Runtime data model

A projector is conceptually a camera frustum (or box) positioned at a bone. Everything inside the volume receives the projected material. A minimal runtime representation looks like:

```
struct LiveProjector {
    uint           boneIndex;
    uint           materialIndex;    // resolved through MATM
    int            projectionType;   // Orthographic (0) or Perspective (1)

    // Frustum / box geometry — all animatable
    float4x4       projMatrix;       // built from the fields below
    float3         offset;
    float          pitch, yaw, roll;

    // Lifetime state machine
    enum Phase { Attack, Hold, Decay, Dead };
    Phase          phase;
    float          phaseTime;        // seconds spent in current phase
    float          phaseDuration;    // resolved duration for this phase
    float          alpha;            // current opacity [0–255]
};
```

F.4.2 Projection volume

`projectionType` selects between the two volume shapes:

Perspective (`projectionType = 1`) — the projector behaves like a camera. The closer the origin is to the surface, the smaller the projected image. Build the projection matrix from:

Field	Description
`fieldOfView`	Vertical FOV in degrees.
`aspectRatio`	Width / height ratio — larger values widen the projection.
`near`	Distance from origin where projection begins to appear.
`far`	Distance from origin where projection ceases.

All four are `AnimRef<f32>` and can be keyframed.

Orthographic (`projectionType = 0`) — the projector behaves like a box. The projected image is the same size regardless of distance. The volume is defined by six signed extents from the origin:

Field	Art Tools name	Direction
`boxOffsetZBottom`	Depth (−Z)	How far below the origin.
`boxOffsetZTop`	Depth (+Z)	How far above the origin.
`boxOffsetXLeft`	Width (−X)	Left extent.
`boxOffsetXRight`	Width (+X)	Right extent.
`boxOffsetYFront`	Height (−Y)	Front extent.
`boxOffsetYBack`	Height (+Y)	Back extent.

All six are `AnimRef<f32>`. In the Art Tools the orthographic mode exposes simple Width / Height / Depth spinners that symmetrically set these pairs.

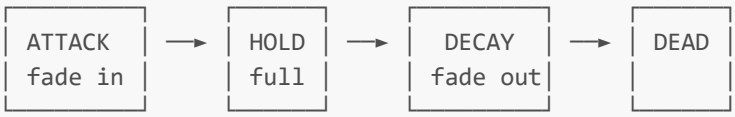
F.4.3 Orientation

The projector is attached to `bone` and offset by `offset` (`AnimRef<Vector3f>`). Its local orientation is controlled by three `AnimRef<f32>` Euler angles: `pitch`, `yaw`, `roll`. Combined with the bone's world transform, these determine the direction the frustum/box points.

If `flags & Static` (`0x1`) is set, the projector locks its world position at spawn time and does not follow the bone afterward — the Art Tools describe this as "Forces the splat to remain in the same location it spawned at."

F.4.4 Alpha lifetime envelope

Unlike particles and ribbons which use Start/Mid/End ramps parameterized by normalized age, projectors use an explicit three-phase state machine:



Phase	Duration fields	Alpha transition
Attack	`lifetimeAttack` / `lifetimeAttackTo`	`alphaInit` → `alphaMid`
Hold	`lifetimeHold` / `lifetimeHoldTo`	Stays at `alphaMid`
Decay	`lifetimeDecay` / `lifetimeDecayTo`	`alphaMid` → `alphaEnd`

Each duration has a base value and a "To" value. When `To` differs from the base, the actual duration is randomized in the range `[base, To]`. The alpha values are floats in the range 0–255 (matching the Art Tools spinner range).

```
float attackDur = randRange(lifetimeAttack, lifetimeAttackTo);
float holdDur   = randRange(lifetimeHold,   lifetimeHoldTo);
float decayDur  = randRange(lifetimeDecay,  lifetimeDecayTo);

// Each frame:
switch (phase) {
```

```
        case Phase.Attack:
            alpha = lerp(alphaInit, alphaMid, phaseTime / attackDur);
            if (phaseTime >= attackDur) { phase = Phase.Hold; phaseTime = 0.0; }
            break;
        case Phase.Hold:
            alpha = alphaMid;
            if (phaseTime >= holdDur) { phase = Phase.Decay; phaseTime = 0.0; }
            break;
        case Phase.Decay:
            alpha = lerp(alphaMid, alphaEnd, phaseTime / decayDur);
            if (phaseTime >= decayDur) phase = Phase.Dead;
            break;
    }
```

If the hold time is set to infinite (the Art Tools "Infinite" checkbox), the projector never enters the Decay phase and persists indefinitely — common for building shadows and terrain paint.

The `active` field (`AnimRef<u32>`) controls whether the projector is alive. Setting it to zero triggers the Decay phase immediately.

F.4.5 Attenuation

`attenuationDistance` (called "Attenuation starts at % distance" in the Art Tools) controls depth-based fade-out. When enabled, the projected material fades as the receiving surface gets farther from the projector origin:

- `attenuationDistance = 0` — standard linear attenuation from the source.
- `attenuationDistance = 1` — equivalent to no attenuation (full opacity across the entire projection depth).

The `falloff` field provides an additional edge-softness control. In practice:

```
float depthFraction = surfaceDepth / projectorRange;
float attenFactor   = saturate((1.0 - depthFraction) / (1.0 - attenuationDistance));
finalAlpha *= attenFactor;
```

F.4.6 Splat layer sorting

The `layer` field determines the draw order of overlapping projectors. The Art Tools define the following layer hierarchy (top draws above bottom):

Value	Layer name	Typical use
0	Material UI Layer	Selection circles, in-world UI elements.
1	Power Layer	Protoss power grid, rarely-active overlays.
2	AOE Layer	AOE cursors, special spell effects.
3	Building Layer	Building placement shadows.
4	Layer 3	Generic — spell effects, blast marks.
5	Layer 2	Generic — spell effects, blast marks.
6	Layer 1	Generic — spell effects, blast marks.
7	Layer 0	Generic — spell effects, blast marks.

Value	Layer name	Typical use
8	Under Creep Layer	Effects above roads but below creep.
9	Hardtile Layer	Roads, terrain paint, grunge, damage marks.

Within the same layer, sort order between projectors is undefined. Between layers, lower value means drawn on top. Note that projectors using `SplatTerrainBake` materials sort among themselves but always draw below projectors using any other material type.

F.4.7 Material

`materialReferenceIndex` resolves through `MATM` like any other material reference. Projectors commonly use:

- **Standard materials** for decals and spell ground effects.
- **SplatTerrainBake** for permanent-looking terrain modifications (paint, grunge) that blend into the terrain texture pipeline.

The material's texture is mapped into the projection volume: U corresponds to the horizontal axis, V to the vertical axis of the projected frustum/box.

F.4.8 Particle-to-projector link

Particle emitters can spawn splats on impact. Two fields on `PAR_` control this:

- `splatProjectionIndex` (i32) — index into the model's `PROJ` array. A value of −1 means no linked projector.
- `splatChance` (f32) — probability [0–1] that a given particle spawns a splat on collision.

When a particle hits a surface and the roll succeeds, the engine creates a projector instance at the collision point using the referenced `PROJ` template. This is how blast marks and impact decals are authored in StarCraft II.

F.4.9 LOD

- `lodCut` — the graphics quality level at which the projector is hidden entirely. The Art Tools advise leaving this at "None" for gameplay-critical splats (e.g. selection circles).
- `lodReduce` — the quality level at which the projector *may* be hidden if too many splats are active. The Art Tools note it is important to leave headroom here for gameplay-critical splats.

F.4.10 Rendering

Projecting onto geometry at runtime requires:

1. **Build the projector matrix.** Combine the bone world transform, the `offset` / `pitch` / `yaw` / `roll` orientation, and the perspective or orthographic projection parameters into a single world-to-projector-UV matrix.
2. **Find affected geometry.** Any surface triangle whose bounding volume intersects the projector frustum/box and that "accepts splats" (a per-mesh flag in the game engine) is a candidate.
3. **Project UVs.** Transform surface positions by the projector matrix to get U and V texture coordinates. Clip or fade fragments outside [0,1].
4. **Sample the material.** Evaluate the projected material at the computed UVs, multiply by the current `alpha` and attenuation factor.
5. **Composite.** Blend the result onto the surface according to the material's blend mode and the splat `layer` ordering.

Because projectors have no per-element simulation, they are significantly cheaper than particles or ribbons — the main cost is the per-fragment projection math and the overdraw from overlapping splats.

Appendix G: Ref Flags Corpus Analysis

The `flags` field in `Ref<T>` is absent in MD33 (`SmallRef<T>`, 8 bytes — see §C.1) and was introduced with the MD34 revision. Corpus analysis of all 52,579 MD34 files (29.8 million individual Refs) reveals the following:

Category	Count	Percentage
`flags == 0`	27,905,092	93.5%
`flags != 0`	1,926,387	6.5%

File-level patterns — the flags value is predominantly a per-file constant rather than a per-reference property:

Pattern	Files	% of files
All Refs have `flags == 0`	48,624	92.5%
All non-null Refs share the same non-zero value	438	0.8%
Mixed values across Refs in one file	3,517	6.7%

Value range: non-zero values almost exclusively use **bits 0–10** (maximum observed common value `0x7FF`). A handful of structural chunks (MODL, BONE) exhibit rare 15-bit values (`0x7FF5`, `0x7FF6`). Null Refs (`entries == 0`) always have `flags == 0`.

Most common non-zero values (top 10):

Value	Bit pattern	Occurrences
`0x000007FF`	bits 0–10 all set	252,217
`0x00000215`	`..001000010101`	125,840
`0x000001E8`	`..000111101000`	91,990
`0x000001D3`	`..000111010011`	82,561
`0x0000028C`	`..001010001100`	76,235
`0x000001E2`	`..000111100010`	75,513
`0x0000014C`	`..000101001100`	58,873
`0x0000021B`	`..001000011011`	52,785
`0x000002B5`	`..001010110101`	50,815
`0x000001F5`	`..000111110101`	49,488

Interpretation: the flags field appears to be **tool/pipeline metadata** written by specific versions of the Blizzard export toolchain. Evidence:

- The vast majority of files (92.5%) have all-zero flags, indicating the primary build pipeline does not set this field.
- In the 438 files where every non-null Ref shares one non-zero value, the value acts as a per-file constant (possibly a tool version hash or build identifier).
- Material-related chunks (LAYR, MAT_, DIS_) occasionally exhibit clean per-Ref bitmask values (`0x1`, `0x2`, `0x8`, `0x10`) whose meaning is undetermined.

Recommendation for parsers: the flags field does not affect model geometry, animation, or rendering. Parsers should read it for round-trip fidelity but may safely ignore its value for processing. Writers should preserve the original value on re-export, or write `0`.

Appendix H: Credits and Acknowledgements

This specification and related implementation notes build on years of community reverse engineering work. Major sources and contributors include:

H.1 `structures.xml` lineage (`Solstice245/m3studio`)

From `https://github.com/Solstice245/m3studio/blob/main/structures.xml` (including all people credited in that file's header comments), with additional credit to repository/file owners:

- Florian Köberle
- NiNtoxicated
- Leruster
- Witchsong
- Teal
- Blue Isle Studios
- Volcore
- Sixen
- der_Ton
- MrMoonKr
- Skizot
- Phygit
- ufoZ
- The SC2Mapster community
- CaptainD001
- Talv
- TangorCraft
- Solstice245 (John Wharton) — owner of `m3studio` and maintainer of this `structures.xml` lineage
- Renee

Also acknowledged:

- SC2Mapster / `m3addon` as the original source lineage referenced in the `structures.xml` header (`https://github.com/SC2Mapster/m3addon`)

H.2 `CaptainD001/M3_Import`

From `https://github.com/CaptainD001/M3_Import`:

- CaptainD001 (repository owner/author)
- Taylor Mouse
- Delphinium
- 一叶尽书繁华
- werd

H.3 HiveWorkshop M3 specification thread

From `https://www.hiveworkshop.com/threads/m3-specifications.243127/`:

- Fernando A. Sahmkow (BlinkBoy), also known as Blinkhawk and Blinkdude — credited as the writer of this M3 specification documentation on HiveWorkshop
- Dr Super Good (technical review/discussion)
- Koward (follow-up analysis and format notes)
- The broader HiveWorkshop reverse-engineering community